

CompTIA Linux+ Training (XK0-006)

Administer, secure, automate and troubleshoot Linux — and walk into the exam having already done the work.

Course Code: C1162

Trainer: Dr. Alfred Ang

Version v1 | 21 September 2026

Tertiary Infotech Academy Pte Ltd | UEN 201200696W



XK0-006 · V8 · 5 DOMAINS · 30 HANDS-ON LABS

Powered by Killercoda — every lab free in the browser

Course Administration

Welcome | facilitator | introductions | resources

Type it, break it, fix it — the exam rewards hands that have done the work.

Your Facilitator



Your Trainer

General Trainer template —
to be completed by the trainer

NAME — _____

TITLE / DESIGNATION — _____

QUALIFICATIONS — _____

AREAS OF EXPERTISE — _____

TRAINING & INDUSTRY EXPERIENCE — _____

CONTACT — _____

Your Facilitator



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE — Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

FOCUS — Linux, cloud, DevOps & cybersecurity — hands-on systems administration

DELIVERS — Courses on Linux, cloud administration, DevOps and software engineering

FOUNDER — Founder and lead instructor, Tertiary Infotech / Tertiary Courses

Let's Know Each Other

Your name and organisation / role.

Your current experience with Linux — beginner to advanced.

Which of the five exam domains matters most to your day job.

Your CompTIA Linux+ exam timeline.

Ground Rules

Phones on silent

Keep the room focused — check messages at breaks.

Participate actively

No question is too small; the discussion is the learning.

Do every lab yourself

Type the commands on Killercoda — muscle memory wins exams.

Mutual respect

Agree to disagree; critique ideas, not people.

Be punctual

Return from breaks on time so nobody re-teaches.

Full attendance

Attend all five days to get the most from the course.

Five-Day Learning Journey



One domain per day · Days 1–5: Domains 1–5 (Labs 1–30) · 9:30am–5:30pm

CompTIA Linux+ — Exam Domains

Domain 1 — System Management (23%)

Objectives 1.1–1.7 · Labs 1–7

Domain 2 — Services and User Management (20%)

Objectives 2.1–2.6 · Labs 8–13

Domain 3 — Security (18%)

Objectives 3.1–3.6 · Labs 14–19

Domain 4 — Automation, Orchestration, and Scripting (17%)

Objectives 4.1–4.5 · Labs 20–24

Domain 5 — Troubleshooting (22%)

Objectives 5.1–5.5 · Labs 25–30

How This Deck Is Organised

The course follows the five XK0-006 exam domains in blueprint order.

Inside each domain, content runs objective by objective (1.1 → 5.5).

Each objective: the concepts, condensed — then its hands-on lab, step by step.

Every lab maps to one exam objective, shown on its overview slide.

The deck closes with exam registration, the practice exam and next steps.

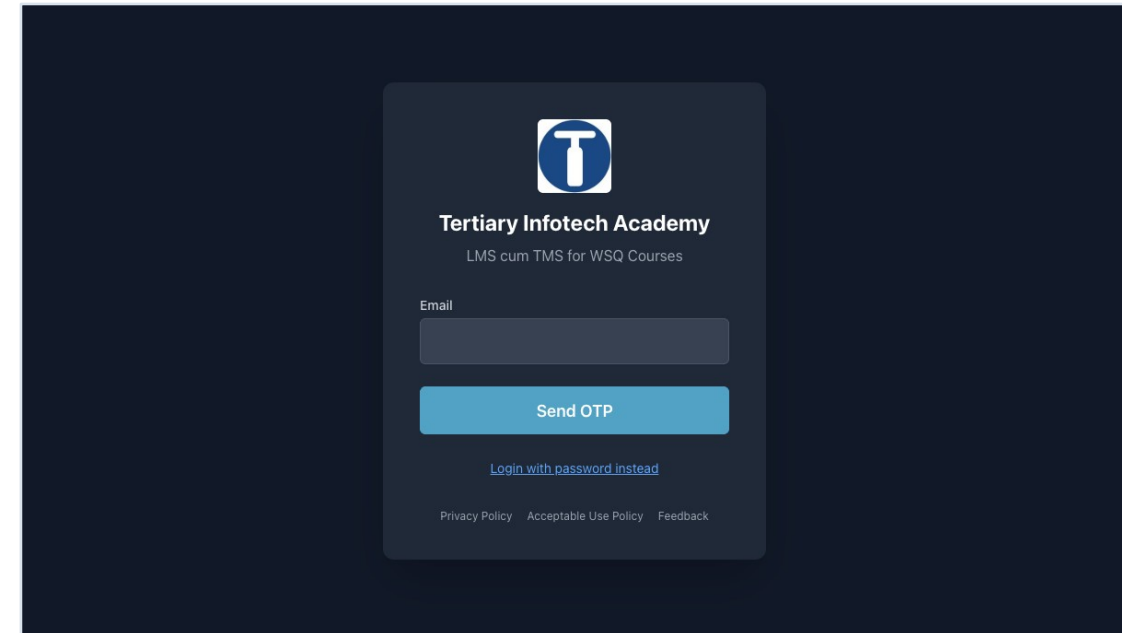
Download Course Material

1 — Go to the portal: lms-tms.tertiaryinfotech.com

2 — Log in with your registered email (Send OTP or password)

3 — Open your course: CompTIA Linux+ Training

4 — Download the slides, Learner Guide and Lesson Plan (open-book allowed)



Access the Hands-On Labs

All 30 labs are on the course GitHub repository — one folder per lab:

<https://github.com/tertiarycourses/C1162-CompTIA-LinuxPlus-Training>

Option A — git clone, then open labs/lab-XX-*/README.md

Option B — Code ► Download ZIP, unzip, open the labs/ folder

Every lab runs free in the browser on Killercode: <https://killercode.com/playgrounds/scenario/ubuntu>

On Linux, everything is a file.

“

Files, directories, devices, sockets and processes are all reachable through the filesystem — master the shell and you master the system.

COURSE MAXIM

System Management

23% of the exam | Objectives 1.1–1.7 | Labs 1–7

Type it, break it, fix it — the exam rewards hands that have done the work.

System Management — Objectives

1.1 · Linux basics

Identify boot process steps, kernel, filesystems, and architectures.

1.2 · Device management

Manage kernel modules, hardware components, and device utilities.

1.3 · Storage management

Configure LVM, RAID, partitions, and mounted storage.

1.4 · Network configuration

Set up hosts, DNS, interfaces, and network tools.

1.5 · Shell operations

Use navigation, editing, redirection, and environment variables.

1.6 · Backups and restores

Perform archiving, compression, and data recovery.

1.7 · Virtualization

Deploy hypervisors, create VMs, and manage disk images.

Key Concepts — System Management

The boot chain: **firmware/UEFI** → **GRUB2** → **kernel + initramfs** → **systemd**; the **FHS** defines where files live.

Storage stacks up: partitions → LVM (PV → VG → LV) → filesystem (ext4/xfs/btrfs), mounted via `/etc/fstab`.

The network stack is read top-down: NetworkManager/Netplan config → ip/ss → DNS resolution → diagnostics.

Shell operations (redirection, pipes, text tools), backup (tar/rsync/dd) and virtualization (QEMU/KVM) round out the domain.

1.1 — Linux basics

Identify boot process steps, kernel, filesystems, and architectures.

Exam objective — Explain basic Linux concepts

Hands-on — Lab 1

Type it, break it, fix it — the exam rewards hands that have done the work.

Directory

Understanding Linux Fundamentals

Directory	Description
bin	Binaries for all users
boot	Kernel, system map, boot files
dev	Device files
etc	Configuration files for the host
home	Home directories for users
lib	Necessary shared libraries/modules
media	Mount points for removable media
mnt	Temporary mount point for the systems administrator
opt	Third-party application software
proc	Kernel and process information
root	The root user's home directory
sbin	System binaries needed for boot
sys	Server-specific and service-related files
tmp	Temporary data
usr	Sharable, read-only data and programs; no host-specific data
var	Variable data, logs, Web, FTP, and so on

Where to Put Programs

The FHS prohibits programs from creating individual directories in the `/usr` directory tree.

`/usr` typically holds non-core programs, with core apps in `/bin` and `/sbin`.

Allowed subdirectories directly under `/usr`:

`bin`: Contains most user commands.

`lib`: Contains libraries.

`local`: Contains local/sharable programs (may be empty).

initrd and initramfs

Linux supports a wide range of hardware and uses a standard kernel image for compatibility.

The system must boot using a generic Linux kernel, but the location of the root filesystem needs to be known.

A temporary early user space filesystem (initrd or initramfs) is used to load necessary modules and hardware configuration.

initrd: An old method using a RAM-based image as the temporary root filesystem, requiring compiled-in drivers for the filesystem.

mkinitrd: Command used to create the initrd image.

initramfs: A more common method using a compressed filesystem archive, loaded into RAM as the temporary root, and does not require hard-coded drivers.

```
# sudo mkinitrd --preload=/boot/initrd.img-5.11.0-37-generic  
5.11.0-37-generic
```

Booting with UEFI

UEFI Secure Boot complicates Linux installation by requiring signed boot images, initially only by Microsoft.

Disabling UEFI Secure Boot in the system configuration simplifies installation.

UEFI is better than the older MBR method overall.

/boot/efi is used in dual-boot systems to store boot files for different OSes.

Some distributions may or may not have a /boot/efi directory.

A FAT32 EFI system partition contains .efi boot files.

```
/boot/efi/EFI/arch/vmlinuz-linux.efi
```


Booting via NFS

To boot a diskless workstation from NFS, follow these steps:

Install/enable DHCP and TFTP services on the server.

Configure TFTP to support PXE booting.

Copy the appropriate image to the TFTP boot directory.

Ensure the server's firewall allows TFTP traffic.

Configure the DHCP server for PXE booting.

Booting via ISO

Booting from an ISO file allows creating an on-disk rescue environment for recovery or password resets.

Steps to boot from an ISO:

Download a Linux distribution's installation ISO.

For a new install, create a rescue partition with an MS-DOS filesystem.

For an existing system, resize partitions to create space for a rescue partition.

Copy the ISO to the partition and note its filename.

GRUB2 Command Names

GRUB2 command names may or may not include the "2" in different distributions.

For example, Ubuntu omits the "2", while Fedora and CentOS include it.

To check the correct command, type the grub portion and press Tab twice to see available options.

```
root@tuxix:/boot# grub
grub                                grub-mklayout
grub-bios-setup                    grub-mknetdir
grub-editenv                       grub-mkpasswd-pbkdf2
grub-file                          grub-mkrelpath
grub-fstest                         grub-mkrescue
grub-glue-efi                      grub-mkstandalone
grub-install                       grub-mount
grub-kbdcomp                       grub-ntldr-img
grub-macbless                      grub-probe
grub-menulst2cfg                   grub-reboot
grub-mkconfig                      grub-render-label
grub-mkdevicemap                   grub-script-check
grub-mkfont                        grub-set-default
grub-mkimage                       grub-syslinux2cfg
```

Installing GRUB2

The first step in getting GRUB2 installed is to have the tool write itself to the master boot record as follows:

The command copies the boot sector image to disk and the remaining files to /boot.

If the bootable partition is not /boot, you can override this default setting (e.g., /mnt/tmpboot).

In Example 1-2, GRUB2 files are installed to /mnt/tmpboot, with directories created for the boot manager.

The GRUB2 image and modules are located in the i386-pc directory.

This method is commonly used for creating boot disks on an existing system with a temporarily mounted disk image.

```
[root@localhost ~]# grub2-install --boot-  
directory=/mnt/tmpboot/dev/sda  
Installing for i386-pc platform.  
Installation finished. No error reported.  
[root@localhost ~]# tree -d /mnt/tmpboot/  
/mnt/tmpboot/  
|-- grub2  
|  -- fonts  
|  -- i386-pc  
|-- locale  
4 directories
```

Using the GRUB2 Command Line

The GRUB2 command line can be accessed during boot by pressing **c** to open a simple command prompt (**grub>**).

If you can't boot normally, use the GRUB2 command line to inspect the system and boot a kernel.

The **ls** command lists available devices in GRUB2.

GRUB2 uses a similar partition naming syntax to Linux.

Disks are numbered starting from 0, while partitions are numbered from 1.

The first partition on the first disk is **hd0,gpt1**, corresponding to **sda1** in Linux.

Possible files are

[Click here to view code image](#)

```
vmlinuz-0-rescue-1bc6a0e5dd394f63b228657b634ba252  
vmlinuz-3.10.0-862.14.4.el7.x86_64
```

Configuring GRUB2

GRUB2's configuration file is `grub.cfg` (or `grub2.cfg` on some distributions).

In GRUB2, the configuration is generated by scripts, making it more complex than GRUB's simpler `grub.conf`.

The `grub-mkconfig` command generates `grub.cfg`, and the output can be displayed on the screen or saved to a specified file.

`grub-mkconfig` takes inputs from `/etc/default/grub` and `/etc/grub.d/`.

`/etc/default/grub` sets various default configurations.

`/etc/grub.d/` contains a series of shell scripts that contribute to the configuration.

```
# grub2-mkconfig -o /boot/grub/grub.cfg
Sourcing file '/etc/default/grub'
Sourcing file '/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-5.4.0-91-generic
Found initrd image: /boot/initrd.img-5.4.0-91-generic
Found linux image: /boot/vmlinuz-5.4.0-80-generic
Found initrd image: /boot/initrd.img-5.4.0-80-generic
Adding boot menu entry for UEFI Firmware Settings
done
```

Common Commands at Boot Time

Boot parameters allow you to modify kernel options at the start of the boot process.

Common boot options include:

`vga`: Set framebuffer resolution (e.g., `vga=2` for certain laptops).

`apm`: Disable APM (Automated Power Management) with `apm=off` or `noapm`.

`init`: Specifies the init process to run first (may not work with all systems).

`panic=#seconds`: Automatically reboots after a panic error, with a set delay.

Kernel Panic Causes

Kernel panic can be caused by various factors, including:

Corrupted hard disk sectors

Incomplete or corrupted kernel modules

File or directory permission issues

Invalid or nonexistent memory address references

Memory address conflicts with key codes

Device Types in /dev

The /dev directory contains special files that serve as access points for physical and pseudo devices.

Block devices (e.g., disks, optical drives) handle entire blocks of data at a time.

Character devices (e.g., keyboards, scanners) handle data one character at a time.

Subdirectories in /dev contain files related to a specific device type, like /dev/input for event-generating devices.

Link files are symbolic links pointing to other files, such as /dev/cdrom pointing to /dev/sr0.

Key special files in /dev:

Installing Software from Source

Advantages of installing programs from source code:

Allows inspection of code for bugs, security flaws, and optimization.

Available earlier than binary packages.

Compilation optimizes the program for the specific system.

Provides access to cutting-edge or beta software.

Familiarizes you with the installation process.

The Makefile

Purpose: Contains instructions and compiling parameters to install software.

Usage: After the configure script updates the makefile, the make and make install commands are used to compile and install the software.

Make Command: Determines which parts of a program need recompilation and issues commands to do so.

Sections in a Makefile:

Platform: Specifies the system's platform.

Debug: Defines error handling.

```
install-prefix = .  
bin_dir = $(install-prefix)/bin  
uparm_dir = $(install-prefix)/lib/uparm  
include_dir = $(install-prefix)/include
```

Storage Concepts

Install the Device: Set up the physical device.

Partition the Device: Divide the device into one or more partitions.

Create a Filesystem: Apply a filesystem to the partition(s).

Choose a Mount Point: Select or create a directory to mount the filesystem.

Mount the Filesystem: Attach the filesystem to the directory.

Configure Auto-Mount: Set the filesystem to mount automatically on boot.

The Proc Filesystem

Unix Philosophy: "Everything is a file," meaning hardware and system/process state info is exposed on the filesystem.

/proc Directory: Contains information about each process and system hardware, including configurations like iports, dma, and interrupts.

procfs: A pseudo-filesystem that exposes kernel information in a well-defined format without existing on real hardware.

Commands: Essential for resolving conflicts and viewing information directly from the /proc directory.

```
cat /proc/interrupts  
cat /proc/iports  
cat /proc/dma  
cat /proc/usb  
cat /proc/pci
```

Displaying DMI Table Information

dmidecode Command: Queries the DMI (Desktop Management Interface) table, which holds information from the system BIOS.

Purpose: Allows processes and commands to access hardware data without direct hardware queries.

Format: Information returned follows a specific format, typically reflecting the structure of the DMI tables.

dmidecode Output Format: The first three items on the line include a unique identifier (handle), the type of information, and the size of the entry.

Viewing Output: To view the typically long output (100+ lines), it's recommended to use a pager like the less command for easier navigation.

Using less Pager: Use less to view dmidecode output, allowing easy scrolling and searching.

```
Handle 0x0005, DMI type 16, 23 bytes
Physical Memory Array
    Location: System Board Or Motherboard
    Use: System Memory
    Error Correction Type: None
    Maximum Capacity: 4 GB
```

Linux basics — More Key Topics

Filesystem Hierarchy Standard

The filesystem starts with the root ("/"), and files are accessible via unique paths. Unix and Linux have conventions for file ...

Basic Boot Process

Booting a Linux system involves a series of events leading to the login prompt. Key topics in the boot process:

What Is the Boot Process?

The Linux boot process transitions from hardware power-on to a functional operating system. There are multiple initialization ...

System Boot Options

There are various ways to boot a system. Some methods require software installation and configuration on the local disk.

Booting via PXE

PXE requires a PXE-capable NIC on the client and a DHCP server that supports PXE on the server. The PXE server is more complex, ...

Boot Loaders and Files

BIOS provides basic functionality to boot a computer, allowing communication with peripherals and loading applications. The BIOS ...

Linux basics — More Key Topics

GRUB Legacy

Early Linux boot managers, like LOADLIN.EXE and LILO, were simple but inflexible. LILO required rewriting to the disk for changes ...

GRUB2

GRUB2 is the current version of GRUB, used as the default since around 2010. GRUB2 was a core rewrite for improved stability and ...

Changes Made for GRUB2

GRUB2 uses grub.cfg instead of menu.lst or grub.conf for configuration. The grub.cfg file is generated by the grub-mkconfig ...

Demystifying Kernel Images

vmlinux is an uncompressed, statically linked kernel image used in earlier kernel distributions, limited to 512KB to fit in the ...

When Kernels Panic

The Linux kernel is complex and may encounter issues such as hardware, driver, or software problems. When critical errors occur, ...

Identifying a Kernel Panic

Kernel panics are often identified by a panic message with trace information about the running process. Common symptoms include ...

Lab 1 — Boot Process & Filesystem Hierarchy

Inspect a running Linux system's boot configuration, kernel command line, initramfs, and the standard Filesystem Hierarchy to understand how GRUB hands off to the ...

You'll build — Identify the distro and kernel, inspect GRUB, the kernel cmdline, and initramfs, then tour the FHS and boot-related ...

Key commands — /etc/os-release, GRUB2, /proc/cmdline, initramfs, lsinitramfs, FHS

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-01-*/README.md on the course GitHub repo

Boot Process & Filesystem Hierarchy

Step 1 — Identify distribution and architecture

```
$ cat /etc/os-release
```

Step 2 — Inspect GRUB configuration

```
$ cat /etc/default/grub 2>/dev/null || true
```

Step 3 — Inspect the kernel command line

```
$ cat /proc/cmdline
```

Step 4 — Explore the initramfs

```
$ ls /boot/initrd* 2>/dev/null || echo "no initrd image present"
```

Boot Process & Filesystem Hierarchy

Step 5 — Tour the Filesystem Hierarchy (FHS)

```
$ for d in /bin /boot /dev /etc /home /lib /proc /sbin /sys /tmp /usr /var /run /o
```

Step 6 — Walk through key boot-related dirs

```
$ ls /etc/systemd/system/ | head
```

✅ **Verify** — You can identify the running distro/kernel/arch and explain how GRUB, the kernel cmdline, and initramfs bring ...

1.2 — Device management

Manage kernel modules, hardware components, and device utilities.

Exam objective — Summarize Linux device management concepts and tools

Hands-on — Lab 2

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 2 — Kernel Modules & Device Management

List, inspect, load, and unload kernel modules and use the standard hardware-enumeration tools to understand modprobe vs insmod, module dependencies, and reading ...

You'll build — Install discovery tools, inspect and load/unload a module, examine module dependencies, read the kernel ring buffer, ...

Key commands — lsmod, modinfo, modprobe, insmod/rmmod, depmod, dmesg

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-02-*/README.md on the course GitHub repo

Kernel Modules & Device Management

Step 1 — Install hardware inspection tools

```
$ apt-get update -qq
```

Step 2 — List loaded modules and inspect one

```
$ lsmod | head -20
```

Step 3 — Load and unload a module

```
$ modprobe dummy 2>&1 || modprobe loop 2>&1 || echo "module load blocked"
```

Step 4 — Module dependencies and depmod

```
$ cat /lib/modules/$(uname -r)/modules.dep 2>/dev/null | head -5
```

Kernel Modules & Device Management

Step 5 — Read the kernel ring buffer

```
$ dmesg | tail -20 2>/dev/null || journalctl -k --no-pager | tail -20
```

Step 6 — Enumerate hardware

```
$ lscpu
```

Step 7 — initramfs builders overview

```
$ which dracut 2>/dev/null || echo "dracut on RHEL/Fedora/SUSE"
```

✅ **Verify** — You can pick the right ls* tool for a hardware question and explain insmod vs modprobe vs depmod.

1.3 — Storage management

Configure LVM, RAID, partitions, and mounted storage.

Exam objective — Given a scenario, manage storage in a Linux system

Hands-on — Lab 3

Type it, break it, fix it — the exam rewards hands that have done the work.

Viewing SCSI Device Information

After creating a filesystem, use the `lsscsi` command to check which SCSI-related devices are recognized on the system.

The `lsscsi` output includes details such as host, controller, target, and logical unit number (LUN) for each device.

For NVMe devices, the device format differs (e.g., `/dev/nvme1`), but Linux systems often normalize it to the `/dev/sdX` format.

Common `lsscsi` options:

`-b (--brief)`: Provides concise information on one line.

`-l (--long)`: Gives additional details on a second line.

The lsblk Command

If you forget which device files are available, use the `lsblk` command to display block devices and their partitions.

The `/proc/partitions` file also provides partition information.

The `/sys/block` directory contains detailed information about block devices, including partitions, with additional files providing more data on each device.

```
# ls /sys/block/vda
alignment_offset  device          mq              ro              subsys
bdi               discard_alignment power          serial          trace
cache_type       ext_range       queue          size            uevent
capability        holders         range          slaves          vda1
dev               inflight        removable      stat            vda2
```

Performing Disk Partitioning

`fdisk` is used to manage MBR partition tables and changes are not written until explicitly saved.

To start partitioning, run `fdisk /dev/sdb` (specifying the device).

Step-by-step example:

Launch `fdisk` and check the device with the `p` command.

Create a primary partition (n), choose partition type (p), and specify partition size (e.g., +1G).

Verify partition creation with `p`.

Using Logical Volume Manager (LVM)

LVM (Logical Volume Manager) allows combining multiple physical disks into a pool (Volume Group, VG) and dividing it into Logical Volumes (LVs) for easier ...

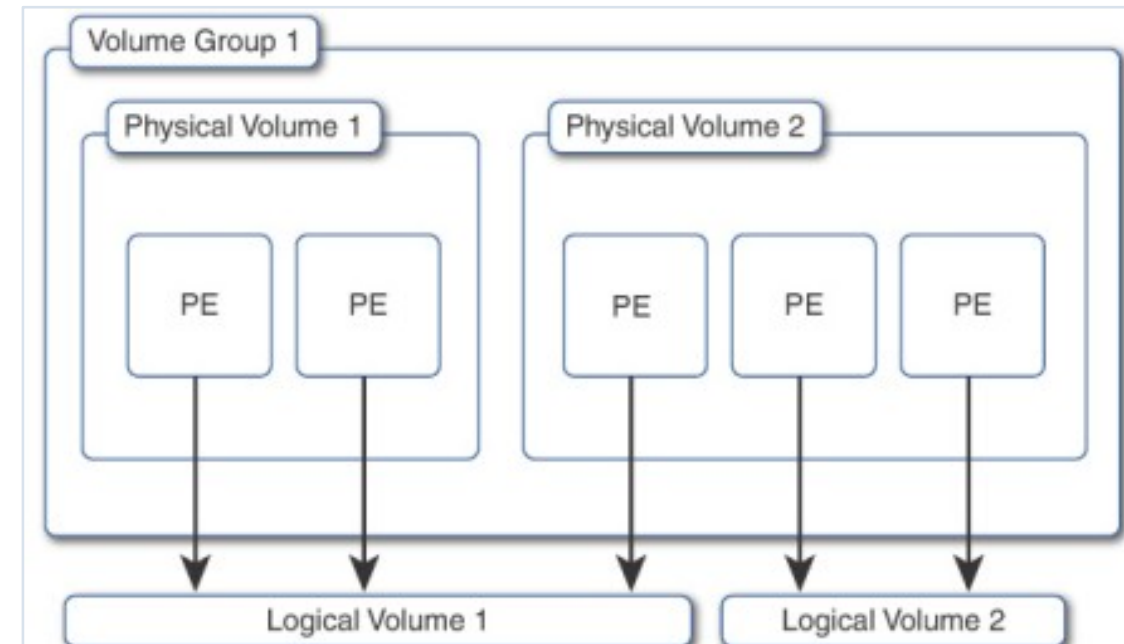
Physical Volumes (PVs): Base storage units, typically hard disk partitions or SAN storage.

Volume Groups (VGs): Pools formed by combining one or more PVs.

Logical Volumes (LVs): Divisions of VGs where filesystems are created for use by the operating system.

Flexible storage management: LVs can grow or shrink based on available space in VGs.

Easy to add more storage by adding new PVs to the VG.



Managing LVM

Creating an LVM Device:

Use `pvccreate` to convert physical devices into physical volumes (PVs).

Use `vgcreate` to combine PVs into a volume group (VG).

Use `lvcreate` to create a logical volume (LV) from the VG space.

Commands for Managing LVM:

`pvs`: Displays physical volumes.

Filesystem Management

Filesystem Checker (fsck):

Used to check and repair filesystems.

Specific tools: e2fsck for ext filesystems, dosfsck for DOS/FAT, fsck.btrfs for Btrfs, and xfs-repair for XFS.

Run fsck on unmounted or read-only filesystems to avoid corruption.

fsck can be auto-executed at boot if the filesystem is unclean.

Ext Filesystem Tools (ext2, 3, 4):

Storage Area Networks/NetworkAttached Storage

NAS (Network-Attached Storage):

Simple setup, relatively inexpensive.

Uses TCP/IP network; provides file-level storage.

Suitable for smaller environments with fewer users.

Volumes are mounted over the network.

May have single points of failure in lower-end devices.

Storage management — More Key Topics

Determining Storage Hardware

To utilize hardware effectively, you first need to identify what hardware is available. Sometimes, hardware may not appear as ...

Viewing Partition and Filesystem Device File ...

After creating a filesystem, it's typically easy to remember the device file used to access it. Upcoming sections will cover ...

The blkid Command

When mounting a filesystem, device filenames (e.g., /dev/sda1) can change if hardware is added or removed. To avoid this, you can ...

The (Non-Linux) fcstat Command

The fcstat command is mentioned in the Linux+ exam objectives but is rarely used in practice. It's a command from AIX, not ...

fdisk

Disk partitioning divides a storage device into isolated sections called partitions, each with its own filesystem and data. It's ...

Partitioning with parted

MBR partition table has limitations: 2TB partition size limit, only 4 partitions (with workarounds using extended/logical ...

Storage management — More Key Topics

partprobe

partprobe is a useful tool for notifying the kernel of partition table changes without needing a reboot. It helps confirm the ...

Creating and Inspecting a Software RAID

To create a software RAID device, use the mdadm command. Example: #
mdadm -C /dev/md0 -l 1 -n 2 /dev/sdb /dev/sdc

Inspecting a Software RAID Device

To view RAID device information, check /proc/mdstat: Example: # more
/proc/mdstat

Understanding Multipath

Network Storage and Multipath: Remote storage devices depend on the network, which can be a point of failure.

Manually Mounting Filesystems

If a filesystem isn't in /etc/fstab, you can mount it manually using the following syntax: # mount -t type -o option device ...

Unmounting Filesystems

The umount command works similarly to mount and can unmount by label, mount point, or device. Example to unmount /mnt/cdrom: # ...

Lab 3 — Storage with LVM, Partitions & Filesystems

Build a complete LVM stack on loopback-backed disks, format with ext4/xfs/btrfs, mount with hardening options, persist in fstab, and grow a filesystem online.

You'll build — Create loopback disks, partition, build PVs/VG/LVs, make filesystems, mount with hardening options into fstab, then ...

Key commands — losetup, parted/GPT, pvcreate/vgcreate/lvcreate, mkfs.ext4/xfs/btrfs, blkid/UUID, /etc/fstab

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-03-*/README.md on the course GitHub repo

Storage with LVM, Partitions & Filesystems

Step 1 — Install tooling and create loopback disks

```
$ apt-get install -y lvm2 xfsprogs btrfs-progs parted util-linux >/dev/null
```

Step 2 — Partition the first loopback disk

```
$ L00P1=$(losetup -j /lab/disks/disk1.img | cut -d: -f1)
```

Step 3 — Create PVs, a VG, and LVs

```
$ L00P2=$(losetup -j /lab/disks/disk2.img | cut -d: -f1)
```

Step 4 — Make filesystems

```
$ mkfs.ext4 -F /dev/labvg/lv_ext4
```

Storage with LVM, Partitions & Filesystems

Step 5 — Mount with hardening options and persist in fstab

```
$ mkdir -p /mnt/ext4 /mnt/xfs /mnt/btrfs
```

Step 6 — Grow a logical volume and its filesystem online

```
$ lvextend -L +100M /dev/labvg/lv_ext4
```

✅ **Verify** — You can go from raw block device to a mounted, UUID-referenced, online-expandable filesystem; note XFS grows ...

1.4 — Network configuration

Set up hosts, DNS, interfaces, and network tools.

Exam objective — Given a scenario, manage network services and configurations

Hands-on — Lab 4

Type it, break it, fix it — the exam rewards hands that have done the work.

The ip Command

The ip command is designed to replace many older network configuration commands (e.g., ifconfig).

The ip addr show command displays network configuration data, similar to ifconfig.

The ip command is used for tasks like managing IP addressing, linking network devices, handling routing tables, and configuring IP tunnels.

It also manages TCP metric info.

The ip command supports abbreviations for convenience.

The ip command allows for shortened versions to save time and keystrokes, providing the same information.

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
   inet6 ::1/128 scope host
   valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pf
   state UP qlen 1000
   link/ether 08:00:27:08:ea:ff brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.22/24 brd 192.168.1.255 scope global eth0
   inet6 fe80::a00:27ff:fe08:eaff/64 scope link
   valid_lft forever preferred_lft forever
```

The ifconfig Command

ifconfig: Used to view or set IP addresses for a host (excluding the default gateway).

You can also use it to bring up or activate network interfaces.

Example: To configure eth0 with IP 192.168.33.2, network mask 255.255.255.0, and activate the interface, use the appropriate command.

ifconfig displays activated interfaces.

Use the -a switch to show all interfaces, including those that are down or not activated.

```
# ifconfig eth0 192.168.33.2 netmask 255.255.255.0 up
```

Configuring via ifcfg-* files and network-scripts

ifcfg- files* are used to configure network interfaces.

On Red Hat systems, these files are located in the /etc/sysconfig/network-scripts directory.

For example, to configure eth0, modify the ifcfg-eth0 file, which can also be edited with system-config-network.

Use a command to display the ifcfg-eth0 file.

A static IPv4 configuration will show output specifying an IP address, network mask, and gateway.

This configuration ensures that the system uses a fixed IP address instead of obtaining one dynamically via DHCP.

```
IPV6INIT=yes
IPV6ADDR=3FFE:F200:0134:AB00:0143:1111:8901:0002
IPV6_DEFAULTGW=3FFE:F200:0134:AB00:0143:1111:8901:0001
```


On Debian-based Systems

Debian uses a single configuration file, `/etc/network/interfaces`, for all network interfaces.

Unlike Red Hat, Debian does not include the `system-config-networking` utility by default.

Debian includes the `netcardconfig` program in some distributions, which performs similar tasks as `system-config-networking`.

The `/etc/network/interfaces` file in Debian is used to configure network interfaces.

The contents of the file define network settings, such as static IP addresses or DHCP configurations.

Example output from the file can be referenced for understanding the structure of the configuration.

```
# /etc/network/interfaces -- configuration file for ifup(8),
# The loopback interface
# automatically added when upgrading
auto lo eth0
iface lo inet loopback
iface eth0 inet static
    address 192.168.15.5
    netmask 255.255.255.0
    network 192.168.15.0
    broadcast 192.168.15.255
    gateway 192.168.15.2
```

The arp Command

Hostnames need to be translated into IP addresses for communication, typically by a DNS server.

The OSI model defines seven layers of networking, with Layer 2 handling communication via MAC addresses.

To communicate, systems need both the IP and MAC addresses of the other system.

When a system needs a MAC address for a remote IP, it sends a broadcast request. The machine with the matching IP responds with its MAC address.

The MAC-IP pairing is stored in the system's ARP table.

The arp command is used to view or modify the ARP table.

```
# arp
Address          HWtype  HWaddress      Flags Mask    Iface
192.168.1.11     ether   30:3a:64:44:a5:02    C           eth0
```

The route Command

The `route` command can help defend against attacks by misrouting traffic.

During a denial-of-service attack, you can add a route to route responses to the attacker's IP address through the loopback address.

This effectively misroutes the traffic, mitigating the attack.

Redirecting traffic to the loopback address can cause the attacking host to time out quickly, temporarily stopping a denial-of-service attack.

This isn't a permanent solution, but it works for local network tests.

To enable IP forwarding on a host, you can echo a 1 into the `/proc/sys/net/ipv4/ip-forward` file. This allows traffic to be forwarded between different ...

```
# route add 10.1.1.69 lo
```

Configuring Name Resolution

Local system name configuration involves several files:

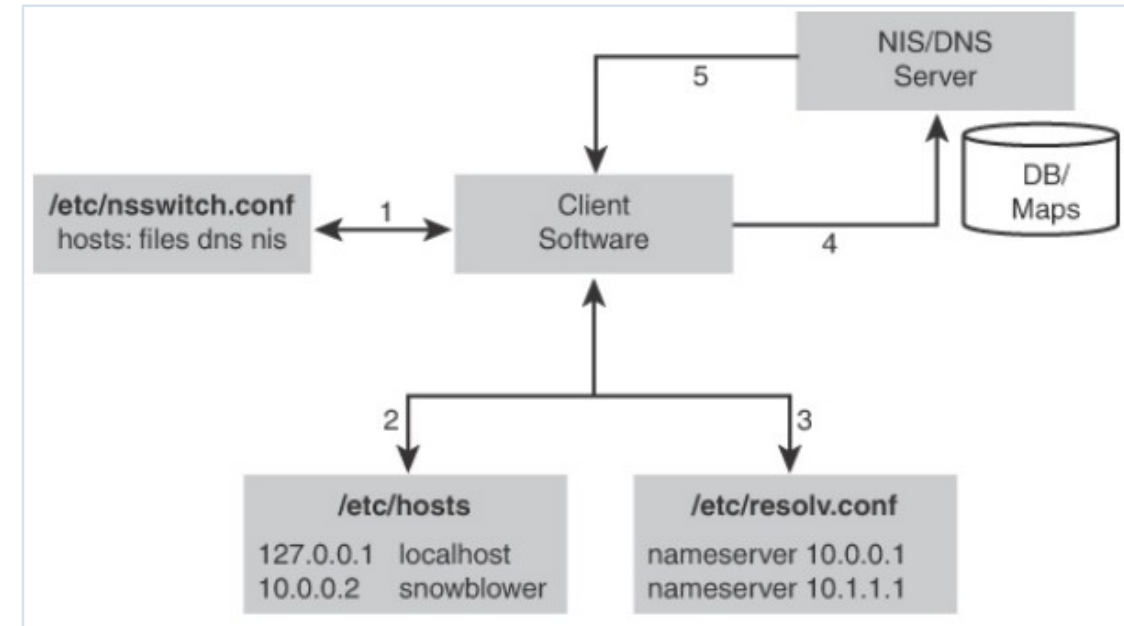
/etc/hosts

/etc/resolv.conf

/etc/nsswitch.conf

These files configure local and remote name resolution and are distribution neutral, relevant for the Linux+ exam.

They work together to resolve the name for a host that client software needs to connect to.



Controlling Resolution

`resolvectl` is a useful tool for resolving and enumerating hostnames, domains, and IP addresses.

It interacts with the `systemd-resolved` service to resolve both IP addresses to hostnames and domain names to destination addresses.

To resolve a hostname locally, use the `query` subcommand followed by the hostname to resolve.

`resolvectl` shows its real power when querying remote domains.

It provides authoritative information about the host, helping with detailed DNS resolution.

`resolvectl` can be used to query a domain's configuration and find specific service records.

```
# resolvectl statistics
DNSSEC supported by current servers: no

Transactions
Current Transactions: 0
Total Transactions: 217

Cache
Current Cache Size: 0
Cache Hits: 26
Cache Misses: 426

DNSSEC Verdicts
Secure: 0
Insecure: 0
Bogus: 0
Indeterminate: 0
```

The ping Command

Ping command: Uses ICMP ECHO_REQUEST and ECHO_RESPONSE packets to check if a host is functioning or reachable.

Determine if a host is alive

Check if a network is reachable

Verify if a gateway is functioning

Traceroute: More useful for determining the route taken by packets.

This returns output similar to the following:

```
PING 192.168.1.1 (192.168.1.1) from 192.168.1.73 : 56(84) bytes of data.  
64 bytes from 192.168.1.1: icmp_seq=1 ttl=150 time=9.23 ms  
64 bytes from 192.168.1.1: icmp_seq=2 ttl=150 time=0.774 ms  
64 bytes from 192.168.1.1: icmp_seq=3 ttl=150 time=0.715 ms  
64 bytes from 192.168.1.1: icmp_seq=4 ttl=150 time=11.3 ms
```

Using netstat

netstat Command Overview: Useful for network statistics, open sockets, and troubleshooting.

Common netstat Options:

-t: Displays TCP statistics.

-r: Shows the routing table.

-a: Lists all open sockets on all interfaces.

-c: Provides a refreshed view of statistics every second.

```
1 Active Internet connections (servers and established)
2 Proto Recv-Q Send-Q Local Address Foreign Address
3 tcp 0 0 *:pop3s *: LISTEN
4 tcp 0 0 *:netbios-ssn *: LISTEN
5 tcp 0 0 *:sunrpc *: LISTEN
6 tcp 0 0 192.168.15.5:domain *: LISTEN
7 tcp 0 0 *:ssh *: LISTEN
8 tcp 0 0 *:smtp *: LISTEN
9 tcp 0 0 *:7741 *: LISTEN
10 tcp 0 1 192.168.15.5:36651 206.235.223.112:smtp SYN
11 tcp 0 48 192.168.15.5:ssh 192.168.15.1:4417 ESTABL
12 tcp 0 0 192.168.15.5:36619 www.certmag.com:www ESTABL
13 0 1 192.168.15.5:36657 206.235.223.112:pop3 SYN
14 tcp 0 1 192.168.15.5:36653 206.235.223.112:pop3 SYN
15 tcp 0 0 192.168.15.5:36594 moviesunlim:www ESTABL
16 tcp 0 0 192.168.15.5:36595 moviesunlim:www ESTABL
```

Swimming with Wireshark

Wireshark Overview: A powerful network sniffer tool for viewing and troubleshooting network traffic.

Installation: Likely not pre-installed; use tools like yum or apt to install it.

Root Access Requirement: Network sniffing requires root access; keep this in mind for production/Internet-facing systems.

Wireshark Tools: Available in both GUI (graphical) and TUI (text-based) versions.

GUI Launch: Start the GUI by executing the wireshark command.

TUI-based Wireshark: Use the tshark command to access the text-based interface of Wireshark.

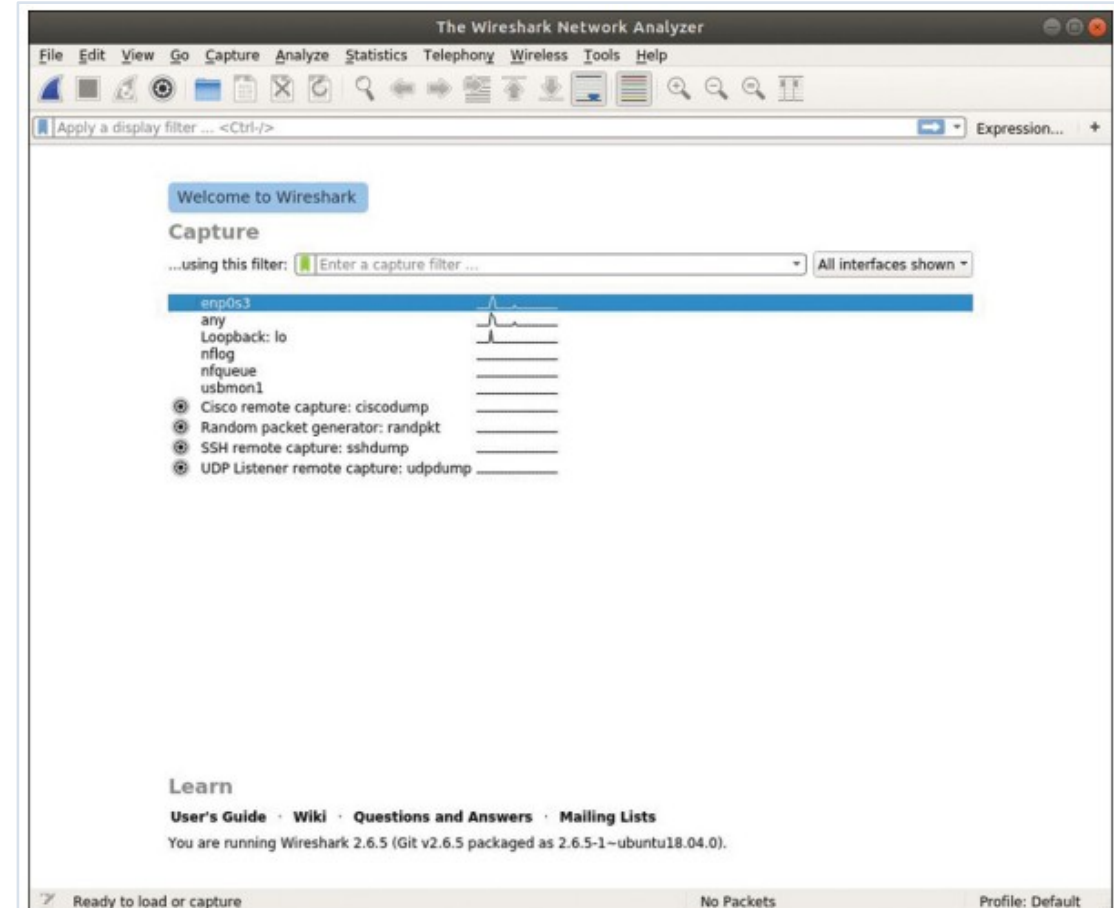


Figure 5-6 The Wireshark GUI

The tcpdump Command

Tcpdump command: A packet sniffer tool used to view network traffic, including traffic not related to the local machine.

Default behavior: Displays all network traffic until terminated, which can result in a large amount of data.

Limiting output: Use the `-c` option to specify the number of packets to capture.

Capture based on criteria: The tcpdump command can filter captured packets.

Interface-specific capture: Use the `-i` option to capture packets from a specific network interface.

Limit by protocol: Use the protocol name as an argument to capture packets for a specific protocol (e.g., TCP, UDP).

```
# tcpdump -i eth0 tcp
```

Using curl and wget

wget and curl: Used to script file downloads from remote sources as part of updates or scripts.

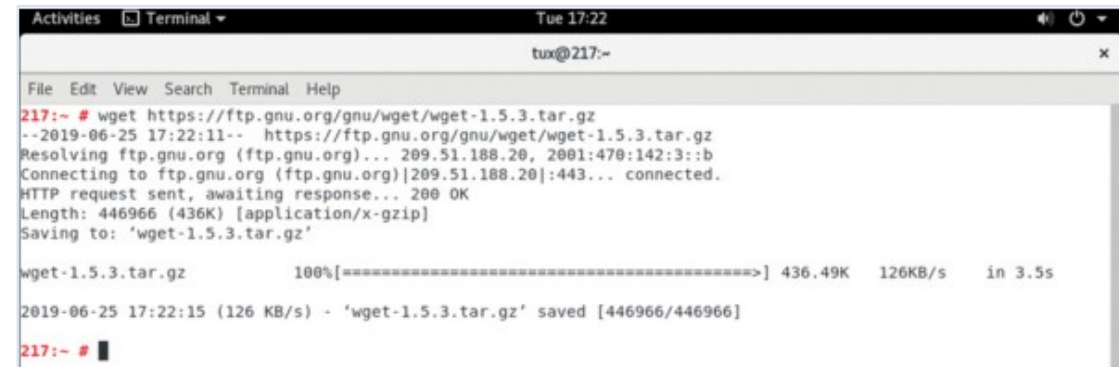
Example: You can use these commands to download files without requiring further interaction.

wget command: Used to download files, including the latest version of source code for tools like wget itself.

Example: The command downloads the latest version of the wget source code.

curl vs wget: Both are used for downloading files, but curl is often used for downloading and executing scripts, especially in automated installations for ...

Example: The same wget source code file can be downloaded using curl.

A terminal window titled 'Terminal' with a menu bar (File, Edit, View, Search, Terminal, Help) and a status bar (Tue 17:22, tux@217:~). The terminal shows the execution of the command 'wget https://ftp.gnu.org/gnu/wget/wget-1.5.3.tar.gz'. The output includes the URL, resolution of ftp.gnu.org, connection details, and a progress bar showing 100% completion. The file is saved as 'wget-1.5.3.tar.gz' with a size of 436.49K in 3.5s.

```
217:~ # wget https://ftp.gnu.org/gnu/wget/wget-1.5.3.tar.gz
--2019-06-25 17:22:11-- https://ftp.gnu.org/gnu/wget/wget-1.5.3.tar.gz
Resolving ftp.gnu.org (ftp.gnu.org)... 209.51.188.20, 2001:470:142:3::b
Connecting to ftp.gnu.org (ftp.gnu.org)|209.51.188.20|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 446966 (436K) [application/x-gzip]
Saving to: 'wget-1.5.3.tar.gz'

wget-1.5.3.tar.gz      100%[=====>] 436.49K  126KB/s   in 3.5s

2019-06-25 17:22:15 (126 KB/s) - 'wget-1.5.3.tar.gz' saved [446966/446966]

217:~ #
```

The nc Command

nc (netcat) command: A versatile tool for working with TCP and UDP, including opening TCP connections, sending UDP packets, port scanning, and supporting ...

Port scanning and testing: Useful for checking if a specific port is blocked by a firewall or for testing services by listening on arbitrary ports.

Error handling: Unlike telnet, nc separates error messages from standard output, making it easier to handle in scripts.

nc command usage: On a local system, use the nc command to listen for connections on a specific port.

Remote connection: On a remote system, use the nc command to connect to the local system by specifying its hostname or IP address.

Blank line: After running the nc command locally, you should see a blank line indicating it's ready to accept a connection.

```
# nc -z localhost 1000-4000
Connection to localhost 3260 port [tcp/iscsi-target] succeeded
Connection to localhost 3333 port [tcp/dec-notes] succeeded!
```

SSH Components

SSH includes several components:

ssh: For remote shell sessions, replacing telnet, rsh, and rexec.

scp: For secure file transfers, replacing rcp.

sshd: The SSH daemon.

ssh-agent: A wrapper that handles authentication for the user's session.

ssh-add: Loads the user's keys into the ssh-agent for authentication.

```
The authenticity of host 'mp3server (192.168.33.44)' can't
established.
RSA key fingerprint is 73:4f:fa:b0:42:a4:3a:a8:64:2c:ad:20
d:b1: 21:e0.
Are you sure you want to continue connecting (yes/no)?
```

The sudoedit Command

The sudoedit command is a safer alternative to using vi/vim when granting users access to edit configuration files with root privileges. Unlike vi/vim, ...

The sudoedit command allows a user to edit files with root access using their preferred editor, without needing to log in as the root user. When a user ...

To configure sudoedit, add the following line to the /etc/sudoers file:

Configure the newsudo group in /etc/sudoers to have the users you want to use sudoedit, and then all they need to do is run the command:

To grant a user (e.g., ross) the ability to run all commands as the root user, you would add the following line to the /etc/sudoers file:

To limit a user to a specific command, such as the /etc/rc.d/init.d/network command, add the following line:

```
ross ALL=(ALL) /etc/rc.d/init.d/network
```

The su Command

To gain access to another user account with the **su** command, use the following syntax:

For example, to switch to the root account, execute the following command:

Using the **-l** option (or just **-**), you can provide a login shell for the root user. A login shell ensures you get the full user environment, including ...

If you don't specify an account name with the **su** command, it defaults to switching to the root account. Therefore, the following commands are equivalent ...

```
su - root
```

```
su -l root
```

```
# su - root
# su -l root
```

Network configuration — More Key Topics

Interface Management

Network addresses are typically auto-configured during Linux installation. Deviating from the default configuration requires ...

iproute2

The ip command from the iproute2 toolset is designed to replace the ifconfig and route commands from the older net-tools package. ...

Interface Management Commands

The two main iproute2 tools for managing IP information are ip and ss. The ip command is the most commonly used and is now the ...

Examples of the ip Command

The ip command is versatile and can be used for various tasks, which are important for answering ip-related questions on the ...

The ss Command

The ss command displays socket information. Running ss without options lists all open sockets.

NetworkManager

NetworkManager simplifies networking on Linux by automatically managing primary network connections (Ethernet, WiFi, Mobile ...

Network configuration — More Key Topics

The nmtui Command

nmtui is an interactive tool that comes with NetworkManager. It is used for configuring network devices.

The nmcli Command

nmcli is a command-line tool for controlling NetworkManager. It's used to manage interface statuses and is ideal for scripts or ...

The net-tools Suite

net-tools has been widely used but is being phased out since 2009. Linux+ exam requires knowledge of the following net-tools ...

The hostname Command

The hostname command is used to view and set the system's hostname and domain name. The hostname can be set using this command or ...

Name Resolution

Resolving names, such as local and remote hostnames, requires a system to translate human-friendly names into numerical IP ...

Setting Hostnames on systemd Systems

The hostnamectl command is used for managing hostnames in systemd-based systems. There are three types of hostnames:

Lab 4 — Network Configuration

Inspect interfaces, addresses, and routes with ip; walk the name-resolution stack; validate a netplan YAML; and use the standard diagnostic toolbox to read a Linux ...

You'll build — Install network tools, inspect interfaces/routes, walk the resolution stack, run DNS diagnostics, validate netplan, ...

Key commands — ip -br link/address/route, /etc/nsswitch.conf, /etc/resolv.conf, getent, dig/nslookup, netplan generate

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-04-*/README.md on the course GitHub repo

Network Configuration

Step 1 — Install tools

```
$ apt-get update -qq
```

Step 2 — Interfaces, addresses, routes

```
$ ip -br link
```

Step 3 — Name resolution stack

```
$ cat /etc/hosts
```

Step 4 — DNS diagnostics

```
$ dig +short killercoda.com
```

Network Configuration

Step 5 — NetworkManager and netplan syntax

```
$ nmcli device status 2>&1 | head
```

Step 6 — Sockets and connectivity tests

```
$ ss -tulnp | head
```

Step 7 — Packet capture and port scan

```
$ timeout 3 tcpdump -i any -nn -c 5 'icmp or port 53' 2>&1 &
```

✅ **Verify** — You can read the resolution path (nsswitch -> hosts/resolv.conf -> DNS) and pick the right diagnostic: ss, ...

1.5 — Shell operations

Use navigation, editing, redirection, and environment variables.

Exam objective — Given a scenario, manage a Linux system using common shell operations

Hands-on — Lab 5

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 5 — Shell Operations & Text Processing

Work with shell environment variables, login vs interactive startup files, every common redirection form, and the classic text-processing pipeline to assemble ...

You'll build — Explore env vars and PATH, paths and startup files, all redirection operators, then grep/cut/sort/uniq, sed/awk, ...

Key commands — export/env/unset, login vs interactive shells, redirection > >> < << <<< | tee, grep/cut/sort/uniq/wc, sed/awk, tr/xargs

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-05-*/README.md on the course GitHub repo

Shell Operations & Text Processing

Step 1 — Environment variables and PATH

```
$ echo "USER=$USER HOME=$HOME SHELL=$SHELL"
```

Step 2 — Absolute vs relative paths and startup files

```
$ mkdir -p /tmp/lab5 && cd /tmp/lab5
```

Step 3 — Redirection operators

```
$ echo "first line" > out.txt
```

Step 4 — grep, cut, sort, uniq, wc

```
$ cp /etc/passwd .
```

Shell Operations & Text Processing

Step 5 — sed and awk

```
$ sed -n '1,5p' passwd
```

Step 6 — head, tail, tr, xargs

```
$ head -3 passwd
```

Step 7 — Editors: nano and vim, scripted

```
$ apt-get install -y vim nano >/dev/null
```

✅ **Verify** — You can compose grep/awk/sed/cut/sort/uniq/tr/xargs into ad-hoc pipelines and know which startup file each ...

1.6 — Backups and restores

Perform archiving, compression, and data recovery.

Exam objective — Given a scenario, perform backup and restore operations

Hands-on — Lab 6

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 6 — Backup & Restore

Create tar/cpio/dd archives with gzip/bzip2/xz compression, image and recover disks with dd and ddrescue, mirror trees with rsync, and verify integrity with ...

You'll build — Make sample data, archive with tar and cpio, dd a disk image, recover a damaged one with ddrescue, mirror with rsync ...

Key commands — tar (czf/xzf/tzf), cpio, dd, ddrescue, rsync --delete, sha256sum -c

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-06-*/README.md on the course GitHub repo

Backup & Restore

Step 1 — Install tooling and create sample data

```
$ apt-get install -y tar cpio gzip bzip2 xz-utils rsync gddrescue coreutils >/dev/
```

Step 2 — tar with gzip, bzip2, and xz

```
$ cd /lab6
```

Step 3 — cpio (the older archiver)

```
$ cd /lab6
```

Step 4 — dd a disk image to a loopback file

```
$ cd /lab6
```

Backup & Restore

Step 5 — ddrescue from a damaged image

```
$ cd /lab6
```

Step 6 — rsync local-to-local with delete

```
$ mkdir -p /lab6/mirror
```

Step 7 — Integrity and reading compressed files

```
$ cd /lab6
```

✅ **Verify** — You can choose tar/cpio/dd/ddrescue/rsync appropriately and know sha256sum -c and rsync ...

1.7 — Virtualization

Deploy hypervisors, create VMs, and manage disk images.

Exam objective — Summarize virtualization on Linux systems

Hands-on — Lab 7

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 7 — Virtualization with QEMU/KVM & libvirt

Install QEMU and libvirt, manipulate virtual disk images with qemu-img, list libvirt domains/networks with virsh, and boot a tiny guest under QEMU to discuss ...

You'll build — Install QEMU/libvirt, create and inspect disk images, convert/resize them, take qcow2 snapshots, list libvirt ...

Key commands — QEMU vs KVM, /dev/kvm vs TCG, qemu-img (qcow2/raw), qemu-img convert/resize, qcow2 snapshots, virsh list/net-list

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-07-*/README.md on the course GitHub repo

Virtualization with QEMU/KVM & libvirt

Step 1 — Install QEMU and libvirt

```
$ apt-get install -y qemu-system-x86 qemu-utils libvirt-clients libvirt-daemon-sys
```

Step 2 — Create and inspect disk images

```
$ mkdir -p /lab7 && cd /lab7
```

Step 3 — Convert and resize images

```
$ cd /lab7
```

Step 4 — Snapshots inside a qcow2

```
$ cd /lab7
```

Virtualization with QEMU/KVM & libvirt

Step 5 — libvirt: domains and networks

```
$ systemctl start libvirtd 2>/dev/null || service libvirtd start 2>/dev/null || tr
```

Step 6 — Boot a minimal guest under QEMU (TCG fallback)

```
$ cd /lab7
```

Step 7 — Discuss network modes (concept)

```
$ Bridged    : guest is a peer on the LAN with its own DHCP IP.
```

✅ **Verify** — You can distinguish QEMU (emulator) from KVM (accelerator), qcow2 from raw, internal vs libvirt snapshots, ...

Recap — System Management

Lab 1 — Boot Process & Filesystem Hierarchy

Lab 2 — Kernel Modules & Device Management

Lab 3 — Storage with LVM, Partitions & Filesystems

Lab 4 — Network Configuration

Lab 5 — Shell Operations & Text Processing

Lab 6 — Backup & Restore

Lab 7 — Virtualization with QEMU/KVM & libvirt

End of Day 1

See you tomorrow — 9:30 AM

Services and User Management

20% of the exam | Objectives 2.1–2.6 | Labs 8–13

Type it, break it, fix it — the exam rewards hands that have done the work.

Services and User Management — Objectives

2.1 · Files & directories

Control permissions, create links, and manage special files.

2.2 · Account management

Add, remove, and modify users and groups.

2.3 · Process control

Monitor states, adjust priorities, and schedule jobs.

2.4 · Software management

Install, update, or remove packages and repositories.

2.5 · Systems management

Start, stop, and review services, logs, and timers.

2.6 · Containers

Operate container runtimes, manage images, and create networks.

Key Concepts — Services and User Management

Everything is a file: manage files/links/devices, and local accounts via `/etc/passwd`, `/etc/shadow` and `/etc/group`.

Processes have states, priorities and signals; jobs are scheduled with `at`, `cron` and `anacron`.

Software is installed with `apt/dpkg` and `dnf/rpm`, plus language (`pip/npm/cargo`) and sandboxed (`snap/flatpak`) managers.

`systemd` manages services, timers, mounts and targets; containers (`Docker/Podman`) package and isolate applications.

2.1 — Files & directories

Control permissions, create links, and manage special files.

Exam objective — Given a scenario, manage files and directories

Hands-on — Lab 8

Type it, break it, fix it — the exam rewards hands that have done the work.

Advanced Navigation

".." pairs move up one level in the directory hierarchy.

The first ".." takes you to /home1, and the second ".." brings you to the root directory.

After reaching the root, relative paths are used.

The directory you end up in depends on your starting point.

A relative path doesn't begin with a "/" and indicates the path is relative to the current directory.

The single dot (.) represents the current directory, useful in commands like find.

```
# find ./ -iname "*.txt"
```

Listing Files and Directories

The `ls` command lists files and directories.

When used for multicolumn output, it only shows file or directory names, not additional details about them.

Use the `-l` option with the `ls` command for a long listing.

This option shows detailed information about files or directories in a single column.

The `-l` option with `ls` shows permissions, ownership, and link counts, which are not displayed in regular `ls` output.

The `stat` command can also provide detailed information for a single file or directory.

```
# ls -lai
290305 drwxr-x--- 13 root root 4096 Jan 24 18:55 .
2 drwxr-xr-x 20 root root 4096 Jan 24 17:56 ..
292606 -rw-r--r-- 1 root root 1354 Jan 21 00:23 anaconda-ks.c
292748 -rw----- 1 root root 3470 Jan 24 18:16 .bash_history
290485 -rw-r--r-- 1 root root 24 Jun 10 2000 .bash_logout
290486 -rw-r--r-- 1 root root 234 Jul 5 2001 .bash_profile
290487 -rw-r--r-- 1 root root 176 Aug 23 1995 .bashrc
290488 -rw-r--r-- 1 root root 210 Jun 10 2000 .cshrc
```

Touching Files

The `touch` command creates new files if they don't exist or updates their timestamps if they do.

It is commonly used to create blank files (e.g., log files) or update a file's modification time.

The `touch` command can be used with relative pathnames to create files in the current directory.

You can use an absolute pathname with the `touch` command to create a file in a specified location.

The `touch` command is useful for creating log files and using a reference file to mark the last successful backup.

When used on an existing file, `touch` updates all three file times: access, change, and modify.

```
# touch filename
```


Copying Files and Directories

Copying a file creates a new file with a separate inode, which is a unique identifier assigned by the filesystem.

An inode is a pointer to a file on disk and is essential for file existence; a file cannot exist without an inode.

In contrast, moving a file keeps the same inode but changes the file's name (if within the same filesystem).

When copying files, the new file gets its own permissions, which may differ from the original unless the `-p` option is used to preserve permissions and ...

Only the root user can change file ownership; regular users copying files will own the new copies.

A normal copy simply replicates the file to a new location.

```
# cp /home/lukec/*.mp3 .  
# cp /home/lukec/*.mp3 /home/rossb
```

Moving Objects

The `cp` command creates a new file with a separate inode and data, while the `mv` command only changes the file's directory or metadata.

If the `mv` command moves a file within the same device, it only alters the file's metadata (directory entry).

Moving a file across devices involves copying the file to the new device and deleting it from the old one.

You can use the `stat` command to check file details, such as metadata, after creating a file (e.g., `file1`).

When moving a file, the inode and device remain the same, and the change time reflects the renaming.

The `mv` command overwrites the destination file if it exists, but the `-i` option prompts for confirmation before overwriting.

```
# mv file1 file2
# stat file2
File: 'file2'
Size: 0 Blocks: 0 IO Block: 4096 regular empty file
Device: fd00h/64768d Inode: 2261179 Links: 1
```

Creating and Removing Directories

To create a directory, use `mkdir dir1`.

To create a subdirectory inside a directory, use `mkdir dir1/subdir1`.

The last segment in a directory path represents the object being created or removed, while the rest are parent objects.

Both `mkdir` and `rmdir` can handle directory structures, with `mkdir` having the `-p` option to create a full subdirectory tree (creating missing parent ...

The `rmdir` command can only remove empty directories.

It targets the last directory in a path (e.g., `/dir1/dir2/dir3/dir4`).

```
# mkdir -p /home/user1/dir1/dir2/dir3/dir4
```

The file Command

The `file` command uses multiple sources to determine file types, including system calls, the magic number file (`/usr/share/magic`), and character sets like ...

It can also identify files by recognizing strings from programming languages or other formats.

The output of the `file` command can be manipulated and filtered to display useful information about the file type.

Running the `file` command on a known binary reveals details about the file's architecture and layout.

Running the `file` command on a directory shows the types of files, but the output can be filtered using the pipe (`|`) and `grep` commands.

Filtering with `grep` allows you to focus on specific results, such as finding "empty" files.

```
$ file file1
file1: ASCII text
```

The stat Command

The `stat` command is very useful, providing detailed file information.

It can also be used in conjunction with other commands to reveal even more details.

To use `stat`, simply invoke it with a file or filesystem object as the argument to view the information.

The `stat` command provides much more information than `ls -l`, including file type (similar to `file`), inode number, and device location.

Information displayed by `stat` includes:

File: Filename on the filesystem.

```
Access: (0664/-rw-rw-r--) Uid:(1000/ ubuntu)Gid:(1000/ ubuntu)
Access: 2022-01-14 19:32:09.307094382 -0700
Modify: 2022-01-14 19:32:09.307094382 -0700
Change: 2022-01-14 19:32:09.307094382 -0700
Birth: -
```

Symbolic Links

A symbolic (symlink) is used to create a shortcut from one object to another.

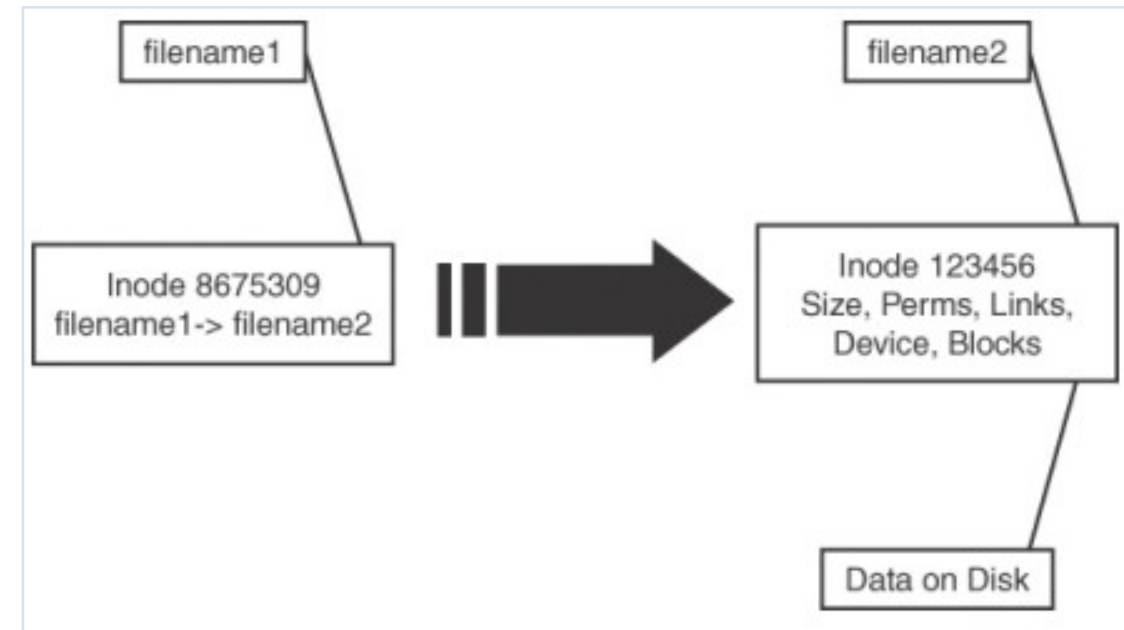
A symlink has its own inode and stores the path to the linked file.

Symlinks can span filesystems and directories because they are independent files with their own inode.

For example, to allow multiple users to access a remote disk mounted at /mnt/projdata, you can create a symlink in each user's home directory pointing to ...

A symlink's listing shows where it points, and its permissions are set to the maximum to avoid affecting the target file's permissions.

Changing permissions on a symlink affects the target file, not the symlink itself.



Hard Links

A hard link is an additional name in a directory that points to the same inode as the original file, sharing all aspects except the filename.

Hard links are useful for ensuring files, like important data, are easily restorable after being accidentally deleted.

For example, to back up Jaime's important `contactlist.txt` file, you can create a hard link from the file in his home directory to a location in the ...

The link count for a hard-linked file indicates how many names point to the same inode.

The `stat` command won't show other filenames for a hard link, only the link count.

A file's name and location are not stored in the inode, which is important for understanding hard links.

```
# ls -li ccontactlist.txt
17392 ccontactlist.txt
find / -inum 17392
/home/jaime/ccontactlist.txt
/backup/home_jaime_ccontactlist.txt
```

Using tar

The tar command is essential for archiving and file management, originating from the term "tape archive."

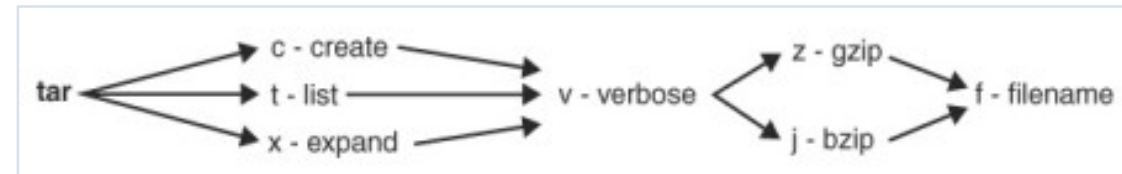
It packages multiple files into one, often for tape backup, but can also output to disk.

tar allows for various transformations, such as splitting archives, excluding files, and changing directory names.

It can extract files from a .tar archive (tarball) and allows selective file extraction and renaming.

tar can also work with compression utilities like gzip, bzip2, and xz, resulting in compressed archive files (.tar.gz, .tar.bz2, .tar.xz).

The tar syntax includes the command itself followed by options (e.g., xvjf) and filenames.



Using Compression Utilities

Linux and Unix tools follow a "toolset mentality," where each tool specializes in a single task and works well with others.

Compression utilities like gzip, bzip2, and xz are commonly used with the tar command to reduce archive file sizes.

These compression utilities have corresponding decompression tools: gunzip, bunzip2, and unxz.

The tar command creates an archive without compressing files, while these compression tools reduce the size of individual files.

To compress a tar archive, use the -z option for gzip and add the .gz suffix to the filename.

To create a tar archive with bzip2 compression, use the -j option.

```
# tar -cjf archive.tar.bz2 /foo
```

Using tar with Compression Utilities

Compression Efficiency (Highest to Lowest):

Highest: xz

Medium: bzip2

Lowest: gzip

Developed on Unix but mainly used on PC platforms.

Unlike tar, zip doesn't automatically replace files with compressed versions; it works more like a combination of tar and a compression tool.

Taking Pity on the Unarchiver

Best Practice for Creating Tar Archives:

It's considered proper to archive files by specifying the directory that contains them, rather than just the files in the current directory.

This ensures that the extracted files appear in a single directory, not directly in the current working directory.

Example of Proper Archiving:

To archive the contents of /etc directory: `tar -cf etc.tar /etc`

When unarchived, the /etc directory appears in the current directory.

Using the dd Command

dd Command Overview:

dd is used for creating backup images (e.g., ISO files) of CDs, DVDs, and for duplicating devices.

It operates on raw device files and full file paths on the system.

Common usage examples:

Creating a boot disk: `dd if=/mnt/cdrom/images/boot.img of=/dev/sdb`

Creating an image from a USB device: `dd if=/dev/sdb of=/root/usb.img`

Compression Tools

Tar vs Compression Commands:

tar gathers files into an archive, while gzip, bzip2, and xz compress the archive.

These compression tools replace the original files with compressed versions (e.g., .gz, .bz2, .xz extensions).

Compression Utilities:

gzip: Widely used, compresses files to .gz.

Compress: gzip *

Navigating Within a File

Cursor movement keys:

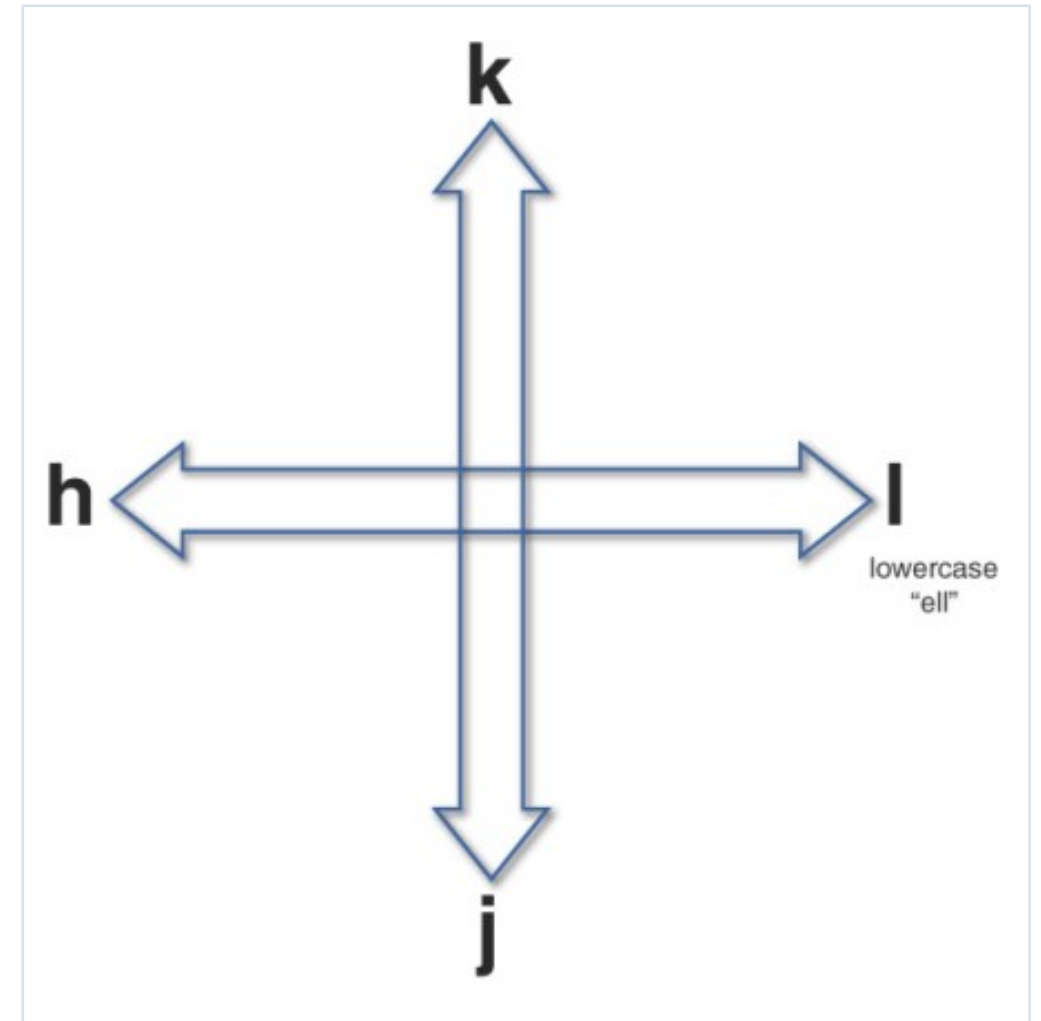
h: Left (like Left Arrow key)

j: Down (like Down Arrow key)

k: Up (like Up Arrow key)

l: Right (like Right Arrow key)

Ctrl key combinations:



nano's Interface

nano has a user-friendly interface with onscreen keystroke hints, unlike vi/vim which is more minimalistic.

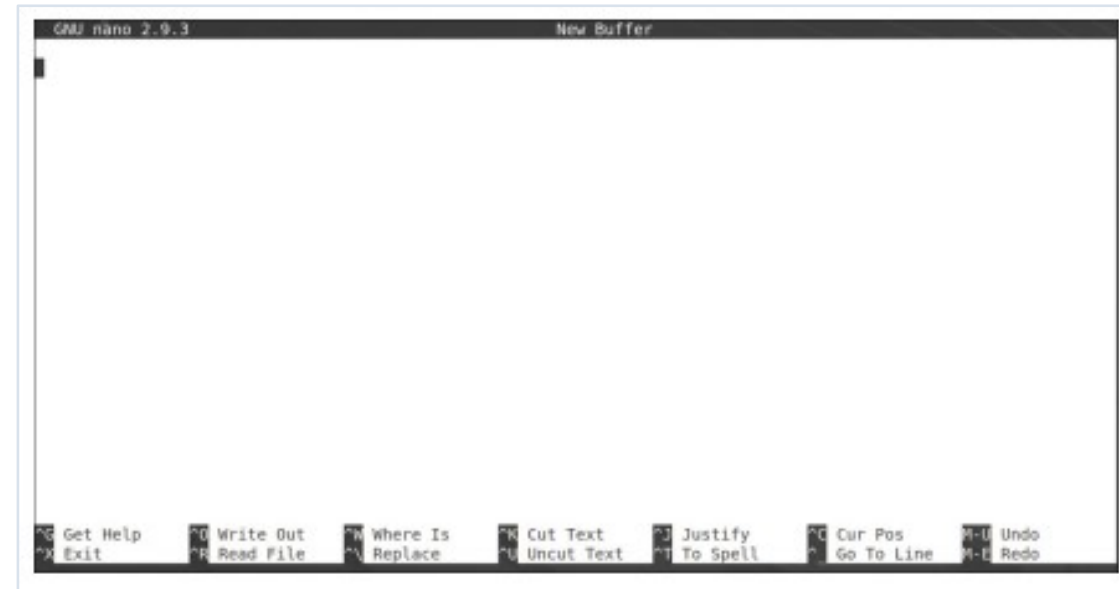
The Ctrl key is used with other keys for actions, such as Ctrl+X to exit (prompts to save or discard changes).

The ^ character in nano's interface denotes the Ctrl key, followed by the character to press.

Uppercase letters in the interface are for readability, but you don't need to press Ctrl+Shift for commands.

Meta key (accessed via Esc key) allows actions like Esc + U for Undo and Esc + E to restore changes.

nano may also allow using the Windows key (PC) or Cmd/Apple key (Mac) as the Meta key, depending on the system configuration.



Files & directories — More Key Topics

Tips for Working with Linux Files

Hidden Files: Files starting with a dot (e.g., .bashrc) aren't truly hidden; they're just not displayed by default. Filenames: ...

Basic Navigation

cd Command: Used to change the current working directory. Absolute Pathname: You can move to any directory (e.g., /etc) using the ...

Removing Objects

The rm command is typically used to remove files, while rmdir is used for removing empty directories. To remove a file without ...

File Metadata and File Types

In Linux, everything is treated as a file, and it's important to understand the different file types. Since Linux doesn't require ...

Displaying File Metadata

Relying only on the ls command means you miss out on valuable information about files. The file command and the stat command ...

Soft and Hard Links

Links are useful for allowing users to access files or run commands without needing to navigate the entire system.

Files & directories — More Key Topics

Linking Files

There are two types of links: symbolic (soft) links and hard links, each with its own advantages and disadvantages. Sysadmins use ...

File Compression, Archiving, and Backup

Archiving and restoring files has evolved significantly over time, moving beyond older methods like tape drives. System operators ...

Useful Creation Options

Useful tar Archive Options: `-b`: Sets the block size for the media you're archiving to (e.g., for tape devices).

Listing Archive Files

Listing Tar Archives: Use `tar -tf archive.tar` to list the contents of a regular tar archive.

Using cpio

cpio Command Overview: The `cpio` command is used for copying files to and from an archive, and can be used for backups, though ...

Backing Up Is Hard to Do

Successful Restore is Crucial: A good backup is only useful if you can restore the data properly and quickly. Backup Methods: The ...

Lab 8 — File & Directory Management

Practice core file and directory operations: navigating, creating, copying, searching, linking, and inspecting special device files with standard Linux utilities.

You'll build — Create a lab tree, then copy, find, link, diff, and inspect files and /dev device nodes.

Key commands — ls -la / stat / file, find predicates (-name, -mtime, -size), locate & updatedb, lsof, diff & sdiff, hard vs symbolic ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-08-*/README.md on the course GitHub repo

File & Directory Management

Step 1 — Navigate and list

```
$ pwd
```

Step 2 — Create, copy, move, touch

```
$ touch src/file1.txt src/file2.log
```

Step 3 — Find files by name, time, and size

```
$ dd if=/dev/zero of=src/big.bin bs=1M count=5
```

Step 4 — locate and lsof

```
$ apt update -y && apt install -y mlocate lsof
```

File & Directory Management

Step 5 — diff and sdiff

```
$ echo -e "alpha\nbravo\ncharlie" > a.txt
```

Step 6 — Hard links vs symbolic links

```
$ echo "original" > target.txt
```

Step 7 — Device files in /dev

```
$ ls -l /dev/sda /dev/null /dev/tty1 2>/dev/null
```

✅ **Verify** — `ls -li` confirms hardlink shares the target's inode while the symlink breaks after the target is removed.

2.2 — Account management

Add, remove, and modify users and groups.

Exam objective — Given a scenario, perform local account management

Hands-on — Lab 9

Type it, break it, fix it — the exam rewards hands that have done the work.

Regular User Accounts

In Linux systems, typical user UIDs range from 500 to 65,000, but this can cause issues in larger environments.

To manage both local and network accounts, it's recommended to assign UIDs from 10,000 to 65,000 for network accounts (such as NFS and NIS) and from 500 ...

When managing a system with both network and local accounts, user account data can be stored in different places: local files like `/etc/passwd` and ...

This command searches both local and network sources in an order determined by the `/etc/nsswitch.conf` file.

`files nis` means “search local files first and then search the database for the specified value.”

```
passwd:  files nis
shadow:  files nis
```

User Entries in /etc/passwd

A user's entry in the /etc/passwd file consists of the following:

The /etc/passwd file contains user account details with the following format:

Username (ross): The user's name (should be 8 characters or fewer, typically lowercase).

x: Indicates that the password is stored in /etc/shadow for security.

500: The user's UID (User ID).

100: The user's primary group ID.

```
ross:x:500:100:Ross Brunson:/home/ross:/bin/bash
```

Group Passwords

A user can have only one primary group, but if they need to assign an object to another group, they can use the `chgrp` command. While this is simple for a ...

Another option is for a user to temporarily change his or her primary group with the `newgrp` command. For example

The `newgrp` command allows users to temporarily change their primary group to any group they're a member of, without altering the `/etc/passwd` or `/etc/group` ...

However, users can only switch to groups where they are secondary members. Additionally, a password can be assigned to a group using the `gpasswd` command, ...

```
# newgrp users
```


Adding Users with useradd

The **useradd** command is used to create new user accounts with default settings, such as the default primary GID and home directory. To add a user with ...

On a Red Hat system, the **useradd** command performs several actions when creating a new user.

Assigns the user a new UID, typically above 500.

Creates a group with the same name as the username and assigns it the same GID.

Sets the user's primary GID to the new group.

Creates a home directory for the user in the `/home` directory (e.g., `/home/snuffy`).

```
# useradd -m -d /home/snuffy -k /etc/skel -g 100 -u 1025 snuffy
```

skel Templates

When using the `useradd` command with the `-m` and `-d` options, a home directory is created for the user. By default, this new home directory is populated with ...

For example, the command `useradd -m -d /home/snuffy snuffy` creates the home directory `/home/snuffy`, copies the contents from `/etc/skel` into it, and ...

You can customize a new user's home directory by adding files or directories to the `/etc/skel` directory. For example, to create a special directory ...

Copy `/etc/skel` to `/etc/skel-payroll-staff`.

Populate `/etc/skel-payroll-staff` with the files and directories needed for payroll staff.

When creating a user from the payroll department, specify this template directory using the appropriate command syntax.

```
# useradd -k /etc/skel-payroll-staff bob
```

Adding Groups with groupadd

The **groupadd** command is used to add a new group and is simpler than the **useradd** command. To add a group with default settings, you can use the following ...

This command creates a group with a GID determined by using the “next available” number above 500. The result should be an entry like the following in the ...

The **groupadd** command creates a group, but no users are added by default (the last field for users is empty). To add users to a group, the **usermod** command ...

```
# groupadd somegroup
```

Removing Users

When a user leaves the company, you can either delete their account and keep the home directory (for a replacement, perhaps) or remove both the account ...

To remove the user along with his home directory, you execute the following command:

You cannot delete a user who is currently logged in. Before deleting the account, you need to figure out why the user is still logged in. On some Linux ...

If you don't receive an error message when attempting to delete a logged-in user, use the ps command to find the login shell's PID. First, lock the ...

You shouldn't remove a user's home directory right away because it may contain important files or evidence. If you're planning to replace the user, it's ...

Seeing a home directory with only a UID (e.g., UID 1002) instead of a username can be a helpful clue that the user has been deleted. Since the home ...

```
# usermod -L snuffy
# kill -9 17098
# userdel -r
```

Removing Groups

Before deleting a group, make sure to:

Confirm the group has no secondary members.

Verify the group isn't the primary group for any user accounts.

Use the find command to locate all files owned by the group, then change the group ownership of those files to another group.

To delete a group, execute the groupdel command:

```
# groupdel grpname
```

Encrypted Passwords and Shadow Fields

The following is an example from the `/etc/passwd` file on a system that doesn't use the Shadow Suite:

In the `/etc/passwd` file, the second field contains an encrypted password string, which can be viewed by all users, posing a security risk. However, when ...

When the Shadow Suite is installed, encrypted passwords are stored in the `/etc/shadow` file for user accounts and in the `/etc/gshadow` file for group ...

A `/etc/shadow` file entry contains the following fields for a user:

Login name: The user's login name (e.g., `snuffy`).

Password: The user's encrypted password.

```
julia:$1$vEE0vj1b$G1zLuD9F..Dj1Qr/WXcJv1:16263:5:90:7:30::
```

Account management — More Key Topics

User Account Fundamentals

User and group accounts are identified by their user ID (UID) and group ID (GID) in the kernel, not by the human-readable names. ...

What Accounts Are What?

The `/etc/passwd` file contains various system accounts, each assigned a UID between 0 and 499. Some of the most important accounts ...

Special Login Files

Several files control a user's login behavior and access on a Linux system: `/bin/false`: Sets the user's shell to prevent login, ...

Group Accounts

In Linux and Unix, groups are used to organize users. Groups cannot be nested. Users are associated with groups in two ways ...

Group Entries in `/etc/group`

The entries in the `/etc/group` file are simpler and shorter than those in the `/etc/passwd` file. Each entry consists of the ...

Adding Users and Groups

While it's possible to edit the `/etc/passwd` and `/etc/group` files directly, doing so can cause significant issues if errors are ...

Account management — More Key Topics

useradd Defaults

On Red Hat-based systems, the useradd command's default settings are defined in the `/etc/default/useradd` file. These defaults can ...

The Default Shell

For example, to change the default shell from `/bin/bash` to `/bin/tcsh`, execute the command. Debian-based systems also use this ...

Modifying Users and Groups

Creating users and groups is only part of system management; maintaining them in a dynamic environment is equally important. This ...

Modifying User Accounts with usermod

To modify a user account using the usermod command, several options can be used based on the changes needed: `-c`: Modifies the ...

Modifying Groups with groupmod

The groupmod command is used to modify a group's attributes, primarily for changing a group's name and GID. To modify a group's ...

Removing Users and Groups

Removing users and groups in Linux is straightforward, but there are important details to consider. Unlike Windows, which uses ...

Lab 9 — Local Account & Group Management

Create and manage local users and groups, set passwords and aging policies, change shells, and distinguish user, system, and service accounts.

You'll build — Create users/groups, set passwords and aging, inspect identity databases, then remove the accounts.

Key commands — /etc/passwd, /etc/shadow, /etc/group, /etc/skel, useradd / adduser / usermod -aG, chpasswd, passwd -S, chage, chsh, ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-09-*/README.md on the course GitHub repo

Local Account & Group Management

Step 1 — Inspect account databases

```
$ head -5 /etc/passwd
```

Step 2 — Create users and groups

```
$ groupadd developers
```

Step 3 — Passwords and aging

```
$ echo "alice:Passw0rd!" | chpasswd
```

Step 4 — Change shell and identify accounts

```
$ chsh -s /bin/sh bob
```

Local Account & Group Management

Step 5 — User vs system vs service accounts

```
$ awk -F: '$3 >= 1000 && $3 < 65534 {print "USER  ", $1, $3}' /etc/passwd
```

Step 6 — Login history and active sessions

```
$ last | head
```

Step 7 — Remove users and groups

```
$ userdel -r bob
```

✅ **Verify** — `getent passwd bob` returns nothing after `userdel -r`, confirming the account and home directory are gone.

2.3 — Process control

Monitor states, adjust priorities, and schedule jobs.

Exam objective — Given a scenario, manage processes and jobs

Hands-on — Lab 10

Type it, break it, fix it — the exam rewards hands that have done the work.

Managing Processes

Managing programs and processes is crucial for running a Linux machine.

Viewing processes can be done using the `ps` command, which shows process information.

The `ps` command has various switches to format the output.

Example: To display processes started by a user and the user's shell, just type `ps` in the terminal.

The default `ps` command provides a simple view of the system's processes.

This basic view omits background or non-terminal-associated processes.

Now type the following:

```
ps -a | wc -l  
66
```

What's the Diff?

Confusion exists about using `ps aux` vs. `ps -aux` due to different command styles.

BSD style: Options can be grouped (e.g., `aux`) without a dash.

POSIX/Unix style: Options must be preceded by a dash (e.g., `-aux`).

Linux supports both styles for the `ps` command.

The `pstree` command can be used to show the hierarchical structure of processes.

The `ps` and `pstree` commands have many options.

```
init-+-apmd
|-atd
|-bdf flush
|-bonobo-activati
|-crond
|-cupsd
|-dhcpcd
|-evolution-alarm
|-gconfd-2
|-gnome-cups-mana
|-gnome-name-serv
|-gnome-panel
|-gnome-settings-
|-gnome-smproxy
|-gnome-terminal-+-bash
| `--mgt-ptty-helper
```

Interpreting Displayed Information from free

The free command has several options.

Before exploring execution examples, it's important to understand the definitions of the output columns.

The definitions for these columns are available in the free man page.

The data displayed by the free command comes from raw files in the /proc/meminfo directory.

The free command organizes and presents this data for easier viewing.

The total column shows the total RAM on the system.

```
total Total installed memory (MemTotal and SwapTotal in /proc/meminfo)
used memory (calculated as total - free)
free Unused memory (MemFree and SwapFree in /proc/meminfo)
shared Memory used (mostly) by tmpfs (Shmem in /proc/meminfo, available on kernels 2.6.32, displayed as zero if not available)
buffers Memory used by kernel buffers (Buffers in /proc/meminfo)
cached Memory used by the page cache (Cached in /proc/meminfo)
```

Using pgrep and pkill to Send Signals

Using `ps` and `kill` is fine for individual processes, but for multiple processes, `pgrep` and `pkill` are more efficient.

`pgrep` is used to search for processes by name, and `pkill` sends signals to those processes.

`pgrep` is like an investigator (searching for processes), and `pkill` is the enforcer (sending signals).

You can search for processes using a string or regular expression (e.g., `ps aux | grep -i "Chrome Helper"`).

Instead of manually killing each PID, you can use `pgrep` to search for processes and return their PIDs efficiently.

`pgrep` returns the PIDs matching the search, similar to the output from `ps aux`, but only shows the PIDs.

```
$ ps aux | grep -i sshd
root          660  0.0  0.6 12212  6368 ?    Ss   Feb16   0:(
sshd: /usr/sbin/sshd -D
root          791  0.0  0.7 15464  7420 ?    Ss   Feb16   0:(
sshd: ubuntu [priv]
ubuntu       914  0.0  0.4 15600  4540 ?    S    Feb16   0:(
sshd: ubuntu@notty
root        7121  0.0  0.7 15464  7764 ?    Ss   18:10   0:(
sshd: ubuntu [priv]
ubuntu       7230  0.0  0.4 15600  4400 ?    S    18:10   0:(
sshd: ubuntu@pts/0
ubuntu       7251  0.0  0.0  7692   668 pts/0    S+   18:16
grep --color=auto sshd
```


Killing Processes Using Other Criteria

Sometimes, you can't kill a process by PID, or there are too many similarly named processes to kill individually.

killall is used to kill multiple processes by name, making it easier to target all processes of a certain type (e.g., Chrome or Netscape).

To use killall, you can first run `ps aux` to find the process names, then execute `killall` followed by the process name to terminate them.

pkill is a more advanced process kill command that can target processes based on criteria like name, PID, username, session ID, and terminal.

When using multiple criteria with pkill, all must match for the signal to be sent.

pgrep is commonly used to test pkill commands before executing them.

```
# renice +5 'pgrep chrome'
```

Job Control

Job control allows running multiple commands in the background while working in the foreground.

To put a command in the background, use the & character (e.g., `command1 -option &`).

Start a task (e.g., editor).

Suspend the task with Ctrl+Z.

Check task status with `jobs`.

Send the job to the background using `bg`.

```
# kill %1
```

```
Vim: Caught deadly signal TERM
```

```
Vim: Finished.
```

Managing Process Priorities

Linux uses a combination of priority and scheduling to manage multiple processes.

On single processors, it creates the illusion of multitasking; on multiple processors, it enables true multitasking.

Process priorities range from -20 (highest priority) to 19 (lowest priority).

A process's default priority is 0 unless set otherwise by the programmer.

Users can start processes with a priority from 0 to 19, but only root can start processes with a higher priority (from 0 to -20).

Too many processes running at a high priority can slow down or even crash the system.



Figure 4-4 Linux process priorities

Booting with systemd

When booting a system with systemd, the process resembles SysVinit's init script structure.

The system starts systemd/init from GRUB, which then begins reading files.

The first file read after the kernel and initramdisk is usually default.target, which is a symlink to multi-user.target.

Despite appearing simple, the boot process involves a more complex structure.

The boot process is more complex than it seems, with dependencies between targets.

graphical.target requires multi-user.target.

```
Kernel
|
v
multi-user.target
|
v
graphical.target
```

Listing Services

To list all services or units on a system, use the appropriate **systemctl** command.

The **systemctl** command shows all active units on the system.

If a unit is in an "exited" state, it may indicate an error or that it is a non-daemon service that has finished running.

To display all loaded units, including inactive ones, use a different **systemctl** command.

The command shows all active and inactive services/units on the system.

It lists all available services/units on the system, regardless of their state or status.

```
# systemctl list-unit-files
```

Service Start, Stop, Restart, and Status

To start a service, use `systemctl start servicename`.

If the service is not already running and you have the necessary permissions, it will be started.

It's a good practice to check the service's status first by using `systemctl status servicename`.

The output in the example shows that the "tuned" service is loaded, active, and enabled by default, meaning it will run after boot.

To stop the "tuned" service, use the `systemctl stop servicename` command.

After stopping the service, you can query its status using the same `systemctl status servicename` command.

```
# systemctl status tuned
tuned.service - Dynamic System Tuning Daemon
   Loaded: loaded (/usr/lib/systemd/system/tuned.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2019-01-05 18:29:04 EST; 2 weeks 2 days ago
     Docs: man:tuned(8)
           man:tuned.conf(5)
           man:tuned-adm(8)
    Main PID: 1170 (tuned)
      Tasks: 5
    CGroup: /system.slice/tuned.service
            |--1170 /usr/bin/python -Es /usr/sbin/tuned -

Jan 05 18:29:01 localhost.localdomain systemd[1]: Starting D
```

Scheduling Services

Masking a service creates a symlink to `/dev/null`, preventing the service from loading.

Even if the service exists elsewhere, the symlink to `/dev/null` takes precedence.

To unmask a service, simply remove the symlink to `/dev/null`.

Cron is the main job scheduler in Linux, executing jobs on a fixed schedule.

Jobs can be any command or script run on the command line.

Cron is typically used to schedule intensive tasks during off-peak times to optimize system performance.

```
# systemctl unmask servicename
```

Issues About PATH

Cron runs with a minimal environment, not executing `.bash-profile` or `.bashrc`.

This results in a basic PATH and potentially missing environment variables.

To check, you can add a cron entry that outputs the environment variables for debugging.

You can set environment variables in the crontab, including the PATH.

Special cron variables include:

MAILTO: Sends job output to a specified email address.

This job runs every minute and dumps the environment to `/tmp/env`:

```
SHELL=/bin/sh
USER=rossb
PATH=/usr/bin:/bin
PWD=/home/bo
LANG=en_US.UTF-8
SHLVL=1
HOME=/home/rossb
LOGNAME=rossb
_=/usr/bin/env
```


Dealing with Output

Cron jobs send their output to the user's email by default.

You can handle output in three ways:

Accept the emails, though this may lead to inbox clutter.

Modify scripts to only generate output in case of an error.

Redirect output to `/dev/null` to discard it.

Each approach has pros and cons, and the best choice depends on the nature of the job.

```
25 * * * * /usr/bin/php /home/bo/fof/update-quiet.php > /dev,  
2>&1
```

The at Command

The at command schedules a task to run once at a specific time.

Jobs are queued in the /var/spool/at directory, with each job represented by a file.

The at command is used for one-off or infrequent tasks, like reminders or sysadmin duties.

When using the at command, type the desired time (e.g., "at 2pm today") to get the at> prompt.

Type the command to be executed, press Enter, and press Ctrl+D to end the task.

The notice marks the end, and job scheduling info is displayed.

```
$ at 2pm today
at> xmessage "take a break"
at> <EOT>
job 1 at 2004-04-02 14:00
```

The batch Command

The batch command is similar to the at command but runs jobs when the system load average is below a certain threshold, indicating the system has enough ...

Both batch and at use system load average to determine when to run jobs.

The load average is displayed when you run the w (who) command.

The load average represents the average number of processes waiting to run over the past 1, 5, and 15 minutes.

A load average of 0.60 over 5 minutes means the system was busy 60% of the time during that interval.

Systems with two CPUs are considered busy when the load average is 2.

```
$ w
16:04:34 up 135 days, 17:40, 2 users, load average: 0.54, 0.
0.51
```

Process control — More Key Topics

The htop Command

The htop command provides a graphical, user-friendly way to monitor system processes. It supports mouse usage and has color ...

The free Command

The free command shows detailed information about system memory usage, including: Amount of free and in-use physical memory.

Blocks and Buffers

Blocks are the addressable units on block devices (like disks), usually 512 bytes in size, but system software may use larger ...

Pages, Slabs, and Caches

The kernel manages memory using pages, which are tracked in the struct_page table. Caches consist of slabs, and slabs are made of ...

Sending Signals to Processes

Signals are typically learned through a list with explanations and are associated with commands like kill and killall. If you're ...

Killing Processes by PID

The kill and killall commands send signals to processes to stop them; they don't remove processes from memory. The default signal ...

Process control — More Key Topics

Introducing lsof

The lsof command lists open files and provides detailed information about them. On Linux, open files come in various types:

Using lsof

You can use the lsof command to list open files on a system, including those on the root directory, as demonstrated in Example ...

What's Different About systemd?

systemd replaced SysVinit and became the default initialization method for most Linux distributions. Major features of systemd:

Units in systemd

In systemd, "init scripts" are replaced by systemd units, which represent the configuration of system services. Units are defined ...

Unit File Directory Locations

systemd units are typically found in three directories, with the following precedence: /usr/lib/systemd/system/: Units installed ...

systemd Environment Variables

systemd environment variables are set using the EnvironmentFile= directive in the [Service] section of a service file (e.g., ...

Lab 10 — Processes, Jobs & Scheduling

Inspect processes, manipulate shell jobs, adjust priorities, send signals, and schedule tasks with `at`, `cron`, and `anacron`.

You'll build — Examine processes via `ps/proc/strace`, control background jobs, `renice`, `signal`, and schedule jobs.

Key commands — `ps auxf` / `pstree` / `pidstat` / `mpstat`, `/proc/<PID>` and `lsuf -p`, process states (R,S,D,Z,T) & `strace`, jobs: `&`, `Ctrl+Z`, `bg`, ...

Workbench — Killercode Ubuntu Playground, free in the browser

Full walkthrough — `labs/lab-10-*/README.md` on the course GitHub repo

Processes, Jobs & Scheduling

Step 1 — Inspect running processes

```
$ ps auxf | head -20
```

Step 2 — Explore /proc and lsof

```
$ sleep 600 &
```

Step 3 — Process states and strace

```
$ ps -o pid,stat,cmd -p $SPID
```

Step 4 — Background, foreground, jobs

```
$ sleep 300 &
```

Processes, Jobs & Scheduling

Step 5 — Priority with nice and renice

```
$ nice -n 10 sleep 200 &
```

Step 6 — Signals: kill, pkill, killall

```
$ sleep 1000 &
```

Step 7 — Scheduling with at and cron

```
$ service atd start 2>/dev/null || systemctl start atd
```

✅ **Verify** — atq lists the queued at job and crontab -l shows the recurring entry, confirming both schedulers accepted the ...

2.4 — Software management

Install, update, or remove packages and repositories.

Exam objective — Given a scenario, configure and manage software

Hands-on — Lab 11

Type it, break it, fix it — the exam rewards hands that have done the work.

Managing Local Debian Packages

Used for manipulating packages on a system, particularly .deb files.

--info option: Allows you to see the contents and details of a .deb package.

Displays detailed metadata from inside a .deb package.

conffiles: Lists configuration files not removed after package removal.

control: Contains metadata like package dependencies.

md5sums: Provides checksums to verify file integrity.

```
# dpkg --info acct_6.5.5-1_amd64.deb
new debian package, version 2.0.
size 120424 bytes: control archive=1899 bytes.
79 bytes, 4 lines conffiles
746 bytes, 18 lines control
1766 bytes, 27 lines md5sums
507 bytes, 30 lines * postinst #!/bin/sh
370 bytes, 26 lines * postrm #!/bin/sh
163 bytes, 7 lines * prerm #!/bin/sh
Package: acct
Version: 6.5.5-1
Architecture: amd64
Maintainer: Mathieu Trudel-Lapierre <mathieu.tl@gmail.com>
Installed-Size: 380
Depends: dpkg (>= 1.15.4) | install-info, libc6 (>= 2.3)
Section: admin
Priority: optional
Homepage: http://www.gnu.org/software/acct/
Description: The GNU Accounting utilities for process and l
accounting
...
```

Installing Packages with dpkg

The most common way to install packages is by using commands from the command line in the current directory.

To install dependencies for a package, either install them first or specify all required packages in the same command.

```
Selecting previously unselected package gcc-4.7-base:amd64.  
(Reading database ... 142987 files and directories currently  
installed.)  
Unpacking gcc-4.7-base:amd64 (from gcc-4.7-base_4.7.2-5_amd64
```

Removing Packages

-r or --remove: Removes the package's files but keeps the configuration files on the disk.

-P or --purge: Removes all of the package's files, including configuration files.

The `conffiles` file lists configuration files associated with an installed package.

This file helps identify which files will be removed during a package removal.

It also indicates which configuration files should be purged during a package purge action.

To remove a package entirely, you can use `apt-get` with two options:

To remove a package but leave its configuration files in place, use one of the following:

```
# dpkg --remove package_name  
# dpkg -r package_name
```

To purge a package, including its configuration files, use one of the following:

```
# dpkg --purge package_name  
# dpkg -P package_name
```

Dependency Issues

Package installations can conflict with existing entries in the package database.

During removal, a package may try to remove files that other packages depend on.

The `--force` option in package tools should be used with caution, as it can cause serious issues.

For the Linux+ exam, you should know how to force an action, but you don't need to memorize all the `--force` options.

To force the removal of a broken package marked for reinstallation, a specific command can be used

Use `--force-reinstreq` to remove packages marked for reinstallation.

```
# dpkg --install --force-depends package_name
```

Querying Packages

When querying packages without referencing a local .deb file, you're querying the package database, which lists all installed packages and their file ...

You can use the `-l` or `--list` option to get information about installed packages, such as their existence, version, name, and description.

You can query a package using its singular name (e.g., `krb5-user` for a Kerberos-related package) or the previous filename to see the package details.

The output shows that there are three packages starting with "krb5." Two are uninstalled (indicated by "un"), while `krb5-locales` is installed with version ...

The Debian package system displays packages previously installed on the system.

You can check the status of a particular package using the `-s` or `--status` option.

```
# rpm -qRp tree-1.7.0-16.fcXX.x86_64.rpm
libc.so.6()(64bit)
libc.so.6(GLIBC_2.2.5)(64bit)
libc.so.6(GLIBC_2.3)(64bit)
libc.so.6(GLIBC_2.3.4)(64bit)
libc.so.6(GLIBC_2.4)(64bit)
rpmlib(CompressedFileNames) <= 3.0.4-1
rpmlib(FileDigests) <= 4.6.0-1
rpmlib(PayloadFilesHavePrefix) <= 4.0-1
rpmlib(PayloadIsXz) <= 5.2-1
rtld(GNU_HASH)
```

Reconfiguring Packages

Some packages include a configuration tool that runs during installation to help with initial setup.

If the tool is included, you can rerun it using the `dpkg-reconfigure` command followed by the package name.

The example shows running `dpkg-reconfigure` for the Postfix mail server.

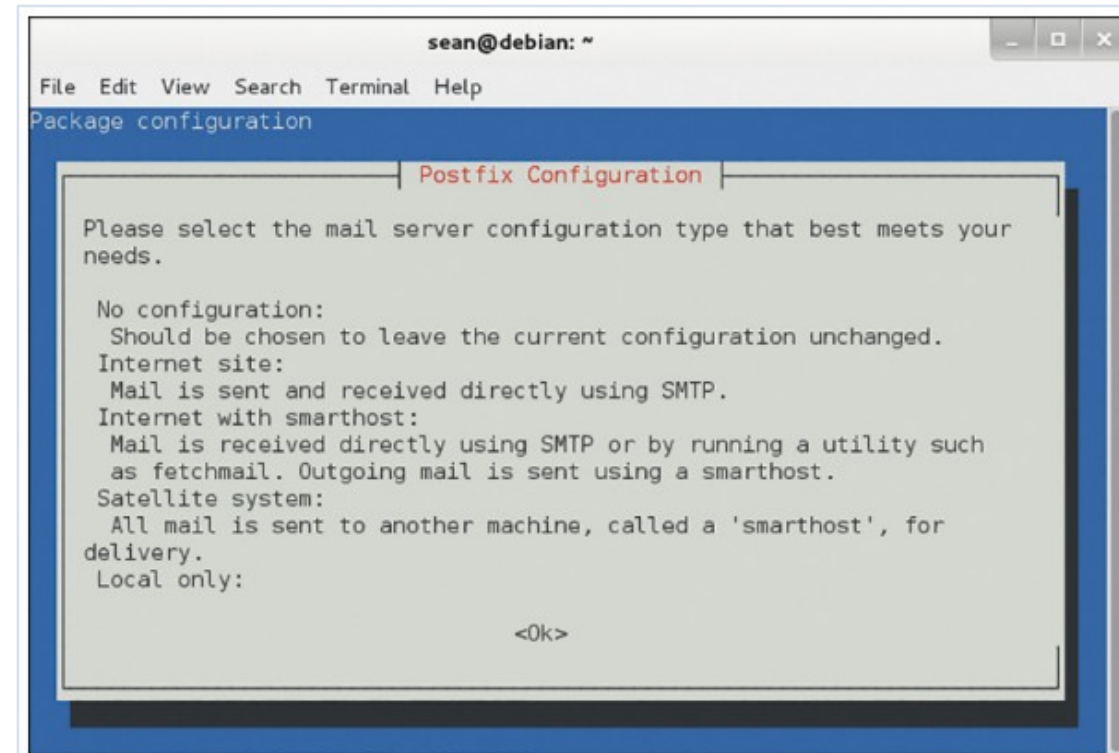


Figure 6-1 Reconfiguring Postfix with **dpkg-reconfigure**

```
# dpkg-reconfigure postfix
```


Using Repositories

CD/DVDs or ISO images may have limited software, but much more is available online through repositories.

Repositories in Debian/Ubuntu can be confusing due to the relationship with Ubuntu and Canonical, so repositories are often discussed from the Ubuntu ...

APT tools maintain a local cache of available packages and their dependencies for faster searching.

The list of remote package repositories is stored in `/etc/apt/sources.list` and individual files under `/etc/apt/sources.list.d`, with each line representing ...

```
deb http://cdn.debian.net/debian/ wheezy main
deb-src http://cdn.debian.net/debian/ wheezy main

deb http://security.debian.org/ wheezy/updates main
deb-src http://security.debian.org/ wheezy/updates main

# wheezy-updates, previously known as 'volatile'
deb http://cdn.debian.net/debian/ wheezy-updates main
deb-src http://cdn.debian.net/debian/ wheezy-updates main
```


What Is a Repository?

A repository is a collection or archive of software that can be accessed by the system, stored locally, on attached disks, or across a network via ...

Ubuntu repositories include:

Main: Canonical-supported free and open-source packages

Universe: Community-maintained free and open-source packages

Restricted: Proprietary driver packages for devices

Multiverse: Software with copyright or legal issues

```
# deb-src http://us.archive.ubuntu.com/ubuntu/ bionic main r
## Major bug fix updates produced after the final release of
## distribution.
deb http://us.archive.ubuntu.com/ubuntu/ bionic-updates main
restricted
# deb-src http://us.archive.ubuntu.com/ubuntu/ bionic-update
restricted
## N.B. software from this repository is ENTIRELY UNSUPPORTED
Ubuntu
## team. Also, please note that software in universe WILL NO
any
## review or updates from the Ubuntu security team.
deb http://us.archive.ubuntu.com/ubuntu/ bionic universe
# deb-src http://us.archive.ubuntu.com/ubuntu/ bionic univer
deb http://us.archive.ubuntu.com/ubuntu/ bionic-updates univ
# deb-src http://us.archive.ubuntu.com/ubuntu/ bionic-update
## N.B. software from this repository is ENTIRELY UNSUPPORTED
Ubuntu
## team, and may not be under a free licence. Please satisfy
as to
## your rights to use the software. Also, please note that s
in
## multiverse WILL NOT receive any review or updates from th
## security team.
deb http://us.archive.ubuntu.com/ubuntu/ bionic multiverse
# deb-src http://us.archive.ubuntu.com/ubuntu/ bionic multiv
deb http://us.archive.ubuntu.com/ubuntu/ bionic-updates mult
# deb-src http://us.archive.ubuntu.com/ubuntu/ bionic-update
multiverse
```

Installing Remote Packages

Once you understand repositories and how to add them, you can use the `apt-get` utility for most remote software installations.

Use the `install` command with `apt-get` to install a package (as shown in Example 6-5).

Example 6-5 shows installing the `nginx-light` package using `apt-get install nginx-light`.

The `nginx-common` package is also required, so both packages are downloaded and installed.

Setup scripts, including `dpkg-reconfigure`, are run during the installation.

Be aware of the differences between `apt-get` and `dpkg` commands, especially regarding options.

```
# sudo apt-get install nginx-light
[sudo] password for tux:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libnginx-mod-http-echo nginx-common
Suggested packages:
  fcgiwrap nginx-doc
The following NEW packages will be installed:
  libnginx-mod-http-echo nginx-common nginx-light
0 upgraded, 3 newly installed, 0 to remove and 8 not upgraded
Need to get 449 kB of archives.
After this operation, 1,503 kB of additional disk space will
Do you want to continue? [Y/n] y
Get:1 http://us.archive.ubuntu.com/ubuntu bionic-updates/ main
nginx-common all 1.14.0-0ubuntu1.2 [37.4 kB]
Get:2 http://security.ubuntu.com/ubuntu bionic-security/universe
libnginx-mod-http-echo amd64 1.14.0-0ubuntu1.2 [21.3 kB]
Get:3 http://security.ubuntu.com/ubuntu bionic-security/universe
nginx-light amd64 1.14.0-0ubuntu1.2 [390 kB]
Fetched 449 kB in 1s (448 kB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 163823 files and directories currently
installed.)
Preparing to unpack .../nginx-common_1.14.0-0ubuntu1.2_all.d
Unpacking nginx-common (1.14.0-0ubuntu1.2) ...
```

Upgrading the System

APT tools maintain a local cache of available packages and their dependencies for faster searching.

You need to run `apt-get update` to update the cache after modifying `sources.list` or to fetch upstream changes.

The `apt-cache` command allows you to:

search term: Search for packages matching a specific term.

show package_name: Display information about a package, including description and dependencies.

showpkg package_name: Show technical details about a package, such as its dependencies and services.

```
# apt-get install cpio
Reading package lists... Done
Building dependency tree
Reading state information... Done
Suggested packages:
  libarchive1
The following packages will be upgraded:
  cpio
```

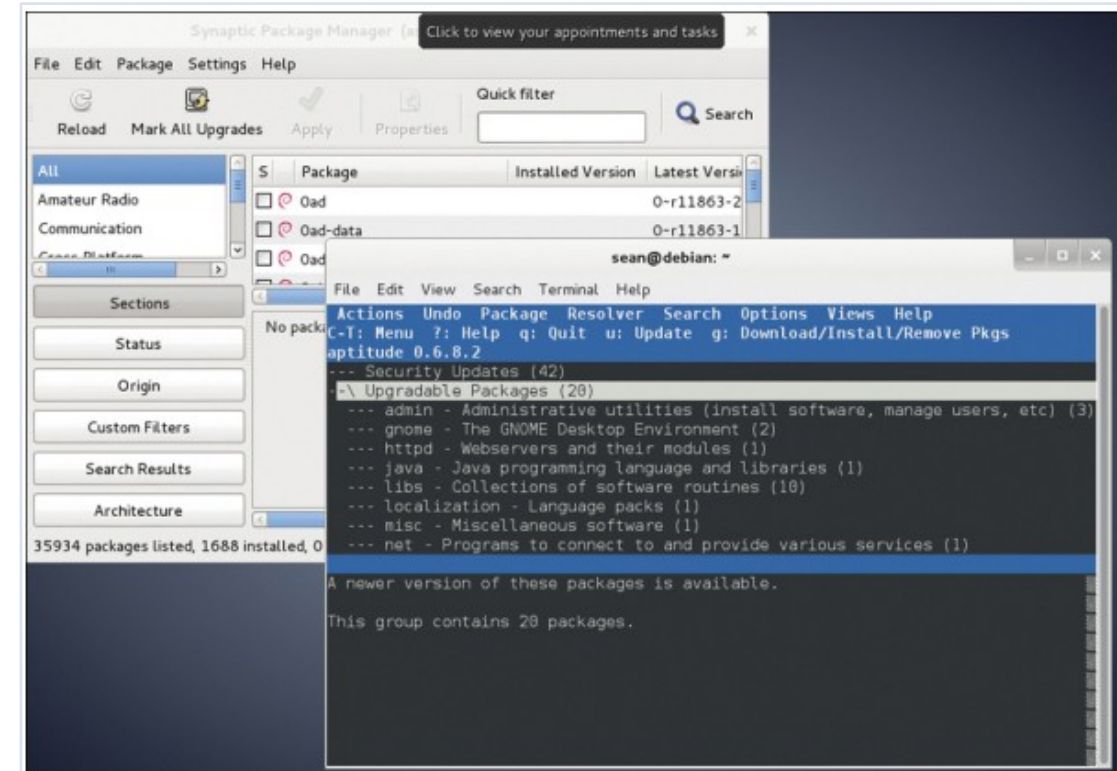
Graphical Managers

Debian offers two graphical package managers: aptitude and synaptic.

Aptitude has a text-based interface.

Synaptic provides a fully graphical interface.

Both are front ends to the APT tools.



The RPM Database

The RPM database is a collection of Oracle Berkeley DB files that track installed packages and their files on a system.

It is located in the `/var/lib/rpm` directory and contains binary files used by the `rpm` command.

Only the root user can modify the RPM database, but unprivileged users can query and verify installed packages.

The RPM database can become outdated or corrupted, requiring a rebuild, which can be done using a specific command as the root user.

The command to rebuild the RPM database produces no output unless errors are found, and the errors are typically informational.

Only one user can access the RPM database at a time to prevent data corruption and simultaneous software installations.

```
warning: waiting for transaction lock on /var/lib/rpm/.rpm.lock
```


Package Name Conventions

Installed packages are usually identified by a short name, though some operations require the use of long package names.

Long package names are typically needed for deletion when multiple packages have matching short names.

To find two packages installed with different versions of the same software, use the appropriate command (though the specific command wasn't provided).

Viewing a specific package's information can be tricky when using options like `-l` and `-i` for listings and info pages.

To see details for just one package, use its long name (e.g., `5.19.0-39-generic.fcXX`) in the query to isolate that package.

```
wireshark
```

Here's an example of a long package name:

```
wireshark-2.6.4-1.fc30
```

Validation of Packages

For security, validate the signatures of packages from non-vendor sources to avoid malware.

RPM checks package integrity using MD5 (checksum) and PGP/GPG (signature tied to a person/organization).

MD5 ensures the package contents are unchanged, while PGP/GPG confirms the package's origin.

Package validation happens during installation or can be triggered explicitly.

"NOT OK" indicates the signatures couldn't be verified.

The issue can be fixed by importing the key from the distribution site.

```
# rpm --import https://getfedora.org/static/8E1431D5.txt  
# rpm -K R-3.5.1-1.fcXX.x86_64.rpm  
R-3.1.2-1.fcXX.x86_64.rpm: rsa sha1 (md5) pgp md5 OK
```

Installation of Packages

Installing software via packages requires root access, the rpm command, and a package.

Before installation, rpm checks:

Enough free disk space.

Existing files won't be overwritten.

All package dependencies are met.

To install a package, use the -i (or --install) option.

```
# rpm -ivh R-3.5.1-1.fcXX.x86_64.rpm
Preparing... ##### [100%]
Updating / installing...
 1: R-3.5.1-1.fcXX ##### [100%]
```


Additional Installation Options

RPM installations may fail if files exist on the system that the package will overwrite, such as from manual installations or failed package operations.

The `--replacefiles` or `--force` options can handle these situations.

The `--force` option overrides safeguards, including the `replacefiles` safeguard, allowing the installation to proceed despite conflicts.

If a file conflict occurs during RPM installation, check the conflicting file.

If the file is unnecessary, remove it.

Alternatively, use the `--force` option to bypass the conflict and proceed with installation.

error: Failed dependencies:

```
id10t = 3.5.1-1.fcXX is needed by pebkac.5.1-1.fcXX.x86_64
loosenut = 3.5.1-1.fcXX is needed by pebkac.5.1-1.fcXX.x86_64
hubproblem = 3.5.1-1.fcXX is needed by pebkac.5.1-1.fcXX.x86_64
```

Verifying a Package's Integrity

Verifying a package may be necessary if users report issues with the software.

It can also be used to check for tampering or potential system integrity problems.

Use the `rpm -V` or `rpm --verify` command to verify an installed RPM package.

This checks if any files installed by the package have been altered or corrupted.

RPM verification returns a series of nine characters indicating different tests:

S: File size differs

```
# rpm -V logrotate
S.5....T. c /etc/logrotate.conf
```

Installing Packages

Install Package with YUM: Use the `yum install` command to install a package, like the R statistical language, from a remote repository.

YUM Output: YUM calculates dependencies, downloads required packages, and installs them. Log files can become large, as shown in the example where the ...

YUM Commands: Unlike RPM commands, YUM commands typically don't use dashes or double dashes.

Automation with YUM: Use the `-y` option to automatically confirm installation, skipping the confirmation step.

Downloading Packages: Instead of installing, you can download packages using:

Answering `d` at the prompt (instead of `y`).

```

java-1.8.0-openjdk-devel x86_64 1:1.8.0.191.b12-8.fcXX upd
java-1.8.0-openjdk-headless
                                x86_64 1:1.8.0.191.b12-8.fcXX upd
perl-XML-XPath                  noarch 1.43-1.fcXX      upd
perl-open                      noarch 1.11-424.fcXX     upd
R-core                         x86_64 3.5.1-1.fcXX    fed
R-core-devel                  x86_64 3.5.1-1.fcXX    fed
R-devel                       x86_64 3.5.1-1.fcXX    fed
R-java                        x86_64 3.5.1-1.fcXX    fed
R-java-devel                  x86_64 3.5.1-1.fcXX    fed
annobin                       x86_64 8.23-1.fcXX     fed
bzip2-devel                   x86_64 1.0.6-28.fcXX    fed
dwz                           x86_64 0.12-9.fcXX     fed
efi-srpm-macros               noarch 3-3.fcXX           fed
expat-devel                   x86_64 2.2.6-1.fcXX    fed
fontconfig-devel              x86_64 2.13.1-1.fcXX   fed
fpc-srpm-macros               noarch 1.1-5.fcXX       fed
freetype-devel                x86_64 2.9.1-2.fcXX    fed
gcc-c++                       x86_64 8.2.1-2.fcXX    fed

Transaction Summary
=====
Install 312 Packages

Total download size: 312 M
Installed size: 706 M
Is this ok [y/N]: Downloading Packages:
Installed:
  R-3.5.1-1.fcXX.x86_64
  perl-File-ShareDir-1.116-3.fcXX.noarch
  perl-List-MoreUtils-0.428-4.fcXX.noarch
  perl-Params-Util-1.07-25.fcXX.x86_64
  java-1.8.0-openjdk-1:1.8.0.191.b12-8.fcXX.x86_64
  java-1.8.0-openjdk-devel-1:1.8.0.191.b12-8.fcXX.x86_64
  java-1.8.0-openjdk-headless-1:1.8.0.191.b12-8.fcXX.x86_64

```

Finding Packages to Install

Use `yum list` to search for packages in remote repositories if you know the package name.

Using Wildcards with YUM:

When packages are split into components, they share a common prefix.

Use the wildcard operator with `yum list` to see related packages (e.g., for Tomcat).

```
# yum list tomcat*
Last metadata expiration check: 0:09:01 ago on Mon 26 Nov 20
11:31:19 AM EST.
Available Packages
tomcat.noarch                                1:9.0.10-1.fcXX
tomcat-admin-webapps.noarch                  1:9.0.10-1.fcXX
tomcat-docs-webapp.noarch                    1:9.0.10-1.fcXX
tomcat-el-3.0-api.noarch                     1:9.0.10-1.fcXX
tomcat-javadoc.noarch                        1:9.0.10-1.fcXX
tomcat-jsp-2.3-api.noarch                    1:9.0.10-1.fcXX
tomcat-jsvc.noarch                           1:9.0.10-1.fcXX
tomcat-lib.noarch                            1:9.0.10-1.fcXX
tomcat-native.x86_64                         1.2.17-2.fcXX
tomcat-servlet-4.0-api.noarch                 1:9.0.10-1.fcXX
tomcat-taglibs-parent.noarch                 3-8.fcXX
tomcat-taglibs-standard.noarch               1.2.5-6.fcXX
tomcat-webapps.noarch                        1:9.0.10-1.fcXX
tomcatjss.noarch                             7.3.5-1.fcXX
```

Configuring YUM

The main configuration file for YUM is `/etc/yum.conf`.

This file contains global options for YUM and defaults for all repositories.

Editing this file is rare but useful for understanding where files are stored.

Example configurations can include default settings for systems like Fedora.

YUM Configuration Directories and Files:

Cached files are stored in `/var/cache/yum`.

```
# CentOS-Base.repo
#
# The mirror system uses the connecting IP address of the client to determine
# the
# update status of each mirror to pick mirrors that are updated
# and
# geographically close to the client. You should use this for system
# updates
# unless you are manually picking other mirrors.
#
# If the mirrorlist= does not work for you, as a fall back you can use the
# try the
# remarked out baseurl= line instead.
#
#

[base]
name=CentOS-$releasever - Base
mirrorlist=http://mirrorlist.centos.org/?release=$releasever
```

Installing Software Packages with zypper

To list available software sources, use the `zypper list repositories` (`zypper lr`) command.

This command shows the configured repositories from which software can be installed.

To refresh metadata from configured repositories, use the `zypper ref` command.

This command updates the repository information on your system, enabling smoother interactions and installations or updates.

To search for a specific package in your repositories, use the `zypper se` command (e.g., `zypper se vim`).

The output will display all packages matching the search term, such as `gvim` when searching for `"vim."`

```
217:~ # zypper lr
Repository priorities in effect:                                     (See 'zypper lr -P' for details)
 99 (default priority) : 5 repositories
100 (lowered priority) : 1 repository

# | Alias | Name | Enabled | GPG Check | Refresh
---|---|---|---|---|---
1 | Basesystem-Module_15-0 | sle-module-basesystem | Yes | (r) Yes | No
2 | Desktop-Applications-Module_15-0 | sle-module-desktop-applications | Yes | (r) Yes | No
3 | Development-Tools-Module_15-0 | sle-module-development-tools | Yes | (r) Yes | No
4 | SLES15-15-0 | SLES15-15-0 | Yes | (r) Yes | No
5 | SLES15_15-0 | SLES | Yes | (r) Yes | No
6 | Server-Applications-Module_15-0 | sle-module-server-applications | Yes | (r) Yes | No
217:~ #
```

Adding a Repository

To add a repository, first list the existing repositories and identify the one you want to add.

Use a web search or another method to find the repository.

For example, adding a non-oss repository (which contains non-open-source software like Opera) follows a specific command format.

The zypper command uses the ar directive to add a repository.

The <uri> is the network URL for querying and downloading packages.

The <alias> is a friendly name for the repository.



```
Activities Terminal Tue 16:41
tux@217:~
File Edit View Search Terminal Help
217:~ # zypper ar http://download.opensuse.org/distribution/leap/15.0/repo/non-oss/ Non-OSS
Adding repository 'Non-OSS' .....[done]
Repository 'Non-OSS' successfully added

URI      : http://download.opensuse.org/distribution/leap/15.0/repo/non-oss/
Enabled  : Yes
GPG Check : Yes
Autorefresh : No
Priority  : 99 (default priority)

Repository priorities in effect:                                     (See 'zypper lr -P' for details)
   99 (default priority) : 6 repositories
  100 (lowered priority) : 1 repository
217:~ #
```

Refreshing the Repository

To get the latest information about a repository, use the `zypper ref` command before querying or installing from it.

New repositories must be accepted, and the package-signing key stored locally.

`zypper` handles this when you choose "y" for "yes this once" or "a" for "always trust this repository."

It's recommended to choose "a" for convenience, as shown in Figure 6-10.

The repository is queried, its information is downloaded, and it's ready for use as an installation and update source.

The package-signing key is stored locally after accepting the repository.



```

Activities Terminal Tue 16:47
tux@217:~
File Edit View Search Terminal Help
217:~ # zypper ref
Repository 'sle-module-basesystem' is up to date.
Repository 'sle-module-desktop-applications' is up to date.
Repository 'sle-module-development-tools' is up to date.
Retrieving repository 'Non-055' metadata .....[\\]

New repository or package signing key received:

Repository:      Non-055
Key Name:        openSUSE Project Signing Key <opensuse@opensuse.org>
Key Fingerprint: 22C07BA5 34178CD0 2EFE22AA 88882FD4 3DBDC284
Key Created:     Mon May 5 02:37:40 2014
Key Expires:     Thu May 2 02:37:40 2024
Rpm Name:        gpg-pubkey-3dbdc284-53674dd4

Do you want to reject the key, trust temporarily, or trust always? [r/t/a/? shows all options] (r): a
Retrieving repository 'Non-055' metadata .....[done]
Building repository 'Non-055' cache .....[done]
Repository 'SLES15-0' is up to date.
Repository 'SLES' is up to date.
Repository 'sle-module-server-applications' is up to date.
All repositories have been refreshed.
217:~ #
  
```


Update with Package Manager

Recommended method: Use the package manager to update the kernel.

Follows the distribution's recommended update path.

Ensures compatibility with the rest of the distribution, as the kernel is packaged with customizations.

Common mistake: Users often accidentally update the entire distribution using too broad a command.

To get the latest package information (example for Ubuntu): Use the appropriate package manager command.

Refresh repositories: Use a command to get the latest repository information.

```
# sudo apt-get upgrade linux-image-generic
```

Linux Kernel Utilities

Linux Kernel Utilities: A set of three scripts to update, compile, and remove old kernels on Ubuntu.

Scripts: `compile_linux_kernel.sh`,
`update_ubuntu_kernel.sh`, `remove_old_kernels.sh`

These scripts help automate the kernel management process.

Step to Retrieve Utilities: Download the utilities from Mark Tompkins' GitHub page and store them in a test directory.

Kernel Update Methods: There are several ways to update the kernel, but the focus is on those requiring a reboot.

Recommended Approach: Stick with methods recommended by your distribution vendor.

Step 2. Make a test directory with `mkdir ~/test`

Step 3. Pull down the Linux kernel utilities with the command:

[Click here to view code image](#)

```
# git clone https://github.com/Tinram/Linux-Utilities
```

Step 4. Change into the `~/test/linux-kernel-utilities` directory and change the permissions with

```
# chmod 750 *.sh
```

Step 5. Then do a git pull.

Step 6. Run the compile script to list the available kernels.

Step 7. Install the latest kernel with the command

[Click here to view code image](#)

```
# ./compile_linux_kernel.sh --latest
```

Step 8. Install the updated kernel with the command

```
# ./update_ubuntu_kernel.sh
```

Step 9. Reboot the system.

Alternatively, you can use the update script with the `--latest` option to get a precompiled version of the kernel from the Ubuntu kernel repository:

[Click here to view code image](#)

```
# ./update_ubuntu_kernel.sh --latest
```

Step 10. Finally, you can remove old versions of the kernel with the remove script:

```
# ./remove_old_kernels.sh
```

Handling .rpmsave and .rpmnew Files

Monitor upgrades for .rpmnew or .rpmsave files: Check log files to confirm the creation of these files during an upgrade.

No service upgrade: If the upgrade doesn't involve a service, there's likely no .rpmnew or .rpmsave file.

Service upgrade: Inspect the service carefully if upgraded and check for .rpmnew or .rpmsave files.

Search for files: Use the find command to locate any .rpmnew or .rpmsave files after an upgrade.

Example 7-1 output: Shows two .rpmsave files and three .rpmnew files, with the most important being the .rpmnew files created due to the noreplace ...

Use diff command: After finding .rpmsave or .rpmnew files, use the diff command to compare the old and new configuration files.

```
# find / -iname "*.rpm[n,s]*" 2> /dev/null  
/home/parallels/file1.rpmnew  
/home/parallels/file2.rpmsave  
/etc/containers/registries.conf.rpmnew  
/etc/cups/cups-browsed.conf.rpmnew
```

Viewing Kernel Parameters

Using sysctl: You can easily view kernel parameters in the running kernel by running a query in the shell.

Example: An example command for viewing kernel parameters is provided in Example 7-2.

Viewing a Specific Parameter: You can use `sysctl` followed by the parameter name to view a particular kernel parameter value, like `sysctl kernel.osrelease`.

Alternative Method: You can also view a parameter's value by directly accessing the corresponding file in `/proc/sys/` by replacing dots with slashes in the ...

```
# sysctl -a
abi.vsyscall32 = 1
debug.exception-trace = 1
debug.kprobes-optimization = 1
```

Software management — More Key Topics

Package Management

Package Definition: A package is a compressed archive containing all necessary files for a command, application, or set of ...

The Most Common Package Types

Common Package Types: .rpm (Red Hat Package Manager)

Package Managers

Purpose of Package Managers: Solve issues related to software version proliferation and leftover files from upgrades or ...

Debian Package Management

Debian and Debian-derived systems: Use a powerful package management system for finding, installing, querying, and removing ...

Adding Repositories

To manually add a repository, you can: Edit the `/etc/apt/sources.list` file.

Working with the Cache

APT tools maintain a local cache of available packages and their dependencies for faster searching. You need to run `apt-get` ...

Software management — More Key Topics

RPM and YUM Package Management

RPM refers to the package format, the rpm command, and the RPM database. The components of RPM package management include:

RPM Package Files

RPM packages come in two types: Binary RPM: Contains compiled binary files, documentation, configuration, and other necessary ...

The rpm Command

The rpm command is used for managing packages on the system, both installed in the RPM database and uninstalled on disk. Common ...

Freshening vs. Upgrading

Daily maintenance of a system involves applying updates and fixes via package updates. The -U and --upgrade options install or ...

Other Removal Options

You might need to force the removal of a package to fix broken dependencies. This should be done under the guidance of ...

Package Management with YUM

YUM Overview: YUM is a front-end to RPM, designed to simplify working with RPM and remote repositories. Ease of Use: Like apt-* ...

Lab 11 — Software & Package Management

Manage Debian packages with apt/dpkg, add a signed third-party repo, run dnf/rpm in a container, and use language and sandboxed package managers.

You'll build — Install/remove apt packages, add a GPG-signed repo, demo dnf/rpm, then pip/npm/cargo and snap/flatpak.

Key commands — apt update/install/remove/purge/autoremove, apt-cache policy, dpkg -I / -L, signed-by GPG keyrings, dnf / rpm -q / rpm ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-11-*/README.md on the course GitHub repo

Software & Package Management

Step 1 — apt basics

```
$ apt update
```

Step 2 — Add a third-party repo with GPG key

```
$ apt install -y curl gnupg lsb-release
```

Step 3 — Remove packages

```
$ apt remove -y tree
```

Step 4 — RPM world inside a container

```
$ apt install -y docker.io
```


Software & Package Management

Step 5 — Language-specific package managers

```
$ apt install -y python3-pip nodejs npm cargo
```

Step 6 — update-alternatives

```
$ update-alternatives --display editor
```

Step 7 — Sandboxed package formats

```
$ which snap && snap list 2>/dev/null || echo "snap not preinstalled on this image"
```

✅ **Verify** — apt-cache policy terraform shows a candidate from the signed HashiCorp repo, confirming the GPG-verified ...

2.5 — Systems management

Start, stop, and review services, logs, and timers.

Exam objective — Given a scenario, manage Linux using systemd

Hands-on — Lab 12

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 12 — Service Management with systemd

Inspect and control systemd units, author custom service/timer/mount units, analyze boot performance, and configure host/time/DNS via systemd helpers.

You'll build — Manage nginx, write a service + timer + mount unit, analyze boot time, and set hostname/timezone/DNS.

Key commands — systemctl list-units / list-unit-files / --failed, start/stop/restart/reload, enable/disable, mask/unmask, custom ...

Workbench — Killercodea Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-12-*/README.md on the course GitHub repo

Service Management with systemd

Step 1 — List and inspect units

```
$ systemctl list-units --type=service | head -20
```

Step 2 — Control nginx

```
$ apt update -y && apt install -y nginx
```

Step 3 — Author a custom service

```
$ echo "lab-hello fired at $(date)" >> /var/log/lab-hello.log
```

Step 4 — Add a timer

```
$ [Unit]
```

Service Management with systemd

Step 5 — Mount unit example

```
$ mkdir -p /mnt/labdata
```

Step 6 — Boot performance

```
$ systemd-analyze
```

Step 7 — Host, time, and DNS

```
$ hostnamectl
```

✅ **Verify** — `systemctl list-timers` shows `lab-hello.timer` scheduled and `journalctl -u lab-hello` records each firing, ...

2.6 — Containers

Operate container runtimes, manage images, and create networks.

Exam objective — Given a scenario, manage applications in a container

Hands-on — Lab 13

Type it, break it, fix it — the exam rewards hands that have done the work.

Installing the Container-Tools Package and Dependencies

```
# sudo yum module install container-tools
```

So enter y to answer yes and allow the install to finish

```
Installing dependencies:
cockpit-bridge                x86_64                264.1-1.el8                rhel-8-baseos-rhui-rpms                534 k
fuse3                          x86_64                3.3.0-15.el8                rhel-8-baseos-rhui-rpms                54 k
fuse3-libs                     x86_64                3.3.0-15.el8                rhel-8-baseos-rhui-rpms                95 k
libnet                         x86_64                1.1.6-15.el8                rhel-8-appstream-rhui-rpms            67 k
nftables                      x86_64                1:0.9.3-25.el8                rhel-8-baseos-rhui-rpms                324 k
podman-catatonit              x86_64                2:4.0.2-6.module+el8.6.0+14877+f643d2d6 rhel-8-appstream-rhui-rpms            354 k
protobuf-c                    x86_64                1.3.0-6.el8                  rhel-8-appstream-rhui-rpms            37 k
python3-pytoml                noarch                0.1.14-5.git7dea353.el8      rhel-8-appstream-rhui-rpms            25 k
shadow-utils-selinux          x86_64                2:4.6-16.el8                rhel-8-baseos-rhui-rpms                112 k
Installing module profiles:
container-tools/common
Enabling module streams:
container-tools                rhel8

Transaction Summary
-----
Install 26 Packages

Total download size: 55 M
Installed size: 190 M
Is this ok [y/N]:
```

Finding Suitable Images:

Container images serve different purposes, such as hosting a database or running a web server. To find the right image, you can search repositories for ...

Searching for UBI (Universal Base Images) in RHEL 8:

To search for UBI images for RHEL 8

Finding and Pulling a Container Image

```
# podman search registry.access.redhat.com | grep -i latest
```


This displays the listing of available “latest release” images, which ...

Finding and Pulling a Container Image

registry.access.redhat.com/ubi9-beta/ubi	Provides the latest release of Red Hat Universal Base Image 9.
registry.access.redhat.com/ubi8/ubi	Provides the latest release of the Red Hat Universal Base Image 8
registry.access.redhat.com/ubi8/ubi-micro	Provides the latest release of Micro Universal Base Image 8
registry.access.redhat.com/ubi9-beta/ubi-minimal	Provides the latest release of the Minimal Red Hat Universal Base Image 9.
registry.access.redhat.com/ubi8/ubi-minimal	Provides the latest release of the Minimal Red Hat Universal Base Image 8.
registry.access.redhat.com/ubi8/ubi-init	Provides the latest release of the Red Hat Universal Base Image 8 Init for

Using skopeo to View Container Image Information:

The skopeo tool allows you to inspect container images and their metadata without pulling them. This is useful for getting detailed information about an ...

Finding and Pulling a Container Image

Pulling an Image

```
"description": "The Universal Base Image is designed and engineered to be the base layer for all of your containerized applications, middleware and utilities. This base image is freely redistributable, but Red Hat only supports Red Hat technologies through subscriptions for Red Hat products. This image is maintained by Red Hat and updated regularly.",  
  "distribution-scope": "public",
```

Pull the image from the repository by using the podman pull command

Pulling an Image

```
# podman pull registry.access.redhat.com/ubi8/ubi:latest
Trying to pull registry.access.redhat.com/ubi8/ubi:latest...
Getting image source signatures
Checking if image destination supports signatures
Copying blob 0c673eb68f88 done
Copying blob 028bdc977650 done
Copying config 2fd9e14788 done
Writing manifest to image destination
Storing signatures
2fd9e1478809c0c53820517b998e1b2a044207515e5d61eccb6295276bbbaffc
```

Inspecting a Pulled Container Image with podman images:

Once you have pulled a container image, you can inspect it by listing all the images available on your host system. This can be done using the following ...

Viewing Local Images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
registry.access.redhat.com/ubi8/ubi	latest	2fd9e1478809	3 week	225MB

Running a Container with podman run:

To run a container and interact with its command line, use the podman run command with the following parameters:

-i: Runs the container in interactive mode.

-t: Attaches a pseudo-terminal (pty), enabling terminal interaction.

```
podman run -i -t 2fd9e1478809
```

Running an Image as a Container

Verifying the Container is Still Running:

To check if the container is still running, use the `podman ps` command (instead of `podman images`): `podman ps`

Detaching from and Attaching to Containers

```
# podman images
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS         NAMES
35c54f588bbc   registry.access.redhat.com/ubi8/ubi:latest
/bin/bash     About 5 minutes ago    Up 5 minutes ago
compassionate_chatelet
```

Example Output:

The output will show the status of all containers, whether running or exited:

Exiting and Ending Execution of a Container

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

ae054514e8f6 registry.access.redhat.com/rhsc/nginx-16-rhel7 nginx -g daemon o... 32 seconds ago Up 31 seconds ago 0.0.0.0:36319->80/tcp, ...

52ee615ae4b7 registry.access.redhat.com/rhsc/n

Exposing and Mapping Ports

```
# podman ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
35c54f588bbc	redhat.com/ubi8/ubi:latest	/bin/bash	About a minute ago	Exited (0) 6 seconds ago		infallible_khayyam

Exposing and Publishing Ports:

To expose a specific port (e.g., port 80) on a container and publish all exposed ports to the host, you can use:

`--expose=80`: Exposes port 80 inside the container.

`--publish-all`: Publishes all exposed ports to the host.

`&`: Runs the container in the background.

Exposing and Mapping Ports

```
# podman run --expose=80 --publish-all registry.access.redhat.com/  
rhsc1/nginx-16-rhel7 &
```


build Command

Create a Containerfile.

Create a script to run as a test.

Set the permissions of the script.

Build the image with the buildah command.

Verify the image with the buildah images command.

Run the image with the podman run command

Containers — More Key Topics

What is a Container?

A lightweight, standalone, executable package of software. Includes everything needed to run an application: code, runtime, ...

Verifying Podman and Skopeo Installation:

A simpler way to verify that Podman and Skopeo are installed and available in your system's PATH is to use the which command: ...

Expected Output: /usr/bin/podman

/usr/bin/skopeo This output shows the location of the executable files for Podman and Skopeo in the system's PATH, confirming ...

Inspecting an Image with podman inspect:

To get detailed information about a container image, you can use the podman inspect command with the shortened Image ID (the ...

Visual Indication:

After running the container, your prompt will change to reflect that you're inside the container. It will show the container's ID ...

Verifying You Are in the Container:

To confirm you're inside the container, you can check the contents of the home directory or check the host release information. ...

Containers — More Key Topics

Assigning a Friendly Name to a Container:

You can assign a friendly name to a container when creating it by using the `--name` option. This makes it easier to reference the ...

After running this command, you will be inside the ...

`[root@35c54f588bbc /]# Assigning a Container-Friendly Name`

Renaming a Container:

If you want to rename a container after it's been created, use the `podman rename` command: `podman rename OpenContainer ...`

Detaching from a Running Container:

When you run a container with the `-it` option, typing `exit` will terminate the container. To detach from the container without ...

Exiting an Interactive Container Session:

To exit from an interactive container session and terminate the container, simply type: `exit` This will stop the container and ...

Verifying Container Status:

To check the status of all containers, including those that have stopped, use the `podman ps -a` command: `podman ps -a` Exiting and ...

Lab 13 — Containers with Docker & Podman

Install Docker and Podman, pull/run images, manage volumes and networks, build an image from a Dockerfile, and contrast rootful vs rootless engines.

You'll build — Run and inspect nginx containers, bind-mount volumes, build an image, wire a user-defined network, and try rootless ...

Key commands — docker.io vs podman (daemonless/rootless), docker pull/run -d -p --name, docker ps, logs / exec -it / inspect / stats, ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-13-*/README.md on the course GitHub repo

Containers with Docker & Podman

Step 1 — Install Docker and Podman

```
$ apt update -y
```

Step 2 — Pull and run a container

```
$ docker pull nginx
```

Step 3 — Inspect, exec, logs

```
$ docker logs web | head
```

Step 4 — Volumes for persistent data

```
$ mkdir -p /data
```

Containers with Docker & Podman

Step 5 — Build a tiny image

```
$ mkdir -p /tmp/hello && cd /tmp/hello
```

Step 6 — Networks and port mapping

```
$ docker network create labnet
```

Step 7 — Rootless Podman and privileged note

```
$ podman run --rm alpine echo "hello from podman"
```

✅ **Verify** — curl against the published ports returns 200 and the alpine container reaches http://api by name, confirming ...

Recap — Services and User Management

Lab 8 — File & Directory Management

Lab 9 — Local Account & Group Management

Lab 10 — Processes, Jobs & Scheduling

Lab 11 — Software & Package Management

Lab 12 — Service Management with systemd

Lab 13 — Containers with Docker & Podman

End of Day 2

See you tomorrow — 9:30 AM

Security

18% of the exam | Objectives 3.1–3.6 | Labs 14–19

Type it, break it, fix it — the exam rewards hands that have done the work.

Security — Objectives

3.1 · Auth & accounting

Configure PAM, LDAP, Kerberos, and enable auditing.

3.2 · Firewalls

Set firewalls using iptables, nftables, UFW, and zone rules.

3.3 · OS hardening

Apply permissions, configure sudo, and secure remote access.

3.4 · Account security

Enforce password policies, restrict shells, and enable MFA.

3.5 · Cryptography

Encrypt files, use hashing, and manage certificates.

3.6 · Compliance

Verify integrity, run scans, and maintain standards.

Key Concepts — Security

AAA: authenticate (PAM/SSSD), authorize (sudo, Polkit) and account (journald, auditd) for every action.

Firewalls filter at every layer — ufw → firewalld → nftables/iptables — with stateful inspection and NAT.

OS & account hardening: permissions/ACLs, SELinux, SSH hardening, fail2ban, password policy and MFA.

Cryptography (GPG, LUKS2, OpenSSL, WireGuard) and compliance/audit (AIDE, rkhunter, OpenSCAP, CIS) protect data.

3.1 — Auth & accounting

Configure PAM, LDAP, Kerberos, and enable auditing.

Exam objective — Summarize authorization, authentication, and accounting methods

Hands-on — Lab 14

Type it, break it, fix it — the exam rewards hands that have done the work.

Permission Trio Bits

The ten positions (one type bit and three trios of permission bits) that make up the permissions section of a file.

The rest of the 10 bits is the type of the object:

.: Indicates that this is a normal file.

l: Indicates that this is a symlink (symbolic link), which is a file that points to another object.

b: Indicates that this is a block device file.

c: Indicates that this is a character device file.

T y p e	User			Group			Other		
	r	w	x	r	w	x	r	w	x
	4	2	1	4	2	1	4	2	1

Octal Mode

The octal notation method uses bits (4 for read, 2 for write, 1 for execute) to set permissions. For example, if the current permissions are 644 and you ...

The chmod command works great on groups of files, too:

The octal notation method also works on directories and their contents. For instance, if the user "bertrandr" wants to set all files in a directory to ...

The chmod command with the -R option changes permissions recursively for the specified object and its contents. In this case, the user "bertrandr" wants ...

The chmod command has several useful options:

-c: Reports only the files that were changed.

```
# chmod -R 640 /home/bertrandr/data/*
```

Changing File Ownership

The **chown** command is used to set the user owner, group owner, or both with a single command. The format for using **chown** is

The **chown** command accepts both of the following formats as valid for specifying the owner and group, where the separator can be either a colon (:) or a ...

The **chown** command also supports the following syntaxes:

owner: Changes only the user owner.

owner:group: Changes both the user owner and the group owner.

owner::: Changes the user owner and sets the group owner to the current user's primary group.

```
# chown snuffy:users file1  
# chown snuffy.users file1
```

Changing Group Ownership

When only the group owner needs to be changed, the simplest method is to use the **chgrp** command. The syntax is straightforward

Changing a large set of files to another group owner requires the use of the **-R** (recursive) option

The **chgrp** command can be used to change the group owner of the **/data** directory and all its subdirectories to "staff," without altering the user owner. The ...

-c: Shows output only for changed objects.

-h: Changes symbolic links, not the original file.

-R: Recursively affects the target and all its children.

```
# chgrp -R staff /data/*
```


Permission Granularity Issues

In Linux and Unix file systems, certain limitations can create challenges for administrators. For instance, without using Access Control Lists (ACLs), ...

To access the contents of a file, a user must either be the file owner (e.g., "bob") or a member of the group that owns the file (e.g., "staff").

Managing access to files for multiple users in various groups can be challenging, especially when users belong to different primary and secondary groups. ...

The bob user creates a file that ends up looking like the following when listed with the `ls -l` command

Although all users are in the same "project" group, "sue" and "steve" cannot view the file because they are not members of the "staff" group. As a result, ...

The "bob" user can change the group ownership of the file to the "project" group, but this would require users to perform the action every time they ...

```
-rw-r----- 1 bob staff 0 Jan 26 13:08 22
```

```
data_for_sue_and_steve.txt
```

Special Bit Permissions

The Linux filesystem offers three special bits:

SUID (Set User ID): Allows a user to run a program as if they were the user owner, typically root. The numeric value is 4XXX, where XXX represents the ...

SGID (Set Group ID): When set on a directory, it gives new files the group ownership of the directory's group owner. When set on a file, it allows the ...

Sticky bit: Prevents non-owners from deleting files in a shared directory. Only the file owner or directory owner can delete the file (root can delete any ...

Special permissions like SUID and SGID should only be used when necessary, as they can pose security risks, especially on executable files. It allows ...

Special permissions can be set using either the numeric or symbolic method, just like other file permissions, through the chmod command. Both methods ...

```
chmod 4XXX /some/program
```

Setting the SUID Bit on Files

A command related to the SUID bit is `chsh`. The `chsh` command allows a regular user to change her login shell.

The "s" in place of the user owner's execute permission indicates that the command has the SUID (Set User ID) permission set. When executed, this command ...

With the SUID permission set, the `chsh` program can access files either as the user running the command or as the owner of the `chsh` program (typically ...

The "s" in place of the group owner's execute permission indicates that the SGID (Set Group ID) permission is set. The `wall` command, which allows users to ...

Users can use the `wall` command because it is an SGID program. When executed, it accesses files as if the user were a member of the "tty" group, which ...

If you are having problems with users abusing the `wall` command, simply take away the SGID access

```
# ls -l /dev/tty1  
crw--w---- 1 root tty 4, 1 Jun 2 17:13 /dev/tty1
```

Setting the Sticky Bit

The sticky bit is used mostly for ensuring that users in a shared directory can't delete anyone else's files. A good example of this is the /tmp directory

The t in place of the others execute permission indicates that this is a sticky bit directory. The /tmp directory is a location where all users must be ...

When the sticky bit is set on a directory, the meaning of the write permission changes. Instead of allowing any user to add or delete files in the ...

The owner of the file

The owner of the directory (usually root)

The root user

```
drwxrwxrwt 29 root 4096 Jun 2 17:27 /tmp
```

Finding Files by Permission

Attackers who breach a system often try to set certain files with the SUID or SGID bits enabled. This allows their otherwise normal accounts to perform ...

The find command has an operator that makes it easy to find vulnerable files. The -perm operator is usually followed by a three- or four-digit set of ...

This searches for any object in the /usr/bin directory, and all subdirectories, that has the exact permissions rwxrwxrwx. This is fine, but it doesn't ...

At first glance, the find command with the -perm operator might seem like it's searching for files with the SUID bit set and no additional permissions. ...

You should install the system and then run a find command that reports any file that has an SUID bit set, like so

Redirect the output of the find command to a file and store it in a safe location, such as on a USB drive. Periodically (especially after installing new ...

```
# diff /root/latestfindperm /mnt/usb/findperm.orig
```

Setting an ACL

Using the `setfacl` command is fairly easy, though things can get complex if you are trying to set multiple groups to access an object with different ...

The example sets user permissions (u) for the user `snuffy` and adds the permissions to read and write but not execute for this object. The `-m` option ...

By setting the ACL, the user `snuffy` is granted read and write (rw) access as a user owner, without affecting the original owner, `ross`. This change can be ...

In the output of the `getfacl` command, you will see a new line detailing the permissions granted to the user `snuffy`, in addition to the default permissions ...

How to set an ACL for a group owner? You simply change the `u` in the command to `g` and replace the user `snuffy` with the proper group and alter the ...

Now retrieve the permissions again with

```
$ getfacl test
# file: test
# owner: ross
# group: users
user::rwx
user:snuffy:rw-
group::rwx
group:itcrowd:r--
mask::rwx
other::rwx
```

Security-Enhanced Linux (SELinux)

This section covers two commonly used context-based methods: SELinux and AppArmor.

SELinux (Security-Enhanced Linux) is a security framework integrated into the Linux kernel and distributions. It offers an added layer of access control ...

SELinux enforces detailed access control policies that limit actions processes and users can take. It extends traditional Linux permissions by assigning ...

An SELinux security policy requires processes to be part of an SELinux security context to access files and directories. While regular permissions are ...

A key concern addressed by SELinux policies is the security risk posed by daemon (or system) processes. For example, a web server process, among other ...

In this output, each line represents a running Apache web server process (/usr/sbin/httpd). The first part of the line shows the user who initiated the ...

```
# ls -l /etc/passwd
-rw-r--r-- 1 root root 2690 Dec 11 2022 /etc/passwd
```

SELinux Mode

Disabled: SELinux is completely inactive, and no checks are performed on file or directory access.

Enforcing: SELinux actively checks access and blocks unauthorized attempts to access files or directories.

Permissive: SELinux checks access but does not block any actions; it only logs potential violations, useful for troubleshooting.

Use the `getenforce` command to determine the current SELinux mode.

The result `Enforcing` means SELinux is installed, and the security policy is currently active.

You can change the SELinux mode using the `setenforce` command, which is helpful for testing new policies or troubleshooting SELinux-related issues.

```
# cat /etc/selinux/config
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#     enforcing - SELinux security policy is enforced.
#     permissive - SELinux prints warnings instead of enforcing.
#     disabled - No SELinux policy is loaded.
SELINUX=disabled
# SELINUXTYPE= can take one of these two values:
#     targeted - Targeted processes are protected,
#     mls - Multi Level Security protection.
SELINUXTYPE=targeted
```


SELinux Booleans

While creating a policy is beyond the scope of the Linux+ exam, you can modify a policy's behavior using booleans. A boolean is a true/false value that ...

An SELinux policy typically has dozens of different booleans. To determine what a boolean is used for, use the semanage command.

In the example, the xdm_sysadm_login boolean controls whether the root user can log in via the GUI using XDM (a display manager). The default and current ...

You can also use the getsebool command to display just the current value of a boolean.

When a boolean is provided as an argument, the getsebool command gives only that boolean's current value.

To set an SELinux boolean as either on or off, use the setsebool command.

```
# semanage boolean -l | head
SELinux boolean          State  Default Description

privoxy_connect_any      (on   ,  on)   Allow privoxy to connect
smartmon_3ware           (off  ,  off)  Allow smartmon to 3ware
mpd_enable_homedirs      (off  ,  off)  Allow mpd to enable hom
xdm_sysadm_login         (off  ,  off)  Allow xdm to sysadm log
xen_use_nfs              (off  ,  off)  Allow xen to use nfs
mozilla_read_content     (off  ,  off)  Allow mozilla to read c
ssh_chroot_rw_homedirs   (off  ,  off)  Allow ssh to chroot rw
mount_anyfile            (on   ,  on)   Allow mount to anyfile
```

SELinux Contexts

SELinux enhances file and directory security by applying SELinux contexts, which determine whether processes can access certain files. Each file has a ...

An SELinux label, or security context label, uniquely identifies and categorizes system resources like files, processes, and sockets. Each process runs ...

The security context (e.g., system_u:system_r:httpd_t:s0) in SELinux is made up of various components, but the key part for understanding is httpd_t, ...

Files and directories also have an SELinux security context that is defined by the policy. To see a security context for a specific file, use -Z option to ...

The SELinux context contains extensive information, so much so that file names may not fit on a single line. A process with the context ...

The context u:object_r:httpd_sys_content_t:s0 is automatically applied to files placed in certain directories, such as /var/www/html. When a new file is ...

```
# ps -eZ | grep httpd | head -2
system_u:system_r:httpd_t:s0 root      1109      1  0  2022 ?
00:51:56 /usr/sbin/httpd
system_u:system_r:httpd_t:s0 apache  1412  1109  0 Dec24 ?
00:00:09 /usr/sbin/httpd
```

Auth & accounting — More Key Topics

Sysadmin

As a sysadmin, controlling access to system resources is crucial. Before ACLs (Access Control Lists), Linux lacked the ability to ...

File Permissions

Permissions control how users access resources on a system. System security is configured using the user's UID (user ID), GIDs ...

Manipulating Permissions

The chmod command is used to modify an object's permission bits, and only the root user or the object's owner can change ...

Symbolic Mode

When using the chmod command with symbolic values, you can change all permissions like in numeric mode, but it requires a longer ...

File and Directory Ownership

Ownership is crucial in Linux/Unix systems, as every object must have at least a user owner and a group owner (distinct from ...

Understanding and Using umask

The umask command is used to specify default permissions for when a new file or directory is created.

Auth & accounting — More Key Topics

Setting the SGID Bit on Files

The SGID permission is rarely set on les. Consider the following program with the SGID bit set

Setting the SGID Bit on Directories

Consider the earlier scenario in which the bob user (primary group staff, secondary group project), the sue user (primary group ...

Viewing and Changing File Attributes

In addition to file permissions, file attributes are "advanced settings" for files or directories on the filesystem. You can ...

Displaying File Attributes

The primary way to view a file's attributes is by using the lsattr command. It displays a line of attribute notations, typically ...

Key File Attributes

There are several important file attributes that significantly affect files on a filesystem: a: The file can only be appended to, ...

Setting File Attributes

Setting an attribute on a file couldn't be simpler Now if you were to display the le's attributes again, you'd see

Lab 14 — AAA: sudo, PAM, and Polkit

Configure sudo authorization, inspect PAM authentication stacks, and enable auditd accounting to record security events. Finish with a conceptual look at ...

You'll build — Install sudo, grant a user privileges, inspect PAM, read auth logs, and enable auditd rules.

Key commands — sudo / visudo / sudoers.d, PAM stacks (auth, account, session), auditctl / ausearch, journalctl / rsyslog / logrotate, ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-14-*/README.md on the course GitHub repo

AAA: sudo, PAM, and Polkit

Step 1 — Install sudo and create a user

```
$ apt update && apt install -y sudo
```

Step 2 — Grant sudo via group and drop-in file

```
$ usermod -aG sudo alice
```

Step 3 — Switch identity with sudo and su

```
$ su - alice -c 'sudo -n systemctl status ssh || true'
```

Step 4 — Inspect PAM stacks

```
$ ls /etc/pam.d/
```

AAA: sudo, PAM, and Polkit

Step 5 — Read auth logs via journald and rsyslog

```
$ apt install -y rsyslog
```

Step 6 — Enable auditd and watch /etc/passwd

```
$ apt install -y auditd audispd-plugins
```

Step 7 — Persistent audit rules and Polkit note

```
$ -w /etc/passwd -p wa -k passwd_changes
```

✓ **Verify** — `ausearch -k passwd_changes` returns the recorded write event, confirming accounting is active.

3.2 — Firewalls

Set firewalls using iptables, nftables, UFW, and zone rules.

Exam objective — Given a scenario, configure and implement firewalls

Hands-on — Lab 15

Type it, break it, fix it — the exam rewards hands that have done the work.

nftables: Newer, Tighter, More Dynamic

The nftables package is a newer implementation for packet classification in the kernel, designed to replace the older iptables system while still ...

Available since kernel version 3.13, nftables offers a new command structure, improved implementation, and fewer vulnerabilities, specifically addressing ...

Using nftables helps reduce code duplication and prevents issues from inconsistent ruleset application.

It's faster due to improved packet mapping and simplifies managing both IPv4 and IPv6 stacks. It also supports more dynamic rule updates.

One standout feature is the ability to convert iptables commands into nftables-compatible commands using the iptables-translate tool, making transitions ...

As you can see, the translation is done immediately and you can use that to configure your nftables ruleset.

```
# iptables-translate -A INPUT -p tcp --dport 22 -m conntrack  
--ctstate NEW -j ACCEPT  
nft add rule ip filter INPUT tcp dport 22 ct state new counter  
accept
```

Firewalld: Newer, Flexible, Easier to Use

On most RPM- and Red Hat-based distributions, firewalld is used to configure iptables rules. In the latest release of Red Hat, however, firewalld now ...

For example, you can use `firewall-cmd --get-zones` to display all available zones.

You can apply different rules to different zones and assign these zone rule sets to specific interfaces. For instance, if your `eth0` interface connects to ...

```
# firewall-cmd --get-zones
block dmz drop external home internal public trusted work
```

Using iptables to Filter Incoming Packet

A common firewall task is configuring a system to allow or block incoming packets, which can apply to either a single host or an entire network if the ...

Modern Linux distributions often come with default firewall rules. To view the current firewall rules, you can execute the appropriate command.

At this stage, the specific details of the default rules are not important, as they will be explained later. To remove these rules, you can delete them ...

To block all network packets that originate from a specific host, use the following commands:

The -s option stands for “source.” This value can be either an IP address or a network:

You can use the -A option to add a new rule to the end of the chain. Since rules are evaluated in order, this is important to consider. If you want to ...

```
root@localhost:~# iptables -I INPUT 2 -s 192.168.20.125 -j AC
root@localhost:~# iptables -L INPUT
Chain INPUT (policy ACCEPT)
target      prot opt source                destination
DROP        all  --  192.168.10.100         anywhere
ACCEPT      all  --  192.168.20.125         anywhere
DROP        all  --  192.168.20.0/24        anywhere
```

Filtering by Protocol

It's common to filter packets by protocol, such as ICMP, TCP, UDP, or a protocol tied to a specific port (e.g., Telnet on port 23). To block a protocol ...

The /etc/protocols file lists protocols that can be used with the -p option. To block a specific port, use the -m option along with either --sport (source ...

The -m option is necessary for iptables to utilize an extension module, which adds optional features to iptables. In the previous example, the TCP match ...

The iptables -L command automatically converts port numbers into names (like 23 to Telnet) and IP addresses into hostnames if DNS lookup is possible. To ...

Remember that you can also look up port numbers in the /etc/services file. Recall that ports 1 through 1023 are considered privileged ports.

```
root@localhost:~# iptables -L INPUT -n
Chain INPUT (policy ACCEPT)
target      prot opt source                destination
DROP        icmp -- anywhere              anywhere
DROP        tcp  -- anywhere              anywhere tcp dpt:23
```

Filtering Based on Destination

If you look at the output of the `iptables -L INPUT` command, you will see that there is a destination column

When you have multiple network cards or IP addresses on a single card, you may need different rules for each network interface. You can specify rules for ...

To view network ports in the output of the `iptables -L INPUT` command, you need to use the `-v` (verbose) option, which provides more detailed information, ...

The `-v` option in `iptables` gives more detailed information, such as the number of packets that have matched a specified rule, which is useful for testing ...

DROP

```
root@localhost:~# iptables -A INPUT -i eth1 -s 192.168.200.0,
```

DROP

```
root@localhost:~# iptables -L INPUT
```

```
Chain INPUT (policy ACCEPT)
```

target	prot	opt	source	destination
DROP	all	--	192.168.100.0/24	anywhere
DROP	all	--	192.168.200.0/24	anywhere

Saving the Rules

So far, all changes made to the firewall have only affected the current session. If the system were rebooted, those changes would be lost, and the ...

You can save the current firewall rules to a file using the `iptables-save` command. By default, the output is displayed on the screen, but you can redirect ...

The location where firewall rules are saved and how they are automatically loaded depends on the Linux distribution. Some distributions use front-end ...

If you are not using one of these utilities, there may be another solution enabled on your system. If not, you can just create a shell script that ...

```
root@localhost:~# iptables-restore> /etc/iptables.rules
```

Using iptables to Filter Outgoing Packets

Blocking outgoing packets can be a crucial security measure in organizations to prevent users from accessing potentially harmful sites. For example, if a ...

The DROP target silently discards packets without notifying the sender, making it appear as if the destination is unresponsive. In contrast, the REJECT ...

Perhaps you want to allow this access but create a log entry so you can determine what systems are attempting to access the remote system:

```
root@localhost:~# iptables -F OUTPUT
root@localhost:~# iptables -A OUTPUT -m tcp -p tcp -d 10.10.10.10 --dport 80 -j LOG
root@localhost:~# iptables -L OUTPUT
Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
LOG        tcp  --  anywhere              10.10.10.10
tcp dpt:http LOG level warning
```

Implementing NAT

There are several different forms of NAT:

DNAT (Destination NAT) rules can be used to provide access from an external network, typically applied at the PREROUTING filtering point.

SNAT (Source NAT) is used when an internal network with static private IP addresses needs to access the Internet via a machine with a live IP address. ...

MASQUERADE is similar to SNAT but is used for internal networks with dynamically assigned private IP addresses, like those assigned via DHCP. MASQUERADE ...

MASQUERADE is more commonly used than SNAT, especially in networks where DHCP assigns IP addresses, as it simplifies configuration. Unlike SNAT, which ...

NAT alone isn't sufficient; you also need to enable IP forwarding in the kernel. This involves setting the value of certain files to 1 to allow the system ...

```
root@localhost:~# iptables -t nat -A POSTROUTING -j MASQUERADE
```


The fail2ban Service

The fail2ban daemon monitors specific log files for IP addresses that repeatedly attempt to breach the system. It is not usually installed by default, so ...

After installing fail2ban, a new configuration directory is created at /etc/fail2ban. The main configuration file is /etc/fail2ban/jail.conf, but it ...

The issue with modifying the jail.conf file directly is that updates to the fail2ban software could overwrite it. To avoid this, you can create .local ...

In addition to the global settings, there are sections for different jails. For example, you can have a section that relates to SSH connections, like the ...

This is a basic example, but the jail.conf file allows for many other features. For instance, you can create custom actions, such as sending an email ...

The action setting in fail2ban defines what actions to take when a rule is triggered. These actions are typically specified in the ...

```
# HOW TO ACTIVATE JAILS:
#
# YOU SHOULD NOT MODIFY THIS FILE.
#
# It will probably be overwritten or improved in a distributed
update.
#
# Provide customizations in a jail.local file or a jail.d/
customisation.local.
# For example to change the default bantime for all jails and
enable the
# ssh-iptables jail the following (uncommented) would appear
.local file.
# See man 5 jail.conf for details.
```

Firewalls — More Key Topics

Firewall

A firewall is a hardware or software device designed to control network traffic by allowing or blocking it. Firewalls can be ...

Common Firewall Technologies

The firewall landscape has evolved significantly, with new options like firewalld and nftables entering the field. While iptables ...

iptables: Old and Reliable, but Complicated

The iptables package has been a key part of Linux system security since 1998. It runs in userspace and configures IP packet ...

UFW: Uncomplicated Indeed

On Debian-based distributions, you can use ufw (Uncomplicated Firewall), which serves as a front-end interface to configure ...

Understanding iptables

To create firewall rules on a system, you can use the iptables command. Block network packets (e.g., close open network ports).

Overview of Filtering Packets

Before creating rules with the iptables command, it's important to understand how the firewall service works. When a packet is ...

Firewalls — More Key Topics

Packet filtering - incoming packets

If the packet passes the PREROUTING filtering point, the kernel then determines whether the packet is meant for the local system ...

Packet filtering - routed packets

Implementing and Configuring Firewalls

Packet filtering - outbound packets

Packets are filtered not only when they are sent to a host but also when they are sent from the host. Rules can be applied at the ...

Important Terms

You learned filtering points, where rules can be applied. These rules, grouped into tables (filter, NAT, or mangle), define ...

Multiple Criteria

You can combine multiple criteria in a rule, and for the rule to match, all criteria must be met. For example, a rule can be ...

Changing the Default Policy

A common firewall setup is to allow specific packets and deny all others by default. This can be achieved by changing the default ...

Lab 15 — Firewalls: ufw, firewalld, iptables, nftables, ipset

Configure firewalls at every layer of the Linux netfilter stack, from the ufw wrapper down to iptables and nftables, and understand stateful inspection. Explore ...

You'll build — Configure ufw, firewalld zones, nftables, ipset, NAT rules, and a stateful conntrack pattern.

Key commands — ufw allow/deny/enable, firewalld zones and rich rules, iptables / nftables chains, ipset hash:ip, SNAT / DNAT / ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-15-*/README.md on the course GitHub repo

Firewalls: ufw, firewalld, iptables, nftables, ipset

Step 1 — Install and configure ufw

```
$ apt update && apt install -y ufw
```

Step 2 — Explore iptables backing ufw

```
$ iptables -L -n -v
```

Step 3 — firewalld zones and rich rules in Rocky

```
$ apt install -y docker.io
```

Step 4 — nftables: table, chain, and a rule

```
$ apt install -y nftables
```

Firewalls: ufw, firewalld, iptables, nftables, ipset

Step 5 — ipset for high-performance IP lists

```
$ apt install -y ipset
```

Step 6 — SNAT vs DNAT and masquerade

```
$ sysctl -w net.ipv4.ip_forward=1
```

Step 7 — Stateful inspection demo

```
$ iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

✅ **Verify** — ufw status and nft list ruleset show the loaded rules; conntrack states drive stateful acceptance.

3.3 — OS hardening

Apply permissions, configure sudo, and secure remote access.

Exam objective — Given a scenario, apply OS hardening techniques

Hands-on — Lab 16

Type it, break it, fix it — the exam rewards hands that have done the work.

Password Policies

By default, Linux requires a minimum password length of six characters, which is relatively weak given today's powerful computing resources. While it ...

PAM uses the `/etc/pam.conf` configuration file by default, and the configuration of services is laid out in lines or rules that follow this format

PAMs can additionally use service configuration files located in the `/etc/pam.d` directory, with each file being the configuration for a given service. In ...

```
service  type  control  module-path  module-arguments
```


Password Length

A PAM rule can govern password length, such as setting a ten-character requirement. This is done by editing the `/etc/pam.d/common-password` file and ...

So that it reads

```
password [success=2 default=ignore] pam_unix.so obscure sha512  
minlen=10
```

User Lockouts

PAM also allows configuring user or password lockout policies to prevent brute-force attacks by imposing a timeout after a certain number of failed login ...

This policy enforces a 10-minute timeout after three failed password attempts by the same user, effectively slowing down brute-force cracking attempts.

```
auth required pam_tally2.so deny=3 unlock_time=600
```

pam_tally2 and faillock

The `pam_tally2` module helps protect systems from brute-force attacks by tracking login attempts, enforcing timeouts, and resetting failed attempt counters ...

You might query a user named `snuffyg`'s record of failed logging attempts with this command.

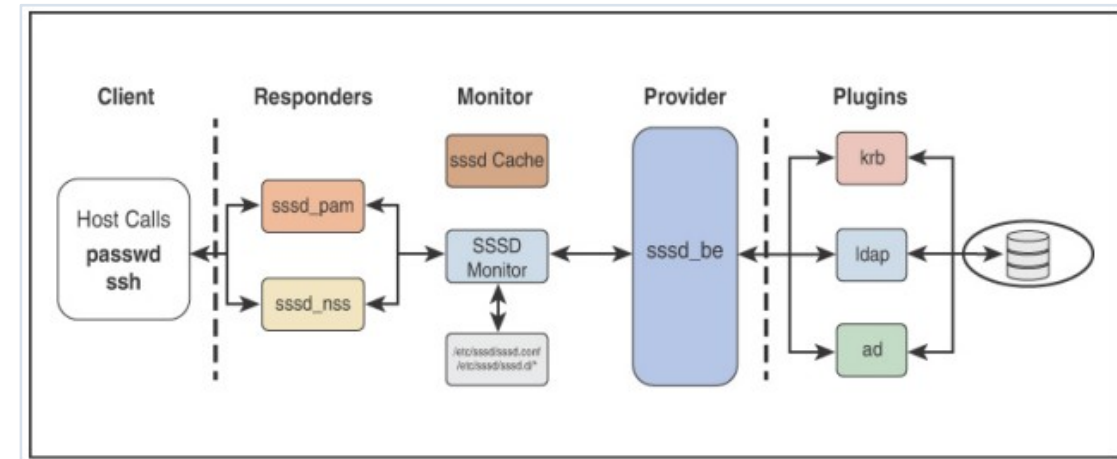
If a user has too many failed login attempts and is in timeout, you can reset their failed attempts to zero and give them a fresh start using a specific ...

```
$ faillock -user snuffyg --reset
```

System Security Services Daemon

The System Security Services Daemon (SSSD) is a PAM extension that enables access to multiple authentication sources, allowing users to authenticate ...

Client processes like ssh and passwd send authentication requests to responders, which interact with the SSSD monitor. The monitor is configured via ...



Single Sign-On (SSO)

SSO is an authentication mechanism that permits a user to sign-in or authenticate into not just a single system or service, but rather to an environment ...

There are multiple possible SSO providers, with more coming on the market all the time. Some of those providers are

Microsoft

OAuth

OpenID

MiniOrange

The nmap Command

The **nmap** command is used to probe a remote system to determine which network ports are reachable from the local system. This is useful for many reasons, ...

Determining what services are available on the remote system

Testing security features on the remote system, such as TCP wrappers

Verifying the effectiveness of the network's firewall by executing nmap from a remote network

To use the **nmap** command, provide either the IP address or hostname of the system that you want to scan.

The output from **nmap** shows open ports, starting with the port number and protocol (e.g., 23/tcp), followed by the corresponding service (e.g., telnet). By ...

```
# nmap -sV 192.168.1.1
Starting Nmap 7.8( http://nmap.org ) at 2022-1-01 09:41 PST
Nmap scan report for 192.168.1.1
Host is up (1.0s latency).
Not shown: 987 closed ports
PORT      STATE SERVICE      VERSION
23/tcp    open  telnet?
25/tcp    open  smtp?
53/tcp    open  domain      dnsmasq 2.15-OpenDNS-1
###Remaining output omitted
```

The nc Command

The **nc (netcat)** command is a versatile tool for working with TCP and UDP. It can open TCP connections, send UDP packets, listen on ports, perform port ...

The nc command has many uses. For example, if you want to check if a specific port is blocked by your company's firewall before launching a service, you ...

After running the nc command, you should see a blank line below it. On a remote system outside your network, you can use the nc command to connect by ...

If the connection is successfully established with nc, a blank line will appear under the command. Any text you type on this line and press Enter will ...

The following are some useful options for nc command

-w: Used on the client side to automatically close the connection after a specified timeout (e.g., nc -w 30 server 333 will close the connection 30 ...

```
# nc -z localhost 1000-4000
```

```
Connection to localhost 3260 port [tcp/iscsi-target] succeeded
```

```
Connection to localhost 3333 port [tcp/dec-notes] succeeded!
```

Using umask

On a typical Linux system, the umask value, set in either the `/etc/bashrc` or `~/.bashrc` file, determines the default permissions for newly created files ...

The default permissions with no umask value set are

For files: `rw-rw-rw-` (or `666`)

For directories: `rwxrwxrwx` (or `777`)

View the umask for your current shell by executing the following command:

The umask value of `0022` means that write permissions for the group owner and others are masked out or not used. The special permissions (the first ...

	For Files	For Directories
Maximum	<code>rw-</code>	<code>rwxrwxrwx</code>
	<code>rw-</code>	
	<code>rw-</code>	
MASK	<code>----</code>	<code>----M--M-</code>
	<code>M--</code>	
	<code>M-</code>	

Enforcing Password Strength

Password strength is crucial for controlling access to a system. It involves complexity. We can ensure passwords are not only strong but also constantly ...

There are three main things to system managers to ensure their passwords are strong and solid.

Set initial password options

Routinely age passwords

Never use group passwords

Setting Password Parameters

The most important file for managing password strength on a Linux system is `/etc/login.defs`. This configuration file is part of the Shadow Suite, which ...

Important settings in `/etc/login.defs` include

ENCRYPT-METHOD: Defines the encryption algorithm for group passwords (options: DES, MD5, SHA256, SHA512). PAM handles user account passwords.

FAIL-DELAY: Specifies the wait time before another login attempt can be made after a failed password attempt.

LOG-OK-LOGINS: By default, only failed logins are logged, but this setting allows logging of both successful and failed logins.

LOGIN-RETRIES: Sets the maximum number of password retry attempts (usually three), though this is also typically managed by PAM.

Aging Your Passwords

To prevent attackers from having enough time to guess passwords, users need to change them regularly. Since most users prioritize convenience over system ...

To age passwords properly, use the chage command. The syntax for the chage command is

This command acts on elds in the /etc/shadow file and has the following options

-m: Sets the minimum number of days a user must wait before changing their password again after a change.

-M: Sets the maximum number of days a user can go before being required to change their password.

-d: Specifies the number of days since January 1, 1970, that the password was last changed. Is rarely modified directly; when a password is changed with ...

Minimum: 0

Maximum: 99999

Warning: 7

Inactive: -1

Last Change: Mar 19, 2022

Password Expires: Never

Password Inactive: Never

Account Expires: Never

Securing Service Accounts

A service account is similar to a user account, but it is not intended for logging in. It is used solely for associating a service with an account owner ...

The lastlog command shows information formatted with the following headers.

The lastlog command displays an entry for every user and service account in the /etc/passwd file, most of which have never logged in as they are service ...

If you notice that a service account, such as ntp, systemd-network, or kernoops, has logged in, it's a red flag. Service accounts should never be used to ...

```
# lastlog | grep -v "Never logged in"
```

Username	Port	From	Latest
ross	pts/1	192.168.33.144	Wed, Feb 22, 08:20 -0700
ulogin	pts/2	192.168.33.88	Wed, Feb 22, 07:10 -0700
shaggy	pts/3	192.168.33.76	Wed, Feb 22, 05:50 -0700

OS hardening — More Key Topics

Public Key Infrastructure

Public Key Infrastructure (PKI) is a system that includes various roles, procedures, and methods to enable secure encryption and ...

Purpose of Certificates

The main purpose of a PKI setup is to ensure that a given certificate (used to sign and identify an entity) belongs to and ...

Certificate Authentication

To understand PKI, consider e-commerce on the Web. When handling payments and personal data, you can either rely on marketing to ...

Self-Signed Certificates

Another method of using PKI is self-signing, where you build a web of trust with other self-signers to establish credibility over ...

Certificate Authorities

A key advantage of being verified by a central certificate authority is that it quickly establishes trust with individual users. ...

Private Keys

In PKI, private keys are paired with public keys. While everyone can share their public key with others, the private key must ...

OS hardening — More Key Topics

Public Keys

In PKI, public keys are used to securely send messages. If someone wants to send a message that is verifiably from them, they use ...

Public Keys Example

If Rebecca wants to send an encrypted file to Robert, she uses her private key and Robert's public key to encrypt it. Robert then ...

Encryption and Hashing

Encryption can involve hashing, where a unique hash sum is generated for an object. Vendors use this to verify the authenticity ...

Digital Signatures

PKI can also be used for digital signatures, allowing documents like contracts to be signed electronically without ever needing ...

Certificate Use Cases

Secure Sockets Layer (SSL) and Transport Layer Security (TLS) are technologies that power HTTPS, ensuring secure connections ...

Authentication

Linux has come far from the days of just using the local `/etc/passwd` or user database `le`, and a key reason for this is the ...

Lab 16 — OS Hardening: Permissions, SELinux, SSH, Fail2ban

Apply layered OS hardening from file permissions and immutability through SELinux labeling, then harden OpenSSH and add brute-force protection. Inventory SUID ...

You'll build — Set special permission bits, ACLs, and immutability, demo SELinux, harden SSH, and run fail2ban.

Key commands — `chmod setuid/setgid/sticky bit`, `umask` / `chattr` immutability, POSIX ACLs (`setfacl/getfacl`), SELinux ...

Workbench — Killercodea Ubuntu Playground, free in the browser

Full walkthrough — `labs/lab-16-*/README.md` on the course GitHub repo

OS Hardening: Permissions, SELinux, SSH, Fail2ban

Step 1 — chmod, chown, special bits

```
$ useradd -m alice 2>/dev/null
```

Step 2 — umask, chattr, lsattr

```
$ umask 027
```

Step 3 — POSIX ACLs

```
$ apt install -y acl
```

Step 4 — SELinux demo inside Rocky

```
$ apt install -y docker.io && systemctl start docker
```


OS Hardening: Permissions, SELinux, SSH, Fail2ban

Step 5 — Harden OpenSSH

```
$ apt install -y openssh-server
```

Step 6 — fail2ban

```
$ apt install -y fail2ban
```

Step 7 — Hunt SUID binaries and review legacy services

```
$ find / -perm -4000 -type f 2>/dev/null | head -n 20
```

✅ **Verify** — fail2ban-client status sshd shows the active jail and sshd -t confirms the hardened config is valid.

3.4 — Account security

Enforce password policies, restrict shells, and enable MFA.

Exam objective — Explain account hardening techniques and best practices

Hands-on — Lab 17

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 17 — Account Hardening

Harden user accounts through password quality and aging policies, brute-force lockouts, and restricted or non-interactive shells. Explore TOTP MFA and Have I Been ...

You'll build — Tune login.defs, enforce pwquality, set aging with chage, add pam_faillock, and lock down shells.

Key commands — /etc/login.defs (PASS_MAX_DAYS), pam_pwquality / pwscore, chage aging, passwd -l/-u, pam_faillock lockout, nologin / ...

Workbench — Killercode Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-17-*/README.md on the course GitHub repo

Account Hardening

Step 1 — login.defs and a test user

```
$ apt update && apt install -y libpam-pwquality
```

Step 2 — Password quality with pwquality

```
$ minlen = 14
```

Step 3 — Password aging and lock/unlock

```
$ chage -M 90 -m 1 -W 7 alice
```

Step 4 — pam_faillock lockout

```
$ deny = 5
```

Account Hardening

Step 5 — Service accounts with nologin and rbash

```
$ useradd -r -s /usr/sbin/nologin svcacct
```

Step 6 — MFA with Google Authenticator (walkthrough)

```
$ apt install -y libpam-google-authenticator
```

Step 7 — Pwned Passwords k-anonymity lookup

```
$ apt install -y curl
```

✅ **Verify** — `chage -l alice` and `passwd -S alice` confirm aging and lock state; the HIBP query flags breached passwords.

3.5 — Cryptography

Encrypt files, use hashing, and manage certificates.

Exam objective — Explain cryptographic concepts and technologies

Hands-on — Lab 18

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 18 — Cryptography: GPG, LUKS, OpenSSL, WireGuard

Practice core Linux cryptography: GPG key generation and signing, LUKS2 disk encryption, OpenSSL certificates, hashing and HMAC, and WireGuard keys. Learn to ...

You'll build — Generate GPG keys, encrypt/sign, build a LUKS2 volume, create an OpenSSL cert, and make WireGuard keys.

Key commands — GPG batch key generation, encrypt/sign, LUKS2 (cryptsetup, loopback), OpenSSL genrsa / req -x509, hashing vs HMAC, ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-18-*/README.md on the course GitHub repo

Cryptography: GPG, LUKS, OpenSSL, WireGuard

Step 1 — GPG key generation in batch mode

```
$ apt update && apt install -y gnupg
```

Step 2 — Encrypt, decrypt, and detached-sign

```
$ echo "top secret payload" > /tmp/msg.txt
```

Step 3 — LUKS2 on a loopback device

```
$ apt install -y cryptsetup
```

Step 4 — OpenSSL keys and self-signed certificate

```
$ apt install -y openssl
```


Cryptography: GPG, LUKS, OpenSSL, WireGuard

Step 5 — Hashing and HMAC

```
$ sha256sum /tmp/msg.txt
```

Step 6 — WireGuard key pair and sample config

```
$ apt install -y wireguard-tools
```

Step 7 — TLS version and cipher inspection

```
$ echo | openssl s_client -connect example.com:443 -tls1_2 2>/dev/null \
```

✅ **Verify** — `gpg --verify` confirms the signature and `gpg -d` recovers plaintext; the LUKS volume mounts only with the ...

3.6 — Compliance

Verify integrity, run scans, and maintain standards.

Exam objective — Explain the importance of compliance and audit procedures

Hands-on — Lab 19

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 19 — Compliance, Auditing, and File Integrity

Establish file-integrity monitoring and run compliance and audit tooling to detect tampering, rootkits, and misconfigurations. Cover secure deletion, login banners, ...

You'll build — Baseline files with AIDE, scan with rkhunter/lynis/nmap, verify packages, and set banners.

Key commands — AIDE file-integrity baseline, rkhunter rootkit scan, debsums / rpm -V package verification, lynis system audit, nmap ...

Workbench — Killercode Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-19-*/README.md on the course GitHub repo

Compliance, Auditing, and File Integrity

Step 1 — AIDE baseline and tamper check

```
$ apt update && apt install -y aide
```

Step 2 — Rootkit hunting with rkhunter

```
$ apt install -y rkhunter
```

Step 3 — Package verification: debsums and rpm -V

```
$ apt install -y debsums
```

Step 4 — System audit with lynis

```
$ apt install -y lynis
```

Compliance, Auditing, and File Integrity

Step 5 — nmap service detection

```
$ apt install -y nmap openssh-server
```

Step 6 — Secure deletion and wipe

```
$ echo "very sensitive" > /tmp/secret.txt
```

Step 7 — Banners and OpenSCAP / CIS pointers

```
$ echo "Authorized use only. Activity is monitored." > /etc/issue
```

✅ **Verify** — `aide --check` flags the tampered `/etc/hostname`, proving the integrity baseline detects unauthorized change.

Recap — Security

Lab 14 — AAA: sudo, PAM, and Polkit

Lab 15 — Firewalls: ufw, firewalld, iptables, nftables, ipset

Lab 16 — OS Hardening: Permissions, SELinux, SSH, Fail2ban

Lab 17 — Account Hardening

Lab 18 — Cryptography: GPG, LUKS, OpenSSL, WireGuard

Lab 19 — Compliance, Auditing, and File Integrity

End of Day 3

See you tomorrow — 9:30 AM

Automation, Orchestration, and Scripting

17% of the exam | Objectives 4.1–4.5 | Labs 20–24

Type it, break it, fix it — the exam rewards hands that have done the work.

Automation, Orchestration, and Scripting — Objectives

4.1 · Automation

Automate tasks with Ansible, Puppet, and CI/CD tools.

4.2 · Shell scripting

Write and troubleshoot variables, functions, and logic flows.

4.3 · Python basics

Develop scripts using environments, packages, and data types.

4.4 · Version control

Manage code with Git workflows and tagging.

4.5 · AI best practices

Apply code generation and prompt engineering responsibly.

Key Concepts — Automation, Orchestration, and Scripting

Infrastructure as Code (Ansible, Puppet, OpenTofu) makes configuration repeatable and idempotent.

Bash scripting automates tasks with variables, conditionals, loops, functions and shellcheck-clean code.

Python (venv, modules, PEP 8) extends automation beyond the shell; Git version-controls all of it.

AI assistants are used responsibly: verify-before-paste, redact secrets, and follow a corporate AI policy.

4.1 — Automation

Automate tasks with Ansible, Puppet, and CI/CD tools.

Exam objective — Summarize automation and orchestration use cases and techniques

Hands-on — Lab 20

Type it, break it, fix it — the exam rewards hands that have done the work.

Key Characteristics of YAML:

Begins with three hyphens (---).

Ends with three periods (...) to indicate the end of the stream.

Uses spaces for indentation (no tabs).

Comments start with a # and continue until the end of the line, except inside strings.

Lists are represented by a hyphen (-) before each item, and each item is on a separate line. Lists can also be shown inline like [item1, item2, item3].

File Formats

YAML Stream Examples:

Vertical List (more space-consuming): --- # Favorite Bands

- The Psychedelic Furs

- Accept

- Dokken

- Marty Friedman

- Motorhead

Key Characteristics of JSON:

Uses {} for objects and [] for lists.

Allows spaces, tabs, and line breaks for structure.

No comments are allowed.

Lists are represented within square brackets, e.g., "colors": ["red", "green", "blue"].

File Formats

JSON Examples

File Formats

```
{
  "Favorite Band": {
    "name": "The Psychedelic Furs",
    "Begun": 1977,
    "First Album": "The Psychedelic Furs",
    "Soundtrack Credits": "Pretty in Pink, 1986",
    "photo": " https://en.wikipedia.org/wiki/File:Psychedeli
jpg",
    "Albums": [
      "The Psychedelic Furs",
      "Talk Talk Talk",
      "Forever Now",
      "Mirror Moves",
      "Midnight to Midnight",
      "Book of Days",
      "World Outside",
      "Made of Rain"
    ]
  }
}
```

Use version control for everything to easily fix mistakes and restart.

Automate everything possible—avoid manual tasks.

Ensure consistency in your IaC by running it multiple times for identical results.

Keep IaC modular to limit the impact of changes, but avoid excessive modularization.

Perform many dry runs (hundreds or thousands) to ensure automation works as expected.

Develop a quick and thorough testing method for functionality.

Getting Started with IaC

Example Ansible Playbook

Understanding Infrastructure as Code

```
---  
- name: Update web servers  
  hosts: webservers  
  remote_user: root  
  
  tasks:  
    - name: Ensure apache is at the latest version  
      ansible.builtin.yum:  
        name: httpd  
        state: latest  
    - name: Write the apache config file  
      ansible.builtin.template:  
        src: /srv/httpd.j2  
        dest: /etc/httpd.conf  
  
- name: Update db servers  
  hosts: databases  
  remote_user: root  
  
  tasks:  
    - name: Ensure postgresql is at the latest version  
      ansible.builtin.yum:  
        name: postgresql  
        state: latest  
    - name: Ensure that postgresql is started  
      ansible.builtin.service:  
        name: postgresql  
        state: started
```

Salt Configuration Files:

Primary configuration files are located in `/etc/salt/`:

`/etc/salt/master`: Configuration for the Salt Master.

`/etc/salt/minion`: Configuration for the Salt Minions.

Additional configuration files can be placed in the `.d` include directory (e.g., `/etc/salt/conf.d`), which will be read after the main configuration is ...

Salt Configuration Locations

Remote Execution Modules:

Small programs that perform actions like:

Adding users.

Installing packages.

Copying files from the master to the minion.

Other system management tasks.

Salt State Files

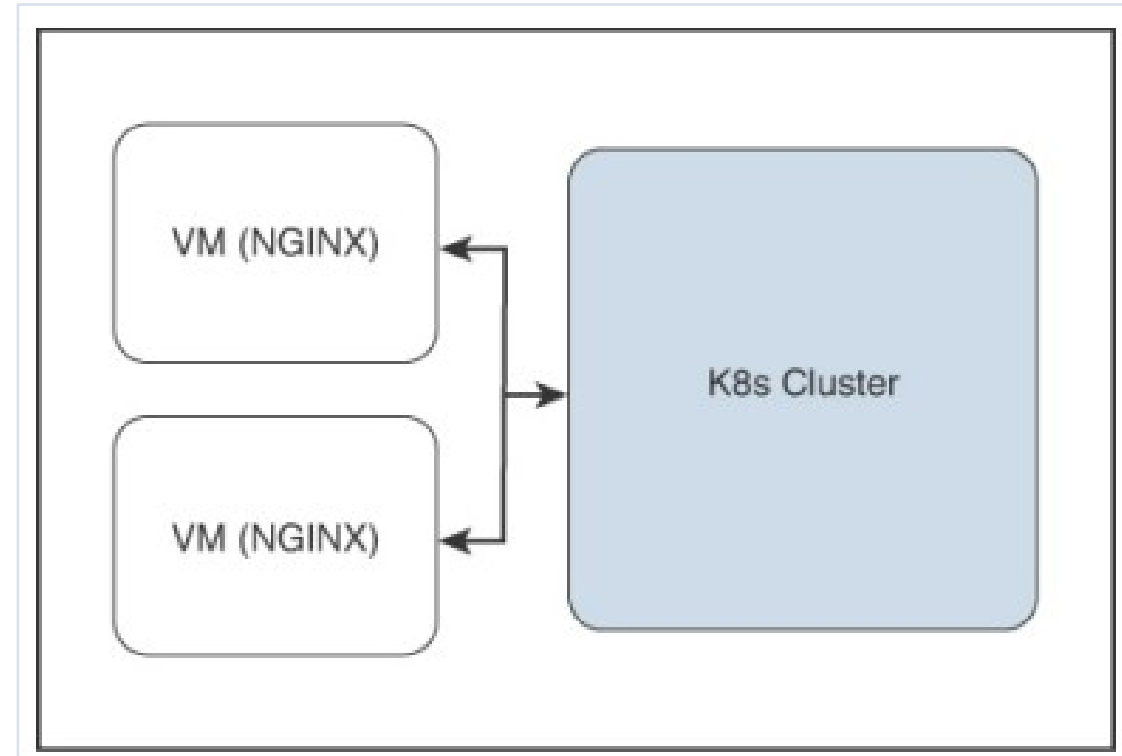
Example of an NGINX State File

Salt State Files

```
nginx:
  pkg:
    - installed
  service:
    - running
    - watch:
      - pkg: nginx
      - file: /etc/nginx/nginx.conf
```

Coding

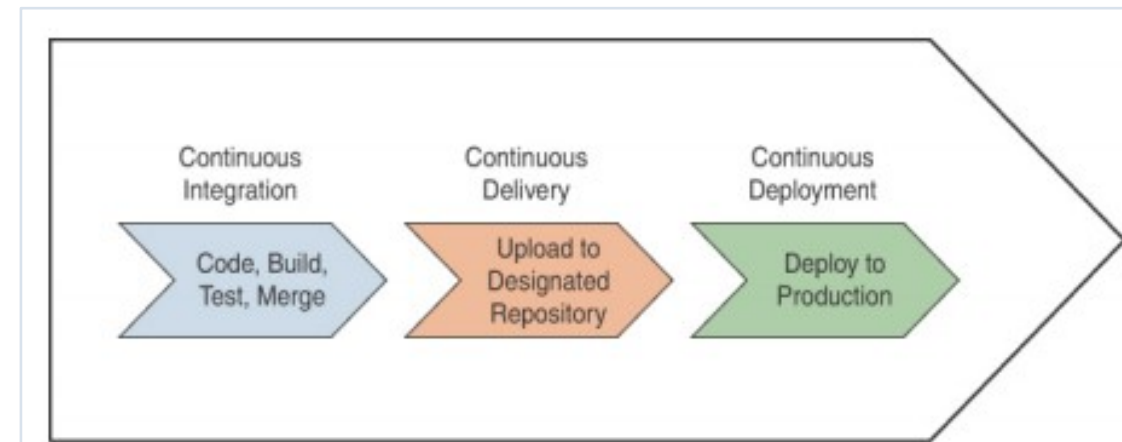
Understanding Infrastructure as Code



CI/CD three-step process

Continuous Integration/Continuous

Deployment



general categories of CI/CD use cases that you might encounter on ...

Web Applications: Automating development, testing, and deployment for web-based services.

Financial Services: Ensuring secure and consistent updates to financial platforms.

Shopping Sites: Streamlining updates to e-commerce platforms for improved user experience and functionality.

Gaming Applications: Facilitating rapid deployment of updates, patches, and new features in games.

Social Media Applications: Managing frequent updates to social platforms with minimal downtime.

CI/CD Use Cases

Kubernetes Control Plane Structure

The Kubernetes control plane includes the following components:

kube-controller-manager: Manages all control loops within Kubernetes.

kube-scheduler: Assigns containers to Pods that don't yet have containers, based on the configured environment requirements.

cloud-controller-manager: Works with cloud providers' APIs to handle tasks like name resolution and resource scaling.

kube-apiserver: Handles front-end operations for the Kubernetes API.

etcd: A highly available key-value store that maintains Kubernetes' database information, logs, and ensures availability across nodes and components.

Kubernetes Node Structure

Kubernetes nodes are systems that run applications in the Kubernetes environment. Key components of a node include:

kubelet: Ensures that the desired state of Pods is met by deploying and maintaining their health.

kube-proxy: Manages proxy tasks for UDP, TCP, and SCTP, helping to provide load-balanced services.

Pods: The smallest deployable unit in Kubernetes, containing one or more containers that share the same network and filesystem.

Containers: While not native to Kubernetes, it supports various container standards for running applications.

Specialized Container Types

In addition to regular application containers, there are two specialized container types to be aware of:

Sidecar

Ambassador

Sidecar: A sidecar container works alongside the main application container(s) within the same pod. It shares the same storage and network resources. ...

Ambassador: An ambassador container acts as an intermediary between a web server and its backend microservice. It allows for seamless redirection of ...

Benefits of Using Kubernetes

Kubernetes is especially popular among large organizations due to its automation benefits. Some key advantages include:

Better resource utilization

Faster software development

Easier application upgrades and maintenance

Ability to containerize older, monolithic apps

Reduced reliance on public cloud services and their costs

Docker Volumes

Objective 3.5 doesn't specifically mention Kubernetes for container persistent storage, so we begin with Docker volumes. You can create a volume on the ...

From there, you can mount the volume to a new container using the following command:

Now you can open a BASH shell in the container and see that the /mydatavol storage location is present

```
$ docker -d -p 5000:5000 -v mydatavol:/mydatavol mydatavol
```

Example Service Mesh

Istio is a popular open-source service mesh provider that integrates into Kubernetes clusters. It uses sidecar proxies within Pods, offering advanced ...

Istio offers several features when deployed as a service mesh, including:

Secure service-to-service communication with TLS encryption and identity-based authentication.

Automatic load balancing for various traffic types.

Fine-grained traffic control with routing rules, retries, and fault injection.

A pluggable policy layer supporting access controls, rate limits, and quotas.

What Are My Container Registry Options?

Top-rated container registries include:

Amazon ECR

Microsoft Azure Container Registry

Docker Hub

GitHub Package Registry

Google Cloud Artifact Registry

Automation — More Key Topics

File Formats

YAML (YAML Ain't Markup Language) is a human-readable, text-based data serialization language commonly used for configuration ...

YAML (YAML Ain't Markup Language) is a human-readable, ...

File Formats

JSON (JavaScript Object Notation) is a lightweight ...

File Formats

Infrastructure as code (IaC) can be defined as the ...

Infrastructure as Code Concepts

Manually managing configuration changes becomes ...

Infrastructure as Code (IaC) automates configuration changes across systems. IaC defines infrastructure and configurations ...

Store configuration files in a Git repository (or ...

Use IaC to deploy and manage services or applications. Make changes to the IaC files and redeploy to implement updates.

Automation — More Key Topics

Functional (Declarative): Focuses on what the final ...

Procedural (Imperative): A set of specific steps or commands to execute in order to reach the desired result. Utilities: ...

The IaC utilities listed in objective 3.4 are all ...

IaC Utility Choices

Ansible: Open-source IT automation tool for ...

Playbooks: YAML files that describe automation tasks. Tasks: Smallest unit of automation in Ansible, which can be combined to ...

Puppet: A declarative tool that describes the desired ...

Chef: An imperative tool that specifies actions for achieving the desired state, using Ruby Domain Specific Language (DSL). Chef ...

SaltStack: A feature-rich, declarative IaC tool used ...

SaltStack

Comprised of a master and minion setup.

Salt Master: The server that controls and manages the minion systems, which are the client systems. Salt Architecture

Lab 20 — Infrastructure as Code with Ansible

Learn Infrastructure as Code using Ansible as the primary agentless configuration management tool, and recognize the wider IaC ecosystem (Puppet, OpenTofu, ...)

You'll build — Install Ansible, write an inventory, run ad-hoc commands, and author an idempotent playbook with variables, templates, ...

Key commands — Ansible, inventory, playbook, idempotency, handlers/templates, ansible-galaxy

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-20-*/README.md on the course GitHub repo

Infrastructure as Code with Ansible

Step 1 — Install Ansible and prepare workspace

```
$ apt update
```

Step 2 — Create inventory pointing at localhost

```
$ [web]
```

Step 3 — Run ad-hoc commands

```
$ cd /root/lab
```

Step 4 — Write a playbook with variable, template, and handler

```
$ - name: Deploy a simple nginx site
```

Infrastructure as Code with Ansible

Step 5 — Install a Galaxy collection and peek at other IaC tools

```
$ ansible-galaxy collection install community.general
```

Step 6 — GitOps, cloud-init, and Kubernetes ConfigMap samples

```
$ package_update: true
```

✅ **Verify** — Idempotent playbooks converge state without unnecessary restarts; Ansible, Puppet, OpenTofu, cloud-init, and ...

4.2 — Shell scripting

Write and troubleshoot variables, functions, and logic flows.

Exam objective — Given a scenario, perform automated tasks using shell scripting

Hands-on — Lab 21

Type it, break it, fix it — the exam rewards hands that have done the work.

Shell Script Elements

Purpose of Shell Scripting: Automates routine work by creating custom tools.

Benefit: Turns multiple commands into a single command, saving time.

Value: Time invested in learning scripting results in significant time savings.

Skill Requirement: No programming expertise needed; basic Linux command-line knowledge is sufficient.

Starting Point: All scripting begins with the Linux command line.

shell script is just a text file that contains some commands. The

```
#!/bin/bash
# Deploy the application. Run as deploy foo-1.2.war
cp $1 /usr/local/tomcat/webapps/application.war
service tomcat restart
echo "Deployed $1" | mail group@example.com
```

Globbering

Definition of a Glob: A wildcard operator that matches multiple files based on a pattern.

Example: `/etc/*` matches files in the `/etc` directory (e.g., `/etc/foo` and `/etc/bar`) but not files in subdirectories (e.g., `/etc/foo/bar`).

Linux Shell Behavior:

The shell expands the glob to the list of matching files.

Example: `ls /tmp/thing*` expands to `ls /tmp/thing1 /tmp/thing2` if `thing1` and `thing2` exist.

Limitations in Renaming Files:

```
$ ls *.foo *.bar
ls: *.bar: No such file or directory
file1.foo file2.foo
$ echo mv *.foo *.bar
mv file1.foo file2.foo *.bar
$ mv *.foo *.bar
mv: target '*.bar' is not a directory
```

Environment Variables and Settings

Purpose of Variables: Store information for the shell or running program.

Process-Specific: Each process maintains its own set of variables.

Types of Variables in BASH:

Available only to the current BASH shell.

Not passed to new programs or processes started from the shell.

Environment (Global) Variables:

```
$ set | head -5
BASH=/bin/bash
BASHOPTS=checkwinsize:cmdhist:expand_aliases:extquote:force_ignore:hi
stappend:hostcomplete:interactive_comments:login_shell:progcomp:prompt
vars:sourcpath
BASH_ALIASES=()
BASH_ARGC=()
BASH_ARGV=()
```

The PATH Variable

You can always refer to an executable in your current directory with one of two methods:

Absolute pathname: `/home/sarah/command`

Relative pathname: `./command`

Manually memorizing command locations or navigating directories is cumbersome.

The PATH variable simplifies this process.

PATH Variable: A colon-separated list of directories used to locate commands.

```
$ echo $PATH
/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/sarah/.local/bin:/home/sarah/bin
```


The SHELL Variable

SHELL Variable: Stores the full path and filename of the user's current shell.

Purpose: Indicates the user's default shell.

```
$ echo $SHELL  
/bin/bash
```

Variable Expansion

What happens when you try to display a variable that hasn't been set? By default, a null string ("") value is returned:

Variable Expansion in BASH: Allows returning a default value if a variable is not set.

Note this command doesn't set the variable but rather uses the provided value for this one situation. To return a value and set the variable, use the ...

There are other variable expansion features. For example, the following syntax returns only the first 24 characters of the variable, resulting in a more ...

```
$ times="It was the best of times, it was the worst of times."  
$ echo ${times: 0:24}  
It was the best of times
```

Running a Script

There are three ways to run a script. The first is the same as if it were a binary application:

Script Execution Methods:

Ensure the script has execute permissions using

Pass the script as an argument to a new shell

Sourcing: Executes the script within the current shell instead of starting a new process.

Sourcing Method Use Case:

```
$!/bin/bash
#my-prog.sh
source set-var.sh
#rest of the code goes here
```

Working with Input/Output Streams

File Descriptors in Linux:

Used to handle data streams on the shell command line.

Facilitate sending data to and from programs and files.

Handle errors efficiently.

Linux File Descriptors:

Name	File Descriptor Number	Associated File
/dev/stdin	0	/proc/self/fd/0
/dev/stdout	1	/proc/self/fd/1
/dev/stderr	2	/proc/self/fd/2

Standard In

Standard Input (stdin):

The default input stream for programs.

Programs typically accept input from

Redirected file argument:

File as an argument:

program file

Standard Out

Standard Output (stdout):

The default stream for displaying text or data produced by a command.

By default, output is sent to the screen or console.

Can be redirected to a file.

```
$ cat /etc/fstab
```

Find Errors on Demand

Generating Errors for Testing:

Use the find command to intentionally produce permission or access errors.

Errors (stderr) indicate restricted access to certain directories.

Useful output (stdout) is mixed with error messages (stderr).

To clean up good data and get rid of bad data, you need to use redirection operators. To see this work, use the Up arrow and rerun the previous command, as ...

This produces output similar to:

```
./ .kde/share/apps/kdeprint/printerdb_cups.txt  
./1.txt  
./2.txt  
./3.txt
```

Here Documents

A method to provide input directly to a program or script through stdin.

Useful for entering multiple lines of text without needing escaping for quotes or variables.

String Limiter: Identifies the end of input (e.g., DONE).

The shell removes instances of the string limiter from the text body.

Variables and quotes inside the heredoc body behave normally.

The string limiter is only recognized when placed alone on its own line.

```
# cat << DONE
> this is input
> this is more input
> DONE

this is input
this is more input

#
```


Redirecting Standard Output

Uses **>** to overwrite a file or **>>** to append to a file.

Single Redirection (**>**):

Overwrites the contents of the target file.

Creates the file if it does not exist.

Double Redirection (**>>**):

Appends the data to the end of the target file.

```
# find / -iname *.txt > foundfiles
```

Redirecting stderr:

Default Behavior: stderr is mixed with stdout but can be separated.

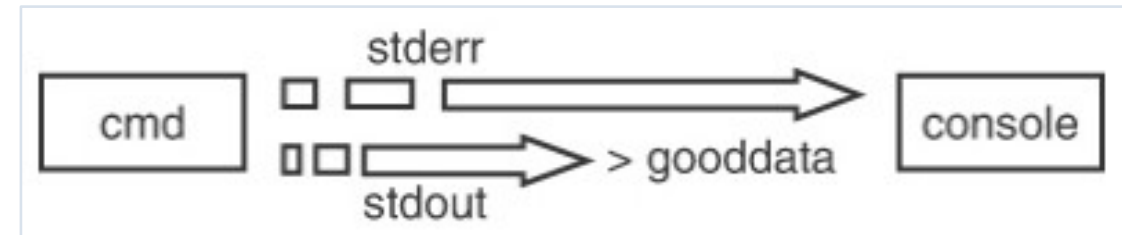
Example

Redirecting Standard Error

```
# find / -iname *.txt 2> errors
```

Understanding what happens when using stdout and stderr is to ...

Redirecting Standard Error



Redirecting stdout and stderr Separately:

>: Redirects stdout (good data) to the file gooddata.

2>: Redirects stderr (errors) to the file baddata.

Nothing is displayed on the console.

Redirecting Standard Error

```
# find / -iname *.txt > gooddata 2> baddata
```

Redirect both stdout and stderr to /dev/null to avoid output on the ...

> /dev/null: Discards stdout.

2>&1: Discards stderr by redirecting it to stdout.

Redirection Redux

```
# find / -iname *.txt > /dev/null 2>&1
```

Determine Current Console:

Use the `tty` command to identify the terminal.

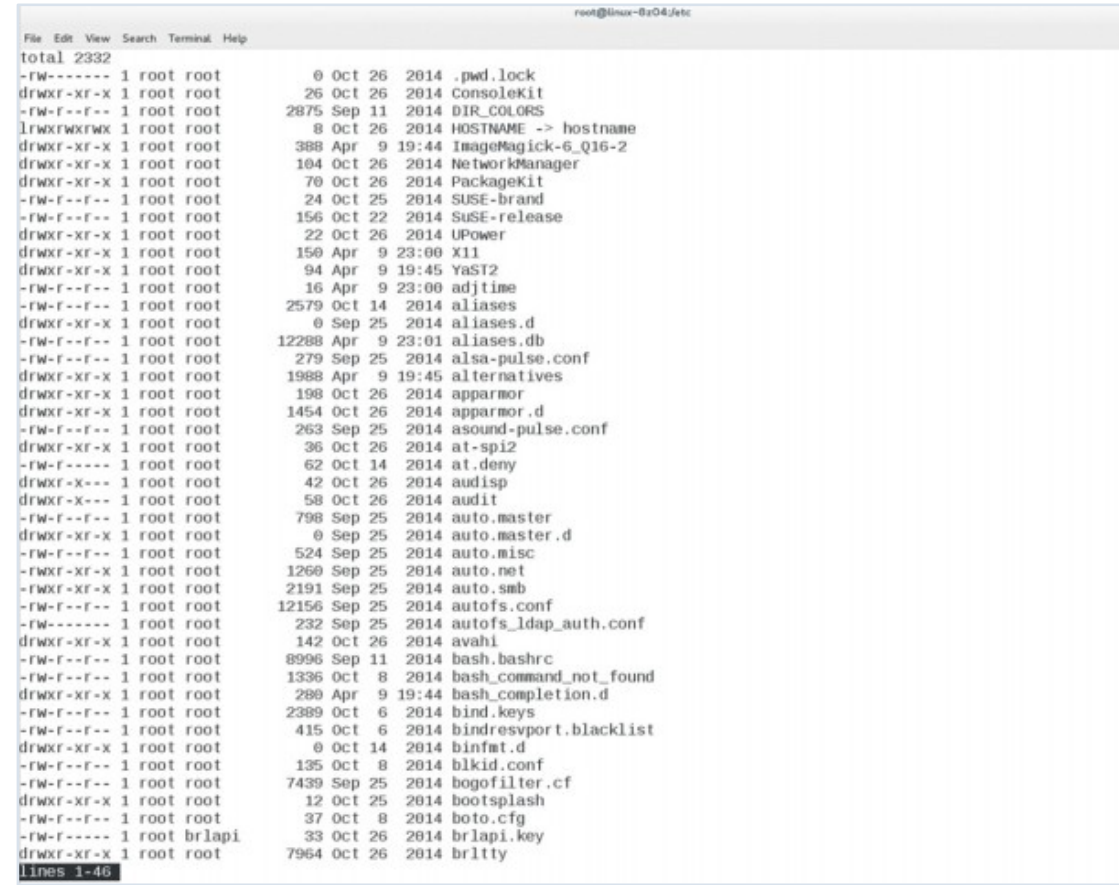
Output indicates you're using a terminal in a GUI environment.

Understanding `/dev/tty`

```
# tty
/dev/pts/0
```

Example, in the Figure the ls command's standard output is sent to ...

Pipes



```

File Edit View Search Terminal Help
root@linux-0x04:etc
total 2332
-rw-r--r-- 1 root root      0 Oct 26 2014 .pwd.lock
drwxr-xr-x 1 root root    26 Oct 26 2014 ConsoleKit
-rw-r--r-- 1 root root 2875 Sep 11 2014 DIR_COLORS
lrwxrwxrwx 1 root root      8 Oct 26 2014 HOSTNAME -> hostname
drwxr-xr-x 1 root root   388 Apr  9 19:44 ImageMagick-6_Q16-2
drwxr-xr-x 1 root root   104 Oct 26 2014 NetworkManager
drwxr-xr-x 1 root root    70 Oct 26 2014 PackageKit
-rw-r--r-- 1 root root    24 Oct 25 2014 SUSE-brand
-rw-r--r-- 1 root root   156 Oct 22 2014 SuSE-release
drwxr-xr-x 1 root root    22 Oct 26 2014 UPower
drwxr-xr-x 1 root root   150 Apr  9 23:00 X11
drwxr-xr-x 1 root root    94 Apr  9 19:45 YaST2
-rw-r--r-- 1 root root    16 Apr  9 23:00 adjtime
-rw-r--r-- 1 root root  2579 Oct 14 2014 aliases
drwxr-xr-x 1 root root      0 Sep 25 2014 aliases.d
-rw-r--r-- 1 root root 12288 Apr  9 23:01 aliases.db
-rw-r--r-- 1 root root    279 Sep 25 2014 alsa-pulse.conf
drwxr-xr-x 1 root root   1988 Apr  9 19:45 alternatives
drwxr-xr-x 1 root root    190 Oct 26 2014 apparmor
drwxr-xr-x 1 root root   1454 Oct 26 2014 apparmor.d
-rw-r--r-- 1 root root   263 Sep 25 2014 asound-pulse.conf
drwxr-xr-x 1 root root    36 Oct 26 2014 at-spi2
-rw-r--r-- 1 root root    62 Oct 14 2014 at.deny
drwxr-xr-x 1 root root    42 Oct 26 2014 audisp
drwxr-xr-x 1 root root    58 Oct 26 2014 audit
-rw-r--r-- 1 root root   798 Sep 25 2014 auto.master
drwxr-xr-x 1 root root      0 Sep 25 2014 auto.master.d
-rw-r--r-- 1 root root    524 Sep 25 2014 auto.misc
drwxr-xr-x 1 root root   1260 Sep 25 2014 auto.net
drwxr-xr-x 1 root root   2191 Sep 25 2014 auto.smb
-rw-r--r-- 1 root root 12156 Sep 25 2014 autofs.conf
-rw-r--r-- 1 root root   232 Sep 25 2014 autofs_ldap_auth.conf
drwxr-xr-x 1 root root    142 Oct 26 2014 avahi
-rw-r--r-- 1 root root  8996 Sep 11 2014 bash.bashrc
drwxr-xr-x 1 root root   1336 Oct  8 2014 bash_command_not_found
drwxr-xr-x 1 root root    289 Apr  9 19:44 bash_completion.d
-rw-r--r-- 1 root root   2389 Oct  6 2014 bind.keys
-rw-r--r-- 1 root root    415 Oct  6 2014 bindresvport.blacklist
drwxr-xr-x 1 root root      0 Oct 14 2014 binfmt.d
-rw-r--r-- 1 root root    135 Oct  8 2014 blkid.conf
-rw-r--r-- 1 root root   7439 Sep 25 2014 bogofilter.cf
drwxr-xr-x 1 root root    12 Oct 25 2014 boot splash
-rw-r--r-- 1 root root    37 Oct  8 2014 boto.cfg
-rw-r--r-- 1 root root    33 Oct 26 2014 brlapi.key
drwxr-xr-x 1 root root   7964 Oct 26 2014 brltty
lines 1-46

```

Redirection vs Pipe:

Pipe (|): Passes the output of one command to the next.

Redirection (>): Redirects output to a file.

Example of Incorrect Command:

```
cat file1 | nl > pr
```

Outcome: Creates a file named pr in the current directory, not sending output through both nl and pr.

Pipes

```
# cat codesamp.c | nl | pr | lpr
```


The following is a good example of using a pipe to produce usable

This command sorts and then numbers the file names.

Pipes

```
# sort < names | nl
```

Command Substitution:

Used to place the output of a command into a variable, typically for scripting purposes.

Achieved by enclosing the command in backticks (`).

Example: `export DATETIME=`date``

`echo $DATETIME`

The output of the date command is stored in the DATETIME variable.

Command Substitution

```
$ PID=$(ps -ef | grep "[n]ginx: master process" | awk "{ print $2 }")  
$ echo nginx is running at $PID  
4846
```

Using the Output of Another Command:

Capture the output of a command and store it in a variable for later use.

Shell Script Elements

```
$ ps -ef | grep nginx
root 4846 1 0 Mar11 ? 00:00:00 nginx: master process /usr/sbin/nginx
-c /etc/nginx/nginx.conf
nginx 6732 4846 0 Mar11 ? 00:00:12 nginx: worker process
nginx 19617 2655 0 18:54 ? 00:00:01 php-fpm: pool www
nginx 19807 2655 0 19:01 ? 00:00:00 php-fpm: pool www
nginx 19823 2655 0 19:03 ? 00:00:00 php-fpm: pool www
sarah 20007 19931 0 19:07 pts/0 00:00:00 grep nginx
```

From here you can pick out just the master process by fine-tuning

your grep statement:

Shell Script Elements

```
$ ps -ef | grep "nginx: master process"
root 4846 1 0 Mar11 ? 00:00:00 nginx: master process /usr/sbin/nginx
-c /etc/nginx/nginx.conf
sarah 20038 19931 0 19:09 pts/0 00:00:00 grep nginx: master process
```

Example: Find Process ID of nginx:

To find the process ID of nginx, use `ps` and filter with `grep`, avoiding the `grep` command itself with a regular expression:

Use `awk` to extract the process ID (column 2)

Shell Script Elements

```
$ ps -ef | grep "[n]ginx: master process"
root 4846 1 0 Mar11 ? 00:00:00 nginx: master process /usr/sbin/nginx
-c /etc/nginx/nginx.conf
```

Example - Reopen Logs:

Shell Script Elements

```
$ echo Telling nginx to reopen logs using pid $PID  
$ kill -USR1 $PID
```

Using \$? for Return Code:

\$? holds the exit status (return code) of the last command executed.

0 means success (true), while any non-zero value indicates failure (false).

Conditionals

```
$ ps -ef | grep -q [n]agios
$ echo $?
0
$ ps -ef | grep -q [d]oesNotExist
$ echo $?
1
```

Negating a Condition:

Use the ! (exclamation point) to negate the outcome of a statement, testing for the absence of a condition.

Conditionals

```
if ! ps -ef | grep -q [n]agios; then  
    echo Nagios is NOT running  
fi
```


Using else for Alternative Actions:

The else block allows you to specify a different action when the condition is false.

Conditionals

```
if ps -ef | grep -q [n]agios; then
    echo Nagios is running
else
    echo Nagios is not running. Starting
    service nagios start
fi
```

Using elif for Multiple Conditions:

The elif (else if) allows you to test multiple conditions, executing different actions based on which condition is true.

Conditionals

```
if ps -ef | grep -q [h]ttpd; then
    echo Apache is running
elif ps -ef | grep -q [n]ginx; then
    echo Nginx is running
else
    echo No web servers are running
fi
```

Using the test Command:

The test command allows you to check conditions on files, strings, and numbers in a simpler way.

Testing Files

```
if test -f /etc/passwd; then  
    echo "password file exists"  
fi
```

The test command is used so much that BASH has the [] (square ...

Anything between the square brackets is considered a test

statement.

Easier Test Syntax

```
if [ test_statement ]; then
```

A test to see whether /usr/bin/nginx is executable could be performed ...

Newer versions of BASH can also use two sets of square brackets:

Easier Test Syntax

```
if [ -x /usr/bin/nginx ]; then  
    echo nginx is executable  
fi
```

Words, also known as strings, can be easily tested, as shown here:

Testing Strings

```
echo -n "Say something: "  
read STRING  
if [[ -z $STRING ]]; then  
    echo "You didn't say anything"  
else  
    echo Thanks for that  
fi
```

Test string equality with a single equals sign:

Testing Strings

```
if [[ 'hostname' = 'bob.ertw.com' ]]; then
    echo You are on your home machine
else
    echo You must be somewhere else
fi
```

Script for Calculating Elapsed Time:

Purpose: Check if a certain amount of time has passed since the last execution of a script and take action accordingly.

Testing Integers

```
LASTRUN=$(cat lastrun)
NOW=$(date +%s)
ELAPSED=$((NOW - LASTRUN))

if [[ $ELAPSED -ge 60 ]]; then
    echo A minute or more has passed since the last time you ran this
    script
    echo $NOW > lastrun
fi
```


Boolean Logic with test and []:

Use -a for AND and -o for OR inside [condition].

Example with []

Boolean Logic with [[]]:

Use && for AND and || for OR inside [[condition]].

Combining Multiple Tests

```
if [[ -f /etc/passwd && -x /usr/sbin/adduser ]]
```

if/elif/else statements get bulky if you have more than three

possibilities. For such situations you can use case

case Statements

```
case $1 in
  start)
    echo "Starting the process"
    ;;
  stop)
    echo "Stopping the process"
    ;;
  *)
```

A String Matched as if It Were a File Glob

case Statements

```
;;  
[1-7]*|[0-9])  
echo "Lots of space left"  
;;  
[8-9]*)  
echo "I'm at least 80% full"  
;;  
*)  
echo "Hmmm. I expected some kind of percentage"  
esac
```

Using a Switch Statement

switch Statements

```
let a = 3 + 3;
switch (a) {
  case 5:
    alert( 'Not Enough ');
    break;
  case 6:
    alert( 'Just Right!' );
    break;
  case 7:
    alert( 'Too Many' );
    break;
  default:
    alert( "Enter a valid value" );
}
```

For Loops

The for loop's basic syntax is

The collection can be fixed:

Loops

```
$ for name in Ross Sarah Mary-Beth; do  
> echo $name  
> done  
Ross  
Sarah  
Mary-Beth
```

Sequences

Loops

```
$ seq 1 5
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

while and until Loops

Loops

```
until [ -f /var/tmp/report.done ]; do  
  
    # Waiting until the report is done  
    sleep 10  
  
done  
  
rm /var/tmp/report.done
```

Ignoring Output from Commands:

When running programs like `cp`, `awk`, `sed`, `grep`, etc., you may want to suppress or ignore their output.

Redirect the standard output (stdout) and standard error (stderr) to `/dev/null` to discard them.

Interacting with Other Programs

```
grep -r foo /etc > /dev/null 2>&1
```


Using the shift Keyword

Accepting Arguments

```
while [[ $# -gt 0 ]]; do
    echo Processing $1
    shift
    echo There are $# to go
done
$ ./test.sh one two three
Processing one
There are 2 to go
Processing two
There are 1 to go
Processing three
There are 0 to go
```

Reordering Fields with awk:

Use awk to transform the /etc/passwd file by reordering and formatting its fields.

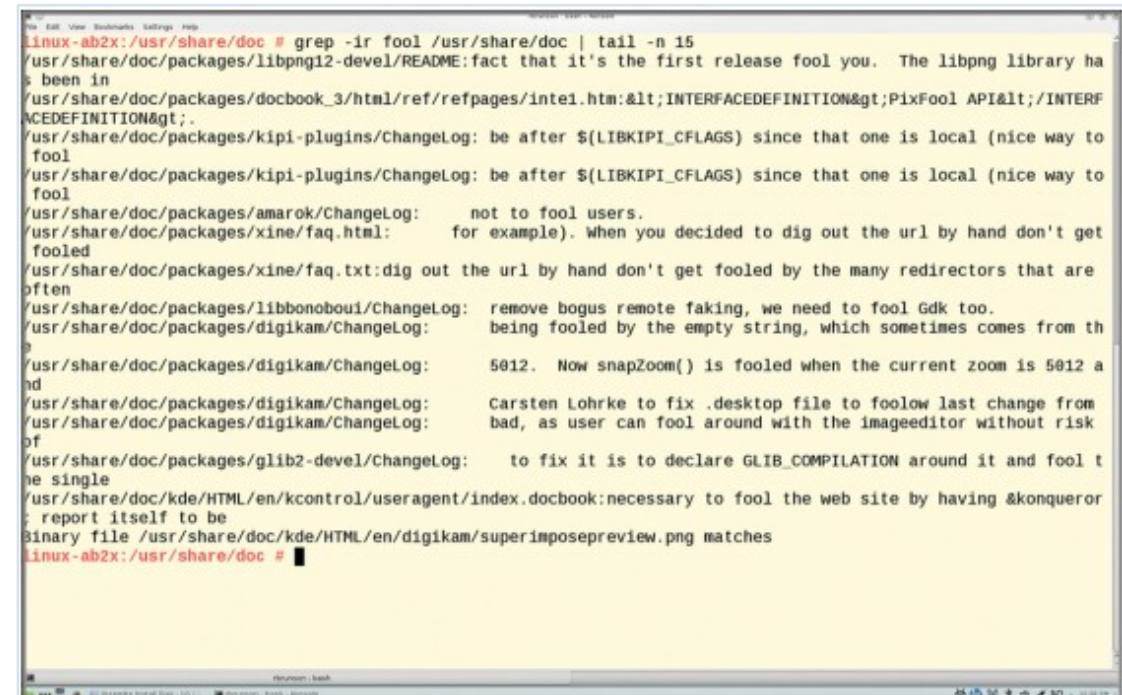
Feeling a Bit (awk)ward

```
# awk -F: '{print $1"\t""ID #"$3"\t""In the role of""\t"$5}' /etc/passwd
gnats      ID #41      In the role of Gnats Bug-Reporting System (admin)
nobody     ID #6553     In the role of nobody
kernoops   ID #106      In the role of Kernel Oops Tracking Daemon,,,
lightdm    ID #110      In the role of Light Display Manager
speech     ID #117      In the role of Speech Dispatcher,,,
ross       ID #900      In the role of Ross,,,

```

To search for the word fool in the additional documentation ...

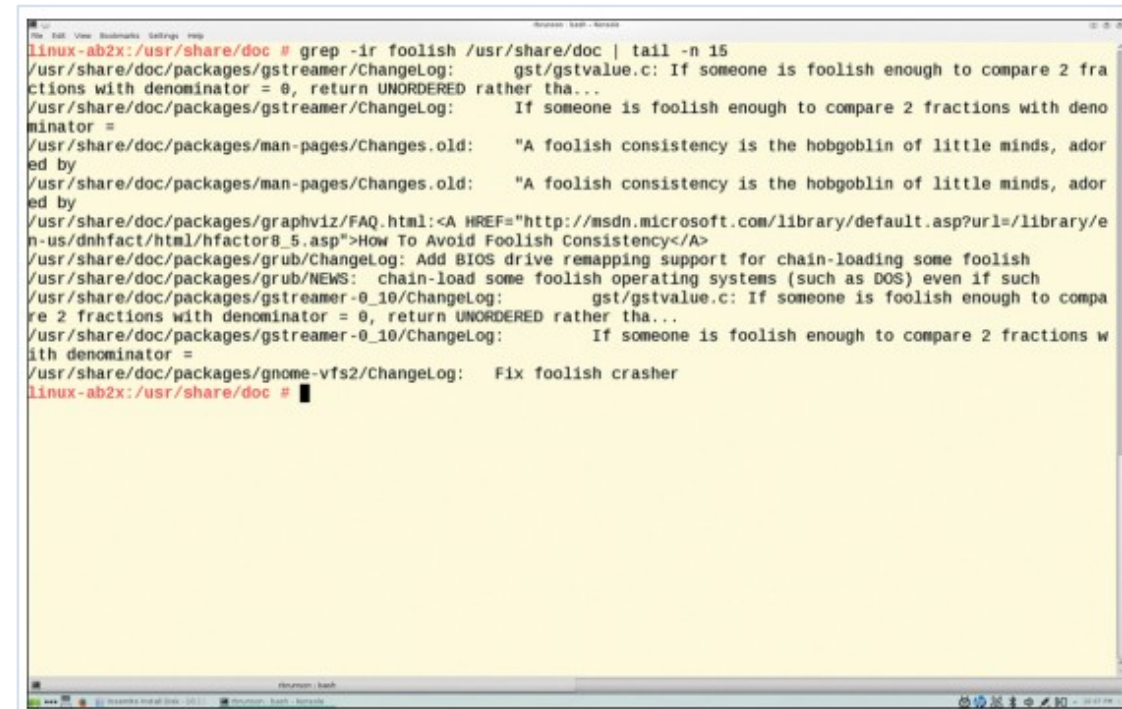
Getting a grep



```
Linux-ab2x:/usr/share/doc # grep -ir fool /usr/share/doc | tail -n 15
/usr/share/doc/packages/libpng12-devel/README:fact that it's the first release fool you. The libpng library has
been in
/usr/share/doc/packages/docbook_3/html/ref/refpages/intel.htm:<INTERFACEDefinition>PixFool API</INTERF
ACEDefinition>.
/usr/share/doc/packages/kipi-plugins/ChangeLog: be after $(LIBKIPI_CFLAGS) since that one is local (nice way to
fool
/usr/share/doc/packages/kipi-plugins/ChangeLog: be after $(LIBKIPI_CFLAGS) since that one is local (nice way to
fool
/usr/share/doc/packages/amarok/ChangeLog: not to fool users.
/usr/share/doc/packages/xine/faq.html: for example). When you decided to dig out the url by hand don't get
fooled
/usr/share/doc/packages/xine/faq.txt:dig out the url by hand don't get fooled by the many redirectors that are
often
/usr/share/doc/packages/libbonoboui/ChangeLog: remove bogus remote faking, we need to fool Gdk too.
/usr/share/doc/packages/digikam/ChangeLog: being fooled by the empty string, which sometimes comes from th
p
/usr/share/doc/packages/digikam/ChangeLog: 5012. Now snapZoom() is fooled when the current zoom is 5012 a
nd
/usr/share/doc/packages/digikam/ChangeLog: Carsten Lohrke to fix .desktop file to foolow last change from
bad, as user can fool around with the imageeditor without risk
of
/usr/share/doc/packages/glib2-devel/ChangeLog: to fix it is to declare GLIB_COMPILATION around it and fool t
he single
/usr/share/doc/kde/HTML/en/kcontrol/useragent/index.docbook:necessary to fool the web site by having &konqueror
; report itself to be
Binary file /usr/share/doc/kde/HTML/en/digikam/superimposepreview.png matches
Linux-ab2x:/usr/share/doc #
```

Output of the grep command search for fool

Getting a grep



```
linux-ab2x:/usr/share/doc # grep -ir foolish /usr/share/doc | tail -n 15
/usr/share/doc/packages/gstreamer/ChangeLog:    gst/gstvalue.c: If someone is foolish enough to compare 2 fractions with denominator = 0, return UNORDERED rather than...
/usr/share/doc/packages/gstreamer/ChangeLog:    If someone is foolish enough to compare 2 fractions with denominator =
/usr/share/doc/packages/man-pages/Changes.old:    "A foolish consistency is the hobgoblin of little minds, adored by
/usr/share/doc/packages/man-pages/Changes.old:    "A foolish consistency is the hobgoblin of little minds, adored by
/usr/share/doc/packages/graphviz/FAQ.html:<A HREF="http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dnhfact/html/hfactor8_5.asp">How To Avoid Foolish Consistency</A>
/usr/share/doc/packages/grub/ChangeLog: Add BIOS drive remapping support for chain-loading some foolish
/usr/share/doc/packages/grub/NEWS: chain-load some foolish operating systems (such as DOS) even if such
/usr/share/doc/packages/gstreamer-0_10/ChangeLog:    gst/gstvalue.c: If someone is foolish enough to compare 2 fractions with denominator = 0, return UNORDERED rather than...
/usr/share/doc/packages/gstreamer-0_10/ChangeLog:    If someone is foolish enough to compare 2 fractions with denominator =
/usr/share/doc/packages/gnome-vfs2/ChangeLog:    Fix foolish crasher
linux-ab2x:/usr/share/doc #
```

Example 1 - Searching with OR:

Find lines in /etc/passwd that start with the letter "r" followed by either "p" or "t": `egrep '^r(p|t)' /etc/passwd`

This command will return lines like

Getting a grep

```
rpc:x:490:65534:user for rpcbind:/var/lib/empty:/sbin/nologin  
rtkit:x:492:490:RealtimeKit:/proc:/bin/false
```

Shell scripting — More Key Topics

Examples of Using Globs

Automating Tasks via Shell Scripting

Good Script Design

Characteristics of a Good Shell Script: Focus on One Task: Should do one thing and do it well.

Standard Error

Standard Error (stderr): A parallel stream to stdout specifically for error messages.

Redirection of Streams

Redirection Symbols: Redirects a file's contents into a command's stdin stream.

Redirecting Standard Input

Sends a file's contents to a program's stdin stream. Example: `sort < file1`

Explanation:

`2>`: Redirects stderr (file descriptor 2) to a file named errors. stdout remains on the default target (console) or can be ...

Shell scripting — More Key Topics

Outcome:

Good data is stored in gooddata. Errors are stored in baddata.

/dev/tty:

Refers to the terminal connected to the program being executed, ensuring output is directed to the controlling terminal. Redirect ...

Pipe (|):

Chains multiple programs, sending the output of one as input to the next, often for filtering or modifying data. Send output to a ...

Example Command for Formatting and Printing:

`cat codesamp.c | nl | pr | lpr cat`: Displays content of codesamp.c.

Executing Multiple Commands:

Use special characters to run multiple commands with one Enter press. Combine commands for simple execution or use advanced logic ...

Semicolon (;):

Executes commands independently, without regard to each other's exit status. Example: `make modules ; echo "DO MAKE ...`

Lab 21 — Bash Scripting

Practice core Bash scripting constructs (variables, conditionals, loops, functions, option parsing, regex, file tests) and assemble a defensive backup script, then ...

You'll build — Write a series of Bash scripts exercising each language feature, build a real /etc backup script, and lint everything ...

Key commands — shebang, parameter expansion, arrays, test operators `[[ ]]`, `getopts`, `IFS`

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — `labs/lab-21-*/README.md` on the course GitHub repo

Bash Scripting

Step 1 — Set up a scripts directory

```
$ mkdir -p /root/scripts && cd /root/scripts
```

Step 2 — Variables, command substitution, parameter expansion

```
$ name="Linux+"
```

Step 3 — Conditionals, case, and test operators

```
$ num=${1:-0}
```

Step 4 — Loops, functions, and exit codes

```
$ log() { echo "[$(date +%T)] $*"; }
```

Bash Scripting

Step 5 — getopt and IFS-based CSV parsing

```
$ verbose=0
```

Step 6 — Build a real backup script

```
$ set -euo pipefail
```

Step 7 — Lint with shellcheck

```
$ shellcheck /root/scripts/*.sh || true
```

✓ **Verify** — A robust script uses `set -euo pipefail`, quotes variables, and passes shellcheck cleanly.

4.3 — Python basics

Develop scripts using environments, packages, and data types.

Exam objective — Summarize Python basics used for Linux system administration

Hands-on — Lab 22

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 22 — Python for System Administration

Use Python for common Linux admin tasks: create a virtual environment, exercise core data types, walk the filesystem, parse JSON, call a REST API with requests, and ...

You'll build — Set up a venv, write scripts covering data types, `os.walk`, JSON parsing, and HTTP requests, then organize code into a ...

Key commands — `venv/pip`, core types (`bool/int/float/str/list/dict`), type hints, `os.walk`, `json` module, `requests`

Workbench — Killercode Ubuntu Playground, free in the browser

Full walkthrough — `labs/lab-22-*/README.md` on the course GitHub repo

Python for System Administration

Step 1 — Check Python and create a virtual environment

```
$ python3 --version
```

Step 2 — Install dependencies

```
$ pip install requests black flake8
```

Step 3 — Exercise core data types

```
$ """Demonstrate the built-in Python data types."""
```

Step 4 — Walk /etc, count files, emit JSON

```
$ """Walk /etc and print a JSON summary."""
```

Python for System Administration

Step 5 — Parse a JSON file with the json module

```
$ {
```

Step 6 — Call a public API with requests

```
$ """Call a public test API and print selected fields."""
```

Step 7 — Use a package module and apply PEP 8

```
$ mkdir -p ~/pylab/helpers
```

✅ **Verify** — A venv isolates dependencies, requests loudly fails on bad HTTP, and black+flake8 enforce PEP 8 automatically.

4.4 — Version control

Manage code with Git workflows and tagging.

Exam objective — Given a scenario, implement version control using Git

Hands-on — Lab 23

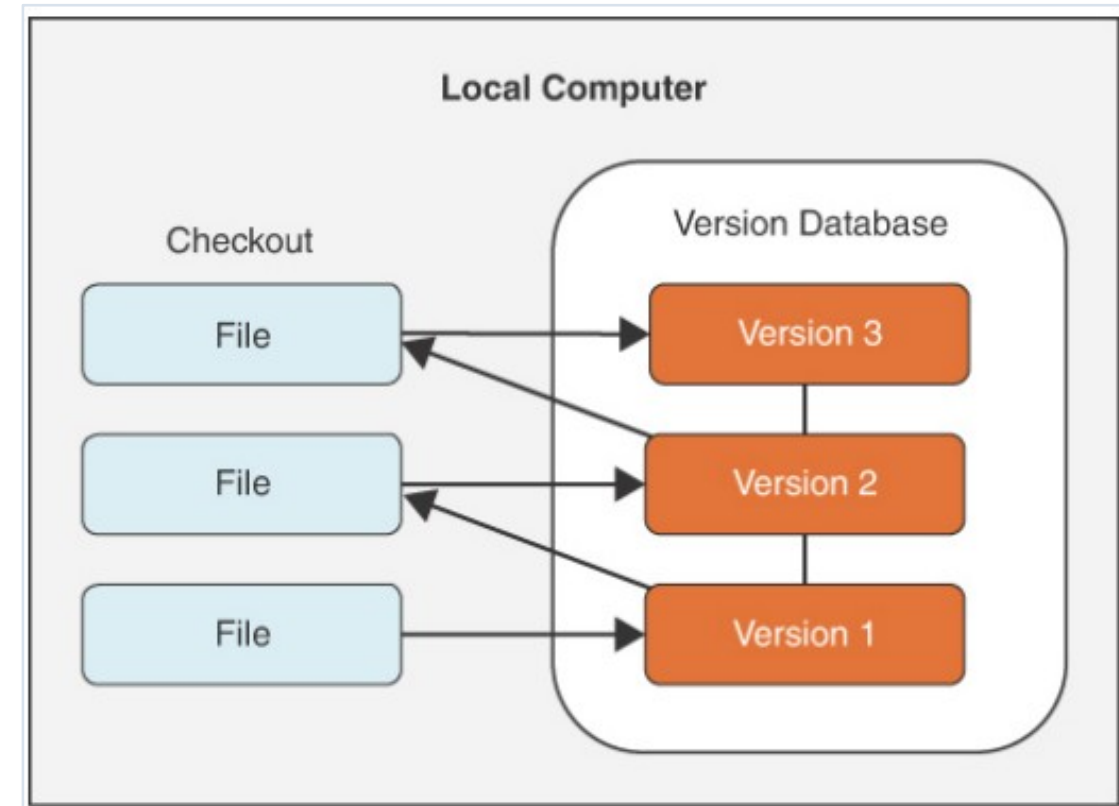
Type it, break it, fix it — the exam rewards hands that have done the work.

First Generation:

Simple version control where developers worked on the same system.

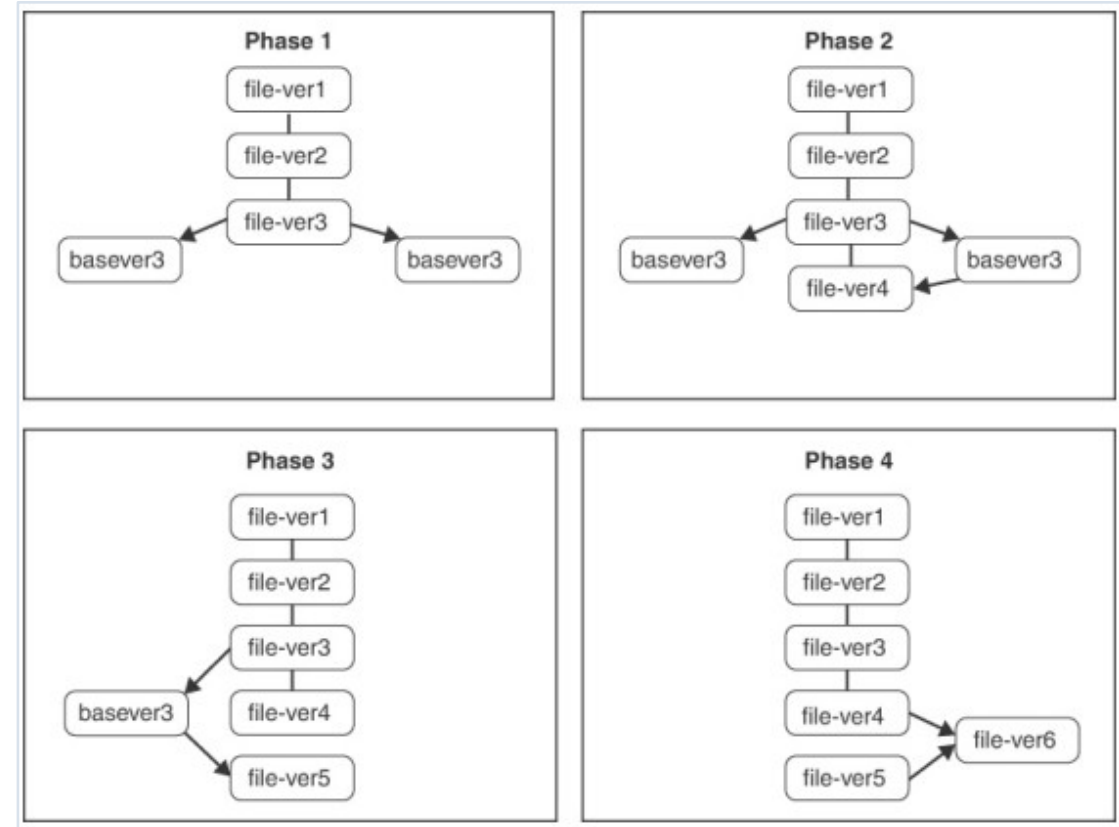
Used file locking: When a developer checked out a file, it was locked, preventing others from editing it.

Version Control Concepts



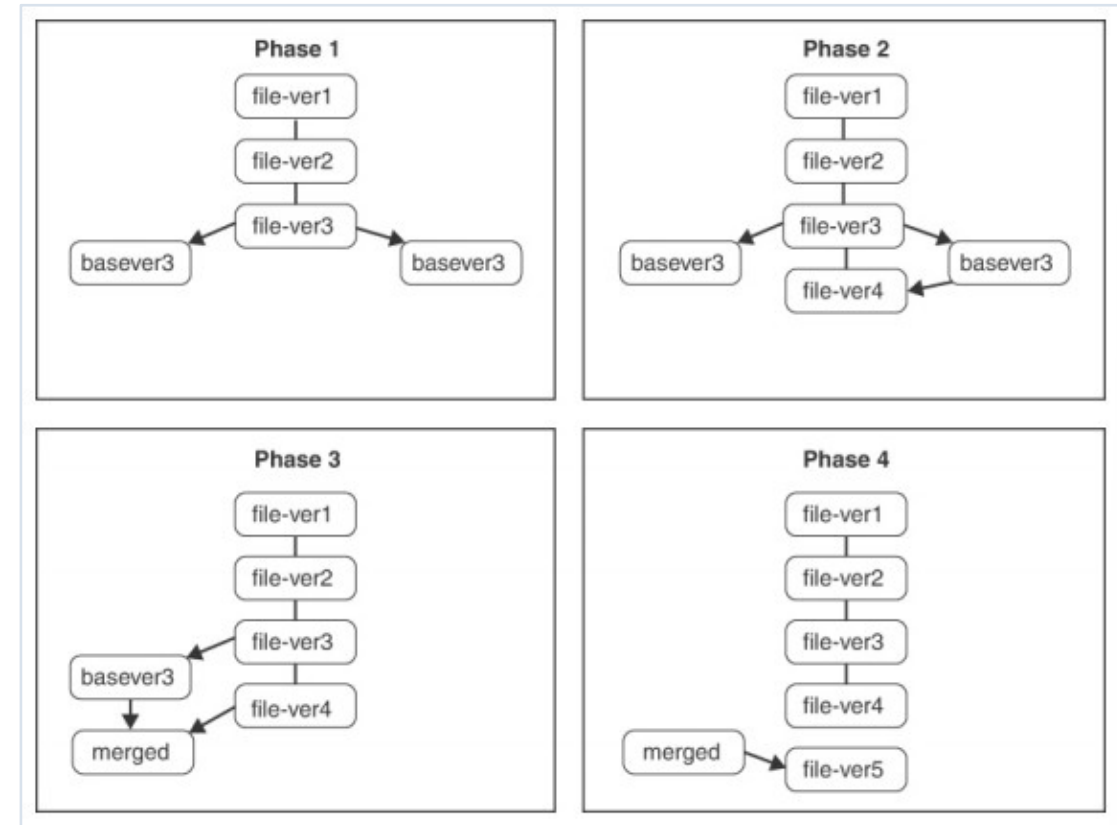
The Third Generation

Performing Basic Version Control Using Git



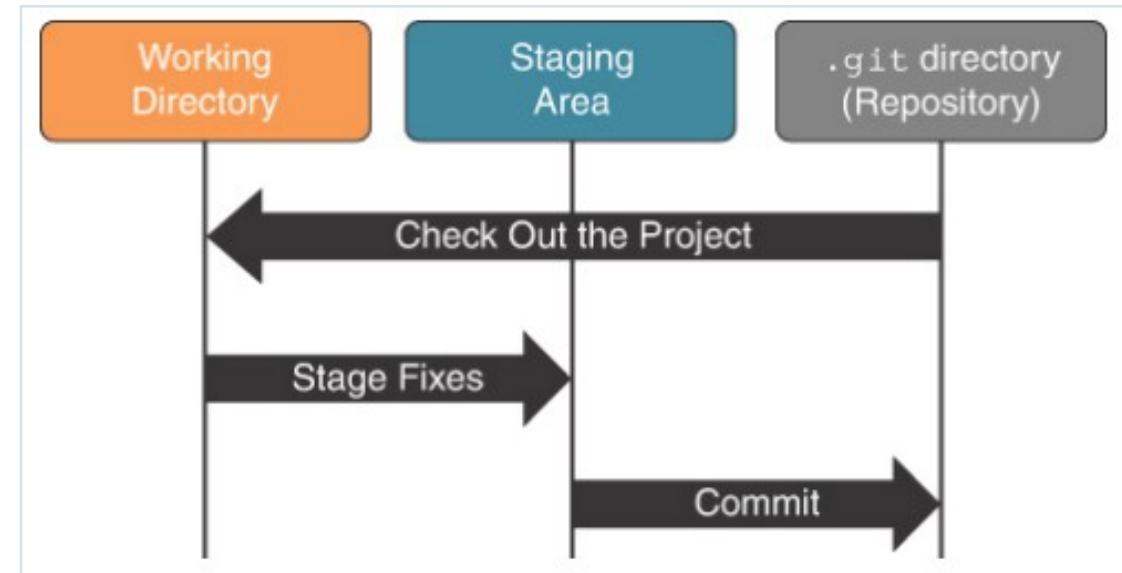
Second-generation merge and commit

Performing Basic Version Control Using Git



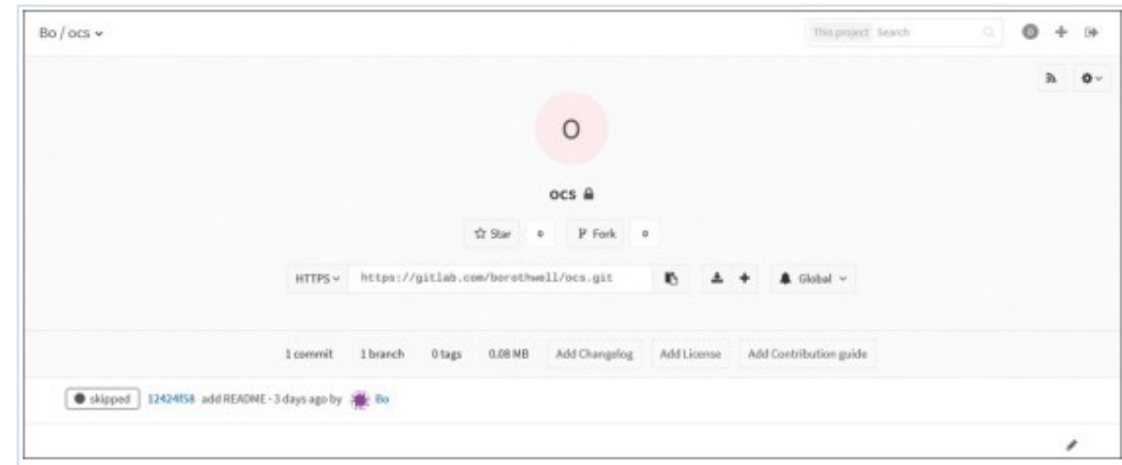
Git stages

Git Stages



Configuring Git

Performing Basic Version Control Using Git



shows that the push was successful.

Configuring Git



Changes Not Staged for Commit:

If changes are made to a file but not staged:

Getting the Status of Files

```
~/ocs$ git status
On branch main
Your branch is up-to-date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working
  directory)

        modified:   first.sh

no changes added to commit (use "git add" and/or "git commit -a")
```

Committed Locally but Not Pushed:

If a file is committed to the local repository but not pushed to the remote:

Getting the Status of Files

```
~/ocs$ git commit -m "demonstrating status"
[main 9eb721e] demonstrating status
1 file changed, 2 insertions(+), 1 deletion(-)
~/ocs$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
    (use "git push" to publish your local commits)

nothing to commit, working directory clean
```

Telling Git to Ignore a File

Performing Basic Version Control Using Git

```
~/ocs$ touch notes
~/ocs$ git status -s
?? notes
~/ocs$ vi .gitignore      #added notes as shown below:
~/ocs$ cat .gitignore
notes
~/ocs$ git status -s
?? .gitignore
```


.gitignore file itself must also be placed in the .gitignore file

Telling Git to Ignore a File

```
~/ocs$ git status -s
?? .gitignore
~/ocs$ vi .gitignore      #added .gitignore as shown below:
~/ocs$ cat .gitignore
notes
.gitignore
~/ocs$ git status -s
```

Git Status Example

Upon running the `git status` command after returning to your project, you might see the following output, indicating that a file (`hidden.sh`) has been ...

Executing Diffs

```
~/ocs$ git status
On branch main
Your branch is up-to-date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working
  directory)

        modified:   hidden.sh

no changes added to commit (use "git add" and/or "git commit -a")
```

git diff Example

If you want to compare the current state of a file (hidden.sh) in the working directory with the version in the last commit, you can use the git diff ...

Executing Diffs

```
~/ocs$ git diff hidden.sh | cat -n
 1  diff --git a/hidden.sh b/hidden.sh
 2  index 05151ce..714482b 100644
 3  --- a/hidden.sh
 4  +++ b/hidden.sh
 5  @@ -1,4 +1,5 @@
 6  #!/bin/bash
 7  #hidden.sh
 8
 9  +echo "Hidden files:"
10  ls -ld .* $1
```

A Single Difference

Performing Basic Version Control Using Git

```
~/ocs$ git diff hidden.sh | cat -n
 1  diff --git a/hidden.sh b/hidden.sh
 2  index 05151ce..6de92ae 100644
 3  --- a/hidden.sh
 4  +++ b/hidden.sh
 5  @@ -1,4 +1,5 @@
 6  #!/bin/bash
 7  #hidden.sh
 8
 9  -ls -ld .* $1
10  +echo "Hidden files:"
11  +ls -ldh .* $1
```

Staged vs. Committed Differences

Performing Basic Version Control Using Git

```
~/ocs$ git diff --staged hidden.sh | cat -n
 1  diff --git a/hidden.sh b/hidden.sh
 2  index 05151ce..6de92ae 100644
 3  --- a/hidden.sh
 4  +++ b/hidden.sh
 5  @@ -1,4 +1,5 @@
 6  #!/bin/bash
 7  #hidden.sh
 8
 9  -ls -ld .* $1
10  +echo "Hidden files:"
11  +ls -ldh .* $1
```

Whitespace Mystery

Dealing with Whitespace

```
~/ocs$ git diff --staged hidden.sh | cat -n
 1  diff --git a/hidden.sh b/hidden.sh
 2  index 6de92ae..519eb3c 100644
 3  --- a/hidden.sh
 4  +++ b/hidden.sh
 5  @@ -1,5 +1,5 @@
 6  #!/bin/bash
 7  -#hidden.sh
 8  +#hidden.sh
 9
10  echo "Hidden files:"
11  ls -ldh .* $1
```

The `git diff --check` command checks for trailing whitespace in the ...

Comparing Versions

```
~/ocs$ git diff --staged --check | cat -n
1  hidden.sh:2: trailing whitespace.
2  +#hidden.sh
```

You can use the git diff command to compare files in different

...

Comparing Branches

```
~/ocs$ git diff --name-status main..test  
M      hidden.sh
```


Between Branches

Performing Basic Version Control Using Git

```
~/ocs$ git diff main:hidden.sh test:hidden.sh
diff --git a/main:hidden.sh b/test:hidden.sh
index 519eb3c..804fcf7 100644
--- a/main:hidden.sh
+++ b/test:hidden.sh
@@ -1,5 +1,5 @@
  #!/bin/bash
-#hidden.sh
+#hidden.sh

-echo "Hidden files:"
-ls -ldh .* $1
+echo "Listing only hidden files:"
+ls -ld .* $1
```

Example Command:

```
git difftool --tool=<tool> file
```

Comparing Branches

```
ocs@ubuntu: ~/ocs
#!/bin/bash
#hidden.sh

echo "Hidden files:"
ls -ldh .* $1

exit 0

<S54iy hidden.sh [R0] 1,1      All hidden.sh [R0]      1,1      All
"hidden.sh" [readonly] 7L, 71C
```

Features Branch

Merging Files

```
~/ocs$ more showmine.sh
#!/bin/bash
#showmine.sh

echo "Your processes:"
ps -fe | grep $USER | more
~/ocs$ git checkout -b feature127
Switched to a new branch 'feature127'
~/ocs$ vi showmine.sh
~/ocs$ more showmine.sh
#!/bin/bash
#showmine.sh

echo -n "Enter name username or press enter: "
read person

echo "${person:-$USER} processes:"
ps -fe | grep "^${person:-$USER}" | more
```

To merge changes, ensure you're in the branch you want to merge into ...

Merging Files

```
~/ocs$ git merge feature127
Updating 4810ca8..2e5defa
Fast-forward
 showmine.sh | 7 +++++--
1 file changed, 5 insertions(+), 2 deletions(-)
```

The current version of showmine.sh that has been committed to the ...

Merging Files

```
~/ocs$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 4 commits.
    (use "git push" to publish your local commits)
ocs@ubuntu:~/ocs$ more showmine.sh
#!/bin/bash
#showmine.sh
```

Merged File

Merging Files

```
~/ocs$ cat -n showmine.sh
 1  #!/bin/bash
 2  #showmine.sh
 3
 4  <<<<<<< HEAD
 5  echo "Your programs:"
 6  ps -fe | grep $USER | more
 7
 8  echo -n "Enter a PID to stop: "
 9  read proc
10  kill $proc
11
12  =====
13  echo -n "Enter name username or press enter: "
14  read person
15
16  echo "${person:-$USER} processes:"
17  ps -fe | grep "^${person:-$USER}" | more
18  >>>>>> main
```

git mergetool using vimdiff

Merging Files

[illegible]

Version control — More Key Topics

Three Generations of Version Control:

Local Version Control: Stored versions locally, no collaboration. Centralized Version Control: Used a central server for multiple ...

Merge Process in Version Control:

Scenario: Bob and Sue check out version 5 of a file (abc.txt). Bob checks in his changes, creating version 6. Later, Sue checks ...

Third Generation: Distributed Version Control Systems ...

Developers checkout the entire project rather than individual files. Central repository server still stores all project files.

Key Difference Between Second and Third Generation ...

Second Generation (CVCS): Merge happens first, then the new version is committed to the repository.

Phases:

Phase 1: Two developers check out a file based on version 3. Phase 2: One developer checks in their changes, creating version 4.

Challenges:

Straight Line Development: The repository progresses linearly (ver1 → ver2 → ver3 → ver4 → ver5). Reluctance to Commit: Merging ...

Version control — More Key Topics

Third-Generation Process: Directed Acyclic Graph (DAG):

DAG is a more complex but better technique used in third-generation version control. Unlike linear development, DAG allows ...

Third-Generation Commit and Merge (DAG):

Phase 1 and 2: Same as the earlier process—developers check out the file and commit their changes. Phase 3: The second check-in ...

Advantages of DAG Over Single Line Development:

Frequent Commits: Developers can commit regularly without immediate merging. Delegated Merging: A specific developer can handle ...

Installing Git:

Check if Git is installed: Run `which git` to see if Git is installed. If installed, it shows the path, e.g., `/usr/bin/git`.

For Debian-based systems:

Use `dpkg -l git` to check if Git is installed. To install Git, run: `apt-get install git`

For Red Hat-based systems:

Use `rpm -q git` to check if Git is installed. To install Git, run: `yum install git`

Lab 23 — Git Version Control

Work through Git end to end: init, commit, branch, merge, resolve conflicts, stash, tag, rewrite history with reset and rebase, and push/pull against a local bare ...

You'll build — Create a working repo and a bare origin repo, then practice the full Git workflow from first commit through conflict ...

Key commands — git init/config, branch/merge --no-ff, conflict resolution, stash/tag, reset --soft/rebase, bare remote

Workbench — Killercode Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-23-*/README.md on the course GitHub repo

Git Version Control

Step 1 — Install Git and set identity

```
$ apt update && apt install -y git
```

Step 2 — Initialise a repo and make the first commit

```
$ mkdir -p /root/lab && cd /root/lab
```

Step 3 — Branch, modify, merge

```
$ git switch -c feature/banner
```

Step 4 — Create and resolve a conflict

```
$ git switch -c feature/edit
```

Git Version Control

Step 5 — diff, stash, tag

```
$ echo "draft note" >> README.md
```

Step 6 — Rewrite history: reset and rebase

```
$ echo "oops" > scratch.txt
```

Step 7 — Add a remote and push/pull

```
$ git init --bare /tmp/origin.git
```

✅ **Verify** — Git tracks history through commits and branches; merge/rebase integrate work and pull --rebase keeps a linear ...

4.5 — AI best practices

Apply code generation and prompt engineering responsibly.

Exam objective — Summarize best practices and responsible uses of AI

Hands-on — Lab 24

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 24 — Responsible AI Use for Linux Administrators

Adopt a verify-before-paste workflow for AI-generated Linux commands: redact secrets before sending prompts to hosted models, lint and sandbox suggestions, weigh ...

You'll build — Build a verification workspace, redact secrets in prompts, shellcheck and sandbox an AI-suggested command, and author a ...

Key commands — verify-before-paste, secret redaction, local vs hosted LLMs, shellcheck, API key via env var, AI usage policy

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-24-*/README.md on the course GitHub repo

Responsible AI Use for Linux Administrators

Step 1 — Set up a verification workspace

```
$ apt update && apt install -y shellcheck jq curl
```

Step 2 — Try a local LLM (optional)

```
$ curl -fsSL https://ollama.com/install.sh | sh || echo "Ollama install skipped"
```

Step 3 — Hosted-API placeholder workflow with redaction

```
$ export LLM_API_KEY="REDACTED-EXAMPLE"
```

Step 4 — Verify an AI-suggested command

```
$ find /var/log -type f -printf '%s %p\n' 2>/dev/null \
```

Responsible AI Use for Linux Administrators

Step 5 — Build a verification checklist

```
$ - [ ] Did I read every line of the suggested command/script?
```

Step 6 — Draft a corporate AI usage policy

```
$ - Local models on company hardware: allowed for internal, non-secret data.
```

Step 7 — Compare local vs hosted at a glance

```
$ Local model (Ollama, llamafile)
```

✅ **Verify — Always verify before paste: redact secrets, shellcheck, sandbox, then run; choose local vs hosted models per ...**

Recap — Automation, Orchestration, and Scripting

Lab 20 — Infrastructure as Code with Ansible

Lab 21 — Bash Scripting

Lab 22 — Python for System Administration

Lab 23 — Git Version Control

Lab 24 — Responsible AI Use for Linux Administrators

End of Day 4

See you tomorrow — 9:30 AM

Troubleshooting

22% of the exam | Objectives 5.1–5.5 | Labs 25–30

Type it, break it, fix it — the exam rewards hands that have done the work.

Troubleshooting — Objectives

5.1 · System monitoring

Monitor health, logs, and event alerts.

5.2 · Hardware/storage

Diagnose boot, mount, and repair issues.

5.3 · Networking

Resolve firewall, routing, DNS, and connectivity problems.

5.4 · Security

Fix SELinux, permission, and vulnerability issues.

5.5 · Performance

Analyze CPU, memory, I/O, and optimize response times.

Key Concepts — Troubleshooting

Monitoring vocabulary (SLA/SLI/SLO) and tools (SNMP, Prometheus, node_exporter) establish a baseline.

Troubleshooting is a loop: baseline → reproduce → identify with the right counter → remediate → verify.

Storage/OS faults (ENOSPC, inode exhaustion, failed units) and network faults (DNS, routing, MTU) each have a signature.

Security faults (SELinux, ACLs, certs, lockout) and performance faults (CPU, memory, I/O) close out the domain.

5.1 — System monitoring

Monitor health, logs, and event alerts.

Exam objective — Summarize monitoring concepts and configurations

Hands-on — Lab 25

Type it, break it, fix it — the exam rewards hands that have done the work.

Lab 25 — Monitoring Concepts and Tools

Learn the monitoring vocabulary (SLA/SLO/SLI) and stand up a working monitoring stack to produce metrics, alerts, webhooks, health checks, and aggregated logs.

You'll build — Install SNMP, node_exporter, Prometheus and Alertmanager, then write and validate alert rules, test webhooks, and ...

Key commands — SLA vs SLO vs SLI, SNMP OID/MIB walk, Prometheus scrape config, node_exporter metrics endpoint, promtool check rules, ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-25-*/README.md on the course GitHub repo

Monitoring Concepts and Tools

Step 1 — Install SNMP and walk the system MIB

```
$ apt update
```

Step 2 — Install and probe node_exporter

```
$ apt install -y prometheus-node-exporter
```

Step 3 — Install Prometheus and scrape node_exporter

```
$ apt install -y prometheus
```

Step 4 — Write and validate an alert rule

```
$ groups:
```


Monitoring Concepts and Tools

Step 5 — Simulate a webhook receiver

```
$ (nc -l -p 8000 -q 1 >/tmp/webhook.log 2>&1 &) ; sleep 1
```

Step 6 — Health check pattern

```
$ curl -fsS http://localhost:9100/metrics >/dev/null && echo "node_exporter OK"
```

Step 7 — Log aggregation via journalctl

```
$ journalctl -u prometheus -n 20 --no-pager
```

✅ **Verify** — `node_exporter`, Prometheus, and its health/ready endpoints all respond, and promtool confirms the alert rule ...

5.2 — Hardware/storage

Diagnose boot, mount, and repair issues.

Exam objective — Given a scenario, troubleshoot hardware, storage, and OS issues

Hands-on — Lab 26

Type it, break it, fix it — the exam rewards hands that have done the work.

Causes and Symptoms of High Latency

The possible causes of I/O wait time include the following

Storage system bottlenecks

RAM shortage

Higher use of swap

Sustained heavy-duty read and write access to devices, particularly off-system sources

RAID array (I/O) errors, regeneration processing, and low RAID system RAM

Diagnosing with the top Command

In the top command output, the key information for diagnosing high latency is found in the %Cpu(s) line, specifically to the right of the "wa" (wait time) ...

The top command is easy to use—just type the command, and it will display system information. Despite its simplicity, it provides valuable insights into ...

```
top - 21:42:48 up 1 min, 1 user, load average: 1.26, 0.62, 0.23
Tasks: 119 total, 1 running, 118 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.3 sy, 0.0 ni, 99.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 976.9 total, 422.2 free, 143.5 used, 411.2 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 685.7 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
721	root	20	0	0	0	0	I	0.3	0.0	0:00.04	kworker/0:5-mm_percpu_wq
1	root	20	0	101900	11544	8504	S	0.0	1.2	0:02.78	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_par_gp
5	root	20	0	0	0	0	I	0.0	0.0	0:00.02	kworker/0:0-cgroup_destroy
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H-kblockd

Diagnosing with the vmstat Command

The **vmstat** command is a versatile tool for monitoring system performance, providing statistics not only for virtual memory but also for processes, memory, ...

It can be used for a single snapshot of system status or for continuous updates over time. Running **vmstat** without options gives a static output showing a ...

You can control the **vmstat** command by specifying options and a delay, allowing it to update its output at set intervals. This differs from the **top** ...

To make **vmstat** display a continuous list of outputs that scrolls down the screen, use a delay value, such as 2, to refresh the information at regular ...

You can quit the continual display by pressing **Ctrl+c**, or if you want to display the output only a specified number of times, with a 2-second delay, you ...

This would run the **vmstat** command with no options at 2-second intervals for a total of ten times and then exit.

```
ubuntu@primary:~$ vmstat 2
```

procs		-----memory-----				---swap--		-----io----		-system--		-----cpu-----				
r	b	swpd	free	buff	cache	si	so	bi	bo	in	cs	us	sy	id	wa	st
0	0	0	351968	53788	450160	0	0	87	18	103	63	0	0	99	0	0
0	0	0	351960	53788	450160	0	0	0	0	79	23	0	0	100	0	0
0	0	0	351960	53788	450160	0	0	0	0	93	24	0	0	100	0	0
0	0	0	351960	53788	450160	0	0	0	0	86	24	0	0	100	0	0
0	0	0	351960	53788	450160	0	0	0	0	75	21	0	0	100	0	0
0	0	0	351960	53788	450160	0	0	0	0	88	22	0	0	100	0	0

Fixing the Problem

To fix high latency, you need to identify the processes causing the issue. One effective way to do this is by using the `iotop` command, which works like ...

Running the `iotop` command by itself will provide basic output, but you can get more focused and helpful results by using specific subcommands. An example ...

One of the most useful `iotop` options is `--only`, which specifies that only the processes that are generating or conducting I/O be shown. You can either run ...

```
Total DISK READ: 0.00 B/s | Total DISK WRITE: 3.78 K/s
Current DISK READ: 0.00 B/s | Current DISK WRITE: 11.33 K/s
  TID  PRIO  USER     DISK READ  DISK WRITE  SWAPIN     IO>   COMMAND
 272  be/3  root      0.00 B/s   3.78 K/s   0.00 %    6.84 % [jbd2/vda1-8]
   1  be/4  root      0.00 B/s   0.00 B/s   0.00 %    0.00 % [init]
   2  be/4  root      0.00 B/s   0.00 B/s   0.00 %    0.00 % [kthreadd]
   3  be/0  root      0.00 B/s   0.00 B/s   0.00 %    0.00 % [rcu_gp]
   4  be/0  root      0.00 B/s   0.00 B/s   0.00 %    0.00 % [rcu_par_gp]
   6  be/0  root      0.00 B/s   0.00 B/s   0.00 %    0.00 % [kworker/0:0H-kblockd]
```

Causes and Symptoms of Low Throughput

The possible causes of low throughput for storage include the following

Failed disk, spinning or solid state

Malfunctioning storage controller RAID disk failure, impending failure, or regeneration

Too many requests from clients to the system's resources or applications

Memory issues, causing rereads and lost pages

Symptoms of low throughput might include

Diagnosing with the df Command

The **df** (disk free) command is used to view statistics for mounted filesystems, including spinning disks, solid-state drives, and virtual filesystems like ...

The output of the **df** command with the **-hT** options, which display disk utilization in percentage format and include the Type column (highlighted) for ...

The **df** command is useful because it provides a comprehensive view of all mounted filesystems in one place. Focus on the Used and Avail columns, or simply ...

```
ubuntu@primary:~$ df -hT
Filesystem      Type      Size  Used Avail Use% Mounted on
udev            devtmpfs  472M   0    472M   0% /dev
tmpfs           tmpfs     98M   932K   97M   1% /run
/dev/vda1       ext4      4.7G   2.8G   2.0G   59% /
tmpfs           tmpfs     489M   0    489M   0% /dev/shm
tmpfs           tmpfs     5.0M   0    5.0M   0% /run/lock
tmpfs           tmpfs     489M   0    489M   0% /sys/fs/cgroup
/dev/loop1      squashfs  47M    47M    0 100% /snap/snapd/16292
/dev/loop2      squashfs  62M    62M    0 100% /snap/core20/1587
/dev/loop3      squashfs  1.2M   1.2M    0 100% /snap/multipass-sshfs/145
/dev/loop4      squashfs  68M    68M    0 100% /snap/lxd/22526
/dev/loop0      squashfs  68M    68M    0 100% /snap/lxd/22753
/dev/loop5      squashfs  128K   128K    0 100% /snap/bare/5
/dev/vda15      vfat      105M   5.2M  100M   5% /boot/efi
tmpfs           tmpfs     98M    0    98M   0% /run/user/1000
:/Users/rossb  fuse.sshfs 1000G   0 1000G   0% /home/ubuntu/Home
/dev/loop8      squashfs  64M    64M    0 100% /snap/core20/1695
/dev/loop7      squashfs  50M    50M    0 100% /snap/snapd/17576
```


Diagnosing with the iostat Command

The **iostat** command displays input and output statistics for the system, gathering data from the **/proc** and **/sys** directories. It also helps correlate the ...

For example, if you wanted to just see the raw iostat output for your mounted filesystems, you'd run the command by itself,

The **iostat** command can generate a lot of data, which may be too much for a standard screen. To make the output more manageable, you can use specific ...

-h: Displays data in a more human-readable format (e.g., 1M for 1024 kilobytes).

-m: Forces output to be displayed in megabytes, rather than kilobytes.

-z: Suppresses output for devices with no activity during the measurement period.

```
ubuntu@primary:~$ iostat
Linux 5.4.0-99-generic (primary)      11/18/22      _x86_64_      (1 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           0.58    0.43    0.53    0.06    0.00   98.40

Device            tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd
loop0              0.00         0.02         0.00         0.00       2423         0         0
loop1              0.17         0.17         0.00         0.00      27151         0         0
loop2              0.01         0.02         0.00         0.00       2515         0         0
loop3              0.01         0.01         0.00         0.00       1062         0         0
loop4              0.00         0.02         0.00         0.00       2422         0         0
loop5              0.00         0.00         0.00         0.00         7         0         0
loop6              0.00         0.01         0.00         0.00       958         0         0
loop7              0.12         0.13         0.00         0.00      19884         0         0
loop8              0.00         0.00         0.00         0.00       477         0         0
loop9              0.00         0.00         0.00         0.00         4         0         0
scd0               0.00         0.00         0.00         0.00       266         0         0
vda                1.01         9.29        26.62        13.86    1467172    4204409    2188453
```

Causes and Symptoms of Capacity Issues

The possible causes of capacity issues for Linux include the following

Disk controller/RAID controller failure

Hard disk or SSD failure

RAID software failure

Disk corruption

Filesystem corruption

Diagnosing with the df Comma

The **df** (disk free) command in this scenario displays disk usage statistics, including the total number of inodes, those in use, and those free. It can ...

The **df -hT** command is used to diagnose capacity-related issues. The **-T** option adds a column for filesystem type, while the **-h** option displays the output ...

The highlighted line shows the **/dev/vda1** filesystem, which is of the type **ext4**. The important statistics here are the values in the **Size**, **Used**, **Avail**, and ...

Additionally, for the purposes of checking to see how many inodes you may be using for each of your filesystems, use the **df -ih** command.

In the output, as highlighted in the example, the **Inodes** column shows the total number of inodes, the **lused** column displays how many are in use, the **lfree** ...

```
ubuntu@primary:~$ df -hT
Filesystem      Type      Size  Used Avail Use% Mounted on
udev            devtmpfs  472M   0    472M   0% /dev
tmpfs           tmpfs     98M   908K   97M   1% /run
/dev/vda1       ext4      4.7G  2.8G  2.0G  59% /
tmpfs           tmpfs     489M   0    489M   0% /dev/shm
tmpfs           tmpfs     5.0M   0    5.0M   0% /run/lock
tmpfs           tmpfs     489M   0    489M   0% /sys/fs/cgroup
/dev/loop0      squashfs  128K   128K   0  100% /snap/bare/5
/dev/loop2      squashfs   68M    68M   0  100% /snap/lxd/22753
/dev/loop3      squashfs   68M    68M   0  100% /snap/lxd/22526
/dev/loop1      squashfs   62M    62M   0  100% /snap/core20/1587
/dev/loop4      squashfs  1.2M   1.2M   0  100% /snap/multipass-sshfs/145
/dev/loop5      squashfs   50M    50M   0  100% /snap/snapd/17576
/dev/loop6      squashfs   64M    64M   0  100% /snap/core20/1695
/dev/loop7      squashfs   47M    47M   0  100% /snap/snapd/16292
/dev/vda15      vfat      105M   5.2M  100M   5% /boot/efi
tmpfs           tmpfs     98M   0    98M   0% /run/user/1000
:/Users/rossb  fuse.sshfs 1000G   0  1000G   0% /home/ubuntu/Home
```

Diagnosing with the find Command

the find command is used to locate and list files above a specified size, starting from a given directory. In the Figure, the find command is used to ...

The command used is slightly longer but here's a breakdown:

find /etc: Starts the search in the /etc directory, including all its contents.

-size +16K: Filters for files larger than 16KB.

-exec du -h {}: Executes the du -h command on each file found, displaying the human-readable size of each item.

;; Tells find to execute the -exec command multiple times for each file.

```
ubuntu@primary:/etc$ find /etc -size +16k -exec du -h {} \; 2> /dev/null
28K    /etc/apparmor.d/usr.lib.snapd.snap-confine.real
64K    /etc/udev/rules.d/70-snap.snapd.rules
28K    /etc/wpa_supplicant/functions.sh
192K   /etc/ssl/certs/ca-certificates.crt
28K    /etc/ld.so.cache
100K   /etc/lvm/lvm.conf
44K    /etc/grub.d/10_linux_zfs
20K    /etc/grub.d/10_linux
24K    /etc/mime.types
524K   /etc/ssh/moduli
88K    /etc/vmware-tools/vgauth/schemas/XMLSchema.xsd
```

Causes and Symptoms of Filesystem Corruption

Filesystem corruption can be caused by several factors, including

Hardware failures (or virtual hardware issues in virtual machines)

Ungraceful dismounting of a filesystem (e.g., USB drives)

Bad sectors on spinning disks or errors in flash memory on SSDs (rare)

Software issues in packages interacting with low-level disk operations (rare)

Power failures or abrupt, ungraceful shutdowns (common)

Diagnosing and Fixing Filesystem Corruption with the fsck Command

The **fsck** command is a suite of tools used to diagnose and repair filesystems. You can either specify a filesystem type as a parameter to the main **fsck** ...

Regardless of the specific error, you'll eventually need to use the **fsck** command suite to diagnose and repair the corrupted filesystem. The focus will be ...

Shows the dire warning message you would receive if you were to attempt to run **fsck** against a write-enabled filesystem (don't do it!).

While a filesystem is mounted as writable, you can perform a dry-run using the **-N** parameter with the **fsck** command. This simulates the actions of a real ...

All filesystems on a Linux device can be unmounted and checked, except for the root filesystem. While it's technically possible to remount the root ...

Switching to **rescue.target** will minimize running processes, allowing you to safely use the **fsck** command on the root filesystem. Alternatively, on an ...

```
ubuntu@primary:~$ sudo fsck -N
fsck from util-linux 2.34
[/usr/sbin/fsck.ext4 (1) -- /] fsck.ext4 /dev/vda1
[/usr/sbin/fsck.vfat (1) -- /boot/efi] fsck.vfat /dev/vda15
```

Diagnosing a Mismatch Issue

The main utility that will show you what the problem is, is the **e2fsck** program, which will report back something similar to the following output.

The main question is whether you simply resized the underlying partition or if you also created and formatted a new partition with the extra "free space." ...

```
$ e2fsck /dev/sda1
e2fsck 1.45.5 (20-Mar-2020)
The filesystem size (according to the superblock) is 1934
blocks
The physical size of the device is 178945672 blocks
Either the superblock or the partition table is likely to
corrupt!
Abort<y>?
```

Types of I/O Schedulers

There are two types of I/O schedulers the older version, used for years, and the newer multiqueue versions, which are the default in recent Ubuntu ...

Older I/O schedulers include:

Deadline: Prioritizes read requests to prevent starvation, making it ideal for read-heavy tasks like informational websites.

CFQ (Completely Fair Queuing): Allocates equal time for reads and writes, which can cause idle time if one operation takes less time than allocated.

Noop: Groups requests without sorting, best suited for SSDs or devices with no seek times, as there's no need for traditional disk head movements.

Newer I/O schedulers are multiqueue-specific and include:

Viewing the Current I/O Scheduler

You can view the current I/O scheduler for any block device by checking files in the `/sys` directory, using a specific formula to query the device's ...

In the output, the active I/O scheduler is displayed in square brackets (e.g., `[mqdeadline]`), while other available schedulers are listed without brackets.

```
$ cat /sys/block<devicename>/queue/scheduler  
[mq-deadline] none
```

Setting the Current I/O Schedule

To change an I/O scheduler, simply echo the desired scheduler string into the appropriate file in the `/sys` directory. For instance, to set the `mq-deadline` ...

Verify that this took effect with

As you can see, it simply made the other scheduler that's available, none, the active one.

```
$ cat /sys/block/sda/queue/scheduler  
[none] mq-deadline
```

Making the I/O Scheduler Change Persistent

To make an I/O scheduler change persistent across reboots, add the configuration line to the `/etc/udev/rules.d/60-schedulers.rules` file. If the file ...

You'll want to add a configuration line per block device that you want to make scheduler changes to, such as the `/dev/vda` device configuration line shown ...

Reboot the system, and when it comes back up, verify that the change is in place with the command

The method of editing the `/etc/udev/rules.d/60-schedulers.rules` file is the simplest and most direct way to make a persistent I/O scheduler change, though ...

```
$ cat /sys/block/sda/queue/scheduler  
[none] mq-deadline
```

Using the nvme Command-Line Too

The **nvme-cli** package includes around 100 tools for managing NVMe devices, allowing you to connect to management frameworks, retrieve and set information, ...

To get a listing of your NVMe devices, use the following command.

The output display from these utilities requires more characters than a standard terminal window can accommodate, indicating room for improvement in how ...

Another command sequence you can use to get more information about the NVMe devices on your system is shown.

For the Linux+ exam, the key takeaway for troubleshooting NVMe is understanding its purpose, how to use it effectively, and being aware of potential ...

```
$: sudo nvme list
```

Node	SN	Model
Namespace	Usage	Format

/dev/nvme0n1	SN123456789012345678	Parallels Virtual N
Disk	1	34.36 GB / 34.36 GB
B + 0 B		

Causes and Symptoms of SSD Issues

There are several possible causes of SSD issues

New devices conflicting with the drive

A cable being unplugged (if applicable; many SSDs are soldered into the motherboard)

Application or software issues

Module or driver issues

Bad blocks or sectors

Causes and Symptoms of RAID Failures

There are a number of possible causes of RAID failures, including

RAID controller suffers from a power surge

Rebuild operations fail, causing catastrophic data loss

Striping operations fail, causing data loss

Multiple disk failures cause complete data loss

Software RAID errors occur

Checking Device Health with smartctl

The smartctl command can report on most devices, but it often cannot retrieve much information for virtual machines or software-based drives. For example, ...

```
~ $: sudo smartctl -a /dev/sda -T verypermissive
smartctl 7.1 2019-12-30 r5022 [x86_64-linux-5.15.0-56-generic] (local build)
Copyright (C) 2002-19, Bruce Allen, Christian Franke, www.smartmontools.org

=== START OF INFORMATION SECTION ===
Device Model:      Ubuntu Linux 20.04-0 SSD
Serial Number:     77K2QZSVM2GM58JB40DD
Firmware Version:  F.TZBJ28
User Capacity:     68,719,476,736 bytes [68.7 GB]
Sector Sizes:      512 bytes logical, 4096 bytes physical
Rotation Rate:     Solid State Device
Device is:          Not in smartctl database [for details use: -P showall]
ATA Version is:    ATA8-ACS, ATA/ATAPI-5 T13/1321D revision 1
SATA Version is:   SATA 2.6, 3.0 Gb/s
Local Time is:     Sat Dec  3 20:23:36 2022 MST
SMART support is:  Unavailable - device lacks SMART capability.
```

Monitoring RAID Array Health

There are several ways to get information about your RAID array. We'll start with the simplest and go up from there.

To quickly monitor your RAID array, you can use the cat command to view the contents of the /proc/mdstat file.

You can additionally use a graphical tool like GParted, which when you click to get information on the array will display output like the example shown

```
[raid5] [raid4]
md0 : active raid0 nvme0n2[1] nvme0n1[0]
      67041280 blocks super 1.2 512k chunks
unused devices: <none>
```


Monitoring RAID Array Health with mdadm

RAID arrays managed with mdadm refer to software-based arrays, as hardware-based RAID typically uses its own set of utilities. mdadm can notify you of ...

The mdadm command will notify you if any of the following events occur

Fail

FailSpare

DegradedArray

TestMessage

```
$ ps aux | grep mdadm
root      13136  0.0  0.0   3188   128 ?        Ss   11:00
0:00 mdadm --monitor --scan --daemonize --mail=root@local
```

Causes and Symptoms of LVM Issues

The numerous possible causes of LVM issues include the following

Disk failures

LVM software failure

Power failure for external LVM

Symptoms of LVM issues might include any of the following

Filesystem errors

Diagnosing LVM Issues

To get more detailed information when using LVM commands, you can add the `-v` option, up to `-vvvv`, to commands like `lvs`, `pvs`, `vgs`, etc., to increase the ...

One useful diagnostic command is `lvmcong`, which shows the configuration information for your LVM setup. Running `lvmcong` on an LVM volume will display ...

The `lvmdump` command is another useful diagnostic tool that collects a large amount of data and packages it into a `tar.gz` or `.tgz` archive. This archive can ...

For more advanced data collection with `lvmdump`, use the `-a` option to gather additional information. To focus solely on the LVM setup metadata, use the `-m` ...

```
Gathering LVM & device-mapper version info...
Gathering dmsetup info...
Gathering process info...
Gathering console messages...
Gathering /etc/lvm info...
Gathering /dev listing...
Gathering /sys/block listing...
Creating report tarball in /root/ lvmdump-zakkhost.in-209.
```

Causes and Symptoms of Mount Options Issues

Possible causes of mount option issues include:

Incorrect setup in the `/etc/fstab` file preventing proper mounting.

Filesystem set up to be mounted only once.

Filesystem errors preventing mounting.

Incorrect or undetectable filesystem type.

Symptoms of mount issues may include:

Understanding Mount Option Gotchas

A common mistake is failing to specify the correct mount command options. At a minimum, the mount command requires the source device and the target mount ...

If the filesystem is set up in the `/etc/fstab` file, then this will work easily, give you no feedback, and enable you to then use the filesystem as you ...

```
# mount /dev/sda7 /data
```

What Are All These UUIDs in My fstab?

A common issue arises when people don't understand using UUIDs for mounting a filesystem, as they are used to partition paths like `/dev/sda5`. The problem ...

To fix the issue of changing device names, the concept of UUIDs (Unique Identifiers) was introduced. UUIDs remain consistent and won't change like device ...

A UUID is a unique identifier assigned to a filesystem during its creation, ensuring uniqueness across machines. A TYPE refers to the filesystem type, ...

```
/dev/sda5: UUID="3df1294e-27da-4408-8a62-2b571efd8eb6" TYPE='  
PARTUUID="1f305916-05"
```

What Is This errors= Option?

Filesystems are typically mounted as either read-only or read-write. For example, CD-ROMs or DVDs are usually read-only, while common filesystems like ...

When a filesystem error is detected before or during mounting, the options are:

Ignore the error (risky)

Mark the filesystem as having an error (still risky)

Panic and halt the system (not always needed)

(Re)mount the filesystem as read-only

```
errors=remount -ro
```

Unit Files

This section covers unit files, which are crucial for systemd, and their structure. Unit files are text-based and formatted with sections like [Unit] and ...

While there are many different unit files, only a handful of them are exam-testable. The next few sections will cover potential issues with the following ...

```
[Section Name]
Directive=avalue
Directive=othervalue

[Other Section Name]
Directive=somevalue
```


Networking Services

A common issue with systemd is that services seem to start all at once, but networking might not be up yet. This is often caused by a firewall starting ...

The network-online.target is the only network-related target unit that ensures services wait until the network is fully up before starting. To make a ...

The network-pre.target is a more passive unit used to order services that rely on the network. You can use the After=network-pre.target directive to ...

```
[Unit]
Otherdirectives=values
After=network-online.target
Wants=network-online.target
```

User

You can configure a systemd service to run as a specific user by specifying the user in the service unit file.

Specifying a user for a service is often necessary due to configuration needs. While most services have their own user settings, like NGINX, which defines ...

```
[Service]
Type=simple
User=rossb
Group=staff
```

Default

The `default.target` is a symbolic link to the designated default target unit. To verify if it's set correctly, use the `systemctl get-default` command.

The `systemctl get-default` shows the default target known to the system. To see the actual linked target, use the `ls -l` command to check the link.

The `default.target` is a symlink to `graphical.target`. If you're encountering target errors, first check this symlink and ensure the target isn't masked or ...

If you find that the `graphical.target` is masked, you can unmask it using the `unmask` subcommand.

After unmasking the target, you can verify it with the appropriate command to ensure it's unmasked and available. A reboot may be necessary to ensure ...

```
$ find /usr -iname default.target -exec ls -l {} \;  
lrwxrwxrwx 1 root root 16 Jun 27 2022 /usr/lib/systemd/syst  
default.target -> graphical.target
```

Multuser

A common issue is when the system boots to a text-only interface instead of the graphical subsystem, typically due to the default target being set to ...

This configures the system to attempt to (if not actually) start and end up with the graphical subsystem as the interface default.

```
$ sudo systemctl set-default graphical.target
```

Graphical

If your system boots to a text login instead of the graphical interface, it likely indicates a dependency issue or a problem loading a required service or ...

The rst thing to do is to look at the After= line in the /usr/lib/systemd/system/graphical.target le, which may look like this:

To ensure the graphical target is loaded, check that the necessary services and targets are running by using the appropriate command.

Look to see if there are any errors about gdm.service, the GNOME Display Manager, by running

To check the status of the gdm.service and related log entries, use journaling logs with the specified command for more detailed information.

This command displays the system journal entries from the latest boot. To view earlier entries, you can reference them in reverse chronological order.

```
$ journalctl --list-boots
-4 7702c4eb2f7d40f193b489b144c7b41e Thu 2022-07-07 12:41:15
2022-07-07 13:16:04 MDT
-3 8a9177df73c94980a4e30104d81e75c2 Mon 2022-07-18 07:42:28
2022-07-18 07:47:31 MDT
-2 05235a33547349c1a36facfefdf7a232 Mon 2022-07-18 07:47:37
2022-07-18 07:51:41 MDT
-1 89437fce32c24b95ba16aa8f5b3e7a84 Sat 2023-03-18 12:14:13
2023-03-18 12:16:33 MDT
0 c9c983a4e4f7460a8fb5bd9d28824855 Sat 2023-03-18 12:16:39
2023-03-20 20:29:04 MDTnetdev
```

Application Crash

When investigating an application crash, start by checking the system log files for any error messages or clues. On systemd-based systems, it's likely ...

To view recent log entries on a syslog-based system, use the `journalctl` command with a specified timeframe, such as `journalctl --since "5 minutes"`, to ...

You can adjust the `journalctl` command to view logs from any specified time period, such as "1 hour ago" or "2 days ago." Additionally, when an application ...

To enable core dumps on a systemd-based system

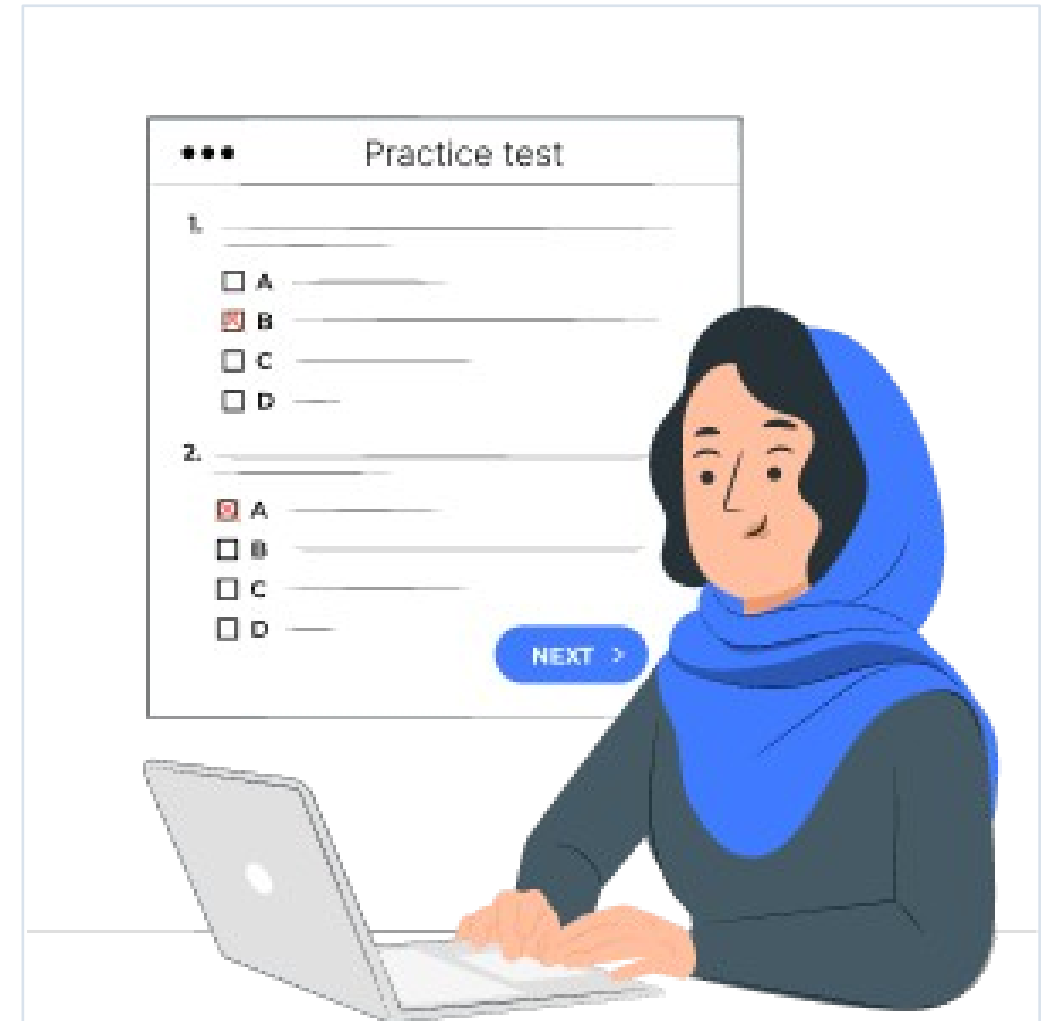
As the root user, set the `ulimit` for crash files to unlimited

Run the `sleep` command and crash it with

```
Mar 20 21:32:47 ubuntu-linux-22-04-desktop dbus-daemon[653]: [system] Activating via systemd: service name='net.reactivated.fprint' unit='fprintd.service' requested by:
Mar 20 21:32:47 ubuntu-linux-22-04-desktop systemd[1]: Starting Fingerprint Authentication Daemon...
Mar 20 21:32:47 ubuntu-linux-22-04-desktop dbus-daemon[653]: [system] Successfully activated service 'net.reactivated.fprint'
Mar 20 21:32:47 ubuntu-linux-22-04-desktop systemd[1]: Started Fingerprint Authentication Daemon.
Mar 20 21:32:47 ubuntu-linux-22-04-desktop gnome-shell[2189]: JS ERROR: Failed to initialize fprintd service: Gio.IOErrorEnum: GDBus.Error:net.reactivated.F
Mar 20 21:32:47 ubuntu-linux-22-04-desktop gnome-shell[2189]: JS ERROR: Failed to initialize fprintd service: Gio.IOErrorEnum: GDBus.Error:net.reactivated.F
Mar 20 21:32:49 ubuntu-linux-22-04-desktop gdm-password[165800]: gkr-pam: the password for the login keyring was invalid
Mar 20 21:32:49 ubuntu-linux-22-04-desktop dbus-daemon[2043]: [session uid=1000 pid=2043] Activating service name='org.freedesktop.FileManager1' requested by:
Mar 20 21:32:50 ubuntu-linux-22-04-desktop NetworkManager[60890]: <info> [1679369570.0323] agent-manager: agent[225548ba5b56b3d2,11.203/org.gnome.Shell.Net
Mar 20 21:32:50 ubuntu-linux-22-04-desktop ubuntu-appindicators@ubuntu.com[2189]: unable to update icon for software-update-available
Mar 20 21:32:50 ubuntu-linux-22-04-desktop ubuntu-appindicators@ubuntu.com[2189]: unable to update icon for livepatch
Mar 20 21:32:50 ubuntu-linux-22-04-desktop dbus-daemon[2043]: [session uid=1000 pid=2043] Successfully activated service 'org.freedesktop.FileManager1'
Mar 20 21:32:50 ubuntu-linux-22-04-desktop dbus-daemon[2043]: [session uid=1000 pid=2043] Activating service name='org.gnome.ArchiveManager1' requested by:
Mar 20 21:32:50 ubuntu-linux-22-04-desktop gnome-shell[2189]: Window manager warning: Overwriting existing binding of keysym 39 with keysym 39 (keycode 12).
Mar 20 21:32:50 ubuntu-linux-22-04-desktop gnome-shell[2189]: Window manager warning: Overwriting existing binding of keysym 32 with keysym 32 (keycode b).
Mar 20 21:32:50 ubuntu-linux-22-04-desktop gnome-shell[2189]: Window manager warning: Overwriting existing binding of keysym 33 with keysym 33 (keycode c).
Mar 20 21:32:50 ubuntu-linux-22-04-desktop gnome-shell[2189]: Window manager warning: Overwriting existing binding of keysym 34 with keysym 34 (keycode d).
Mar 20 21:32:50 ubuntu-linux-22-04-desktop gnome-shell[2189]: Window manager warning: Overwriting existing binding of keysym 35 with keysym 35 (keycode e).
Mar 20 21:32:50 ubuntu-linux-22-04-desktop gnome-shell[2189]: Window manager warning: Overwriting existing binding of keysym 36 with keysym 36 (keycode f).
Mar 20 21:32:50 ubuntu-linux-22-04-desktop gnome-shell[2189]: Window manager warning: Overwriting existing binding of keysym 37 with keysym 37 (keycode 10).
Mar 20 21:32:50 ubuntu-linux-22-04-desktop gnome-shell[2189]: Window manager warning: Overwriting existing binding of keysym 31 with keysym 31 (keycode a).
Mar 20 21:32:50 ubuntu-linux-22-04-desktop gnome-shell[2189]: Window manager warning: Overwriting existing binding of keysym 38 with keysym 38 (keycode 11).
Mar 20 21:32:50 ubuntu-linux-22-04-desktop dbus-daemon[2043]: [session uid=1000 pid=2043] Successfully activated service 'org.gnome.ArchiveManager1'
```

Practice Exams

Access the Practice Exams from the Google Classroom



Hardware/storage — More Key Topics

High Latency Issues

High latency refers to delays or lag in processing or transmitting data, which can make computing tasks take longer than usual. ...

High Latency Overview

High latency is often caused by I/O wait time, which measures how long a CPU waits for disk I/O operations to finish. When there ...

Diagnosing and Fixing High Latency

Three utilities to monitor I/O wait times are: top: Provides an overview of system processes, priorities, and wait times.

Low Throughput Overview

Practically speaking, throughput is how much data can be read or transferred across various system devices and the system bus to ...

Diagnosing and Fixing Low Throughput

To monitor throughput on your system, the following utilities are useful: df: Displays disk space usage and availability, showing ...

Input/Output Operations per Second Scenarios

IOPS (Input/Output Operations per Second) is a performance metric used to evaluate storage devices like HDDs, SSDs, and SANs. It ...

Hardware/storage — More Key Topics

IOPS Overview

IOPS used to be a critical metric when storage relied on spinning disks with read/write heads, as the time to find and access ...

Scenario 1: Transferring Ten 500MB Files

In this scenario, reading ten 1MB files from a disk would require 10 IOPS. A typical spinning disk can handle about 20 times that ...

Scenario 2: Transferring 5,000 1MB Files

In this scenario, reading 5,000 1MB files would require 5,000 IOPS. A typical spinning disk can handle around 200 IOPS, so it ...

Why IOPS Are Effectively Irrelevant

IOPS are only relevant for spinning disks, as flash-based storage doesn't involve seek time. Additionally, using IOPS as a ...

Capacity Issues

Capacity issues can arise from various factors, some of which may be unfamiliar or difficult to understand for those ...

Diagnosing and Fixing Capacity Issues

This section focuses on two common capacity issues in Linux: low disk space and inode exhaustion. For a comprehensive view of ...

Lab 26 — Troubleshooting Hardware, Storage, and OS Issues

Deliberately inject common OS and storage failures then diagnose and repair each, keeping every breakage contained to loopback devices, throwaway units, or ...

You'll build — Reproduce a full filesystem, inode exhaustion, GRUB validation, a runaway CPU process, a failed systemd unit, a broken ...

Key commands — ENOSPC / df -h, inode exhaustion / df -i, grub-mkconfig validation, runaway process / top -bn1, failed systemd unit, ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-26-*/README.md on the course GitHub repo

Troubleshooting Hardware, Storage, and OS Issues

Step 1 — Break: fill a filesystem (ENOSPC)

```
$ mkdir -p /mnt/tinyfs
```

Step 2 — Break: inode exhaustion

```
$ df -i /mnt/tinyfs
```

Step 3 — Validate a GRUB config without committing it

```
$ grub-mkconfig -o /tmp/grub.cfg.new 2>&1 | tail -n 5
```

Step 4 — Break: runaway CPU process

```
$ bash -c 'while ;; do ;; done' &
```

Troubleshooting Hardware, Storage, and OS Issues

Step 5 — Break: a failing systemd unit

```
$ [Unit]
```

Step 6 — Break and recover a broken PATH

```
$ ( export PATH=/nonexistent
```

Step 7 — Simulate a memory leak

```
$ data = []
```

✓ **Verify** — Each fault is confirmed by the right counter (df, df -i, top, systemctl status, ps rss) and reversed cleanly ...

5.3 — Networking

Resolve firewall, routing, DNS, and connectivity problems.

Exam objective — Given a scenario, troubleshoot networking issues

Hands-on — Lab 27

Type it, break it, fix it — the exam rewards hands that have done the work.

Diagnosing Subnets with the ip Command

The `ip` command is sufficient to display most network configuration details, including IP addresses, subnets, and gateway information. It provides similar ...

When the `ip` command displays its information, you have to parse the output to see what you need, so I recommend using the `grep` command to filter the ...

In this situation, the subnet mask of /24 (255.255.255.0) indicates a Class C network where the last octet is used for host addresses. A /24 subnet allows ...

```
$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue state
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
    valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pf
    state UP qlen 1000
    link/ether 08:00:27:08:ea:ff brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.22/24 brd 192.168.1.255 scope global eth0
    inet6 fe80::a00:27ff:fe08:eaff/64 scope link
    valid_lft forever preferred_lft forever
```

Diagnosing Routes Using the ip route Command

The `ip route` command focuses on the route portion of your `ip` command and shows you the information that can be pulled from the `/proc` and probably `/sys` ...

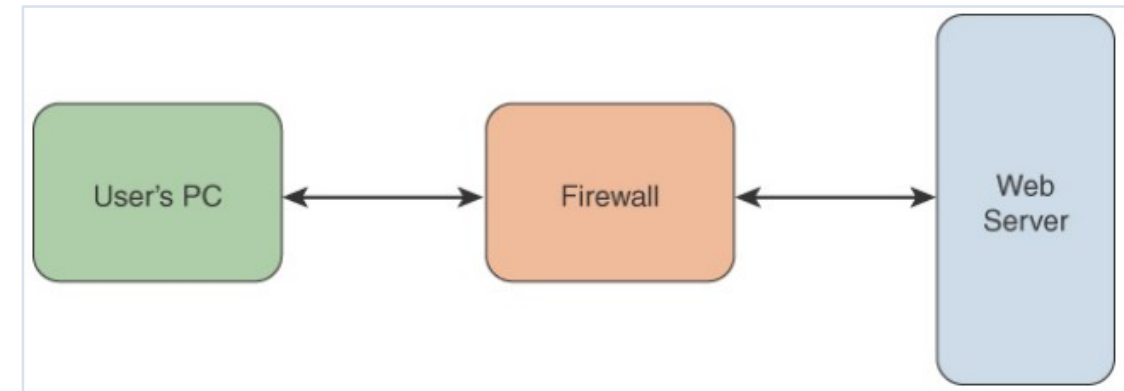
The output of the `ip route show` command provides key details for a properly configured route, starting with the default gateway (e.g., default via ...)

```
$ ip route show
default via 192.168.64.1 dev enp0s2 proto dhcp src 192.168.64.100
192.168.64.0/24 dev enp0s2 proto kernel scope link src 192.168.64.100
192.168.64.1 dev enp0s2 proto dhcp scope link src 192.168.64.100
```

Firewall Refresher

A firewall protects a host from harmful traffic, but the challenge is distinguishing between unwanted traffic and legitimate requests that look similar. ...

For a user's computer to access a web server through a firewall, exceptions must be added to the firewall rules, allowing traffic on ports 80 (HTTP) and ...



Causes and Symptoms of Firewall Issues

The following are possible causes of firewall issues

Service not running

Incorrect ports being listened on

Incorrect ports for the traffic being allowed or denied

Nonstandard ports being used

Symptoms of firewall issues might include any of the following

Diagnosing and Fixing Firewall Issues

Here are a few utilities that can help diagnose firewall issues:

ping: Use to check connectivity with remote hosts, network devices, or gateways.

telnet: Though deprecated, it can still be useful for testing basic connectivity (checking if a port is open).

nmap: Scans IP addresses and ports to detect open ports and services running on a network

To troubleshoot firewall issues, consider the following techniques:

Temporarily disable the firewall: This can help determine if the firewall is causing the issue.

Diagnosing with the telnet Command

Telnet can diagnose firewall issues by testing network connectivity and checking if specific ports are open or blocked.

By providing a port number, telnet attempts to establish a connection to the remote system's application on that port.

If the connection fails, it may indicate a firewall problem.

Note: Telnet should never be used for remote command-line sessions due to security concerns. It's only useful for quick checks on remote port responses.

SSH should always be used for secure connections involving sensitive data, instead of Telnet and R-suite utilities.

Telnet is often used initially to test if a system responds on a specific port, such as checking for a web server on port 80.

Making Sure Services Are Running

Ensuring that services are running is crucial for diagnosing firewall issues because sometimes the problem isn't the firewall, but rather that the service ...

Service is running and accepting connections

It's important to check log files to confirm proper reporting

Example: Telnet command with a 400 error should appear in logs

The 400 error message from NGINX appears in the access.log file instead of the error.log file. Despite this, it confirms that the service is running, ...

```
ubuntu@primary:~$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2022-12-22 20:34:05 MST; 24h ago
     Docs: man:nginx(8)
  Main PID: 5452 (nginx)
    Tasks: 2 (limit: 1132)
   Memory: 4.8M
    CGroup: /system.slice/nginx.service
            └─5452 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
               └─5693 nginx: worker process

Dec 22 20:34:05 primary systemd[1]: Starting A high performance web server and a reverse p
Dec 22 20:34:05 primary systemd[1]: Started A high performance web server and a reverse pr
```

Diagnosing with the ethtool Command

The **ethtool** command with the **-S** option provides a user-friendly way to view detailed network interface statistics, offering an easier alternative to ...

The **ethtool -S <interfacename>** command allows you to view detailed statistics of the network interface card (NIC). You can also filter these stats using ...

The **grep** command can be used to filter for specific text, such as "drop," in the output of network statistics. In this case, the output shows that there ...

```
$ ethtool -S enp0s2 | grep -i 'drop'
rx_queue_0_drops: 0
rx_queue_0_xdp_drops: 0
tx_queue_0_xdp_tx_drops: 0
```

Diagnosing with the netstat Command

You can also run the `netstat` command to identify drops and collisions

The kernel interface table provides valuable information about network interfaces, such as `enp0s2` (equivalent to `eth0` on an Ubuntu VM) and `lo` (localhost). ...

```
$ netstat -i
Kernel Interface table
Iface    MTU     RX-OK   RX-ERR  RX-DRP  RX-OVR   TX-OK   TX-ER
TX-OVR Flg
enp0s2   1500    233716   0        0        0      80587   0
0        BMRU
lo       65536   5234     0        0        0      5234    0
0        LRU0
```

Layer 1: Physical

At the physical layer, you're verifying if the cables or wires are present and transmitting signals correctly. The primary tool for checking this is the ...

To check if a network interface is up and connected, look for the "mode" field in the ip link output, where "UP" indicates the interface is active. ...

The ethtool output provides various settings for an interface, but when focusing on link information, you can use grep to filter out the relevant details, ...

```
$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state
mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
state UP mode DEFAULT group default qlen 1000
    link/ether ca:3b:cc:0e:10:69 brd ff:ff:ff:ff:ff:ff
```

Layer 2: Data Link

At this layer, the focus is on determining if the device can see other devices on the network. If it cannot, it might be due to a VLAN issue or being ...

To check the ARP table, you can use the `arp` command or the `ip neighbor show` command to query the devices on the local network and verify connectivity.

This output shows that there is a host that is reachable by the example host, and given the IP address, it's probably a gateway (and in this case it is).

As suspected, this is a gateway, and it's known that way to the ARP table and `arp` command. You can have it display the IP address instead of `_gateway` in ...

```
$ ip neighbor show  
192.168.64.1 dev enp0s2 lladdr 3e:22:fb:91:6a:64 REACHABLE
```


Running a Vulnerability Scan

In this section, after identifying open ports and running services, the next step is to perform a vulnerability scan on the target host. This scan checks ...

A vulnerability scan with NMAP not only identifies existing vulnerabilities but also provides links to resources explaining how to fix them. It's a ...

```
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect res
https://nmap.org/submit/ .
Nmap done: 2 IP addresses (1 host up) scanned in 10.54 seconds
```

Opening an SSL Connection to a Host

To open an SSL connection to a host and get a lot of useful information back.

You can parse through the extensive output from the `s_client` query to observe the response. You can either use the available commands to interact with the ...

```
$ openssl s_client -connect example.com:443
```

Showing SSL Certificates on a Host

To show the entire certificate chain for a given host.

You'll see the certificate information listed; just parse through it to find out more about the host and its certificate(s).

```
$ openssl s_client -connect example.com:443 -showcerts
```

Testing a Particular TLS Version on a Host

To check if a specific TLS version is available on a host, you can use the relevant command to query that information directly.

The output confirms that the correct TLS version information is displayed in the SSL-Session block, verifying the availability of the desired TLS version.

```
$ openssl s_client -connect example.com:443 -tls1_3
SSL-Session:
    Protocol  : TLSv1.3
    Cipher    : TLS_AES_256_GCM_SHA384
    Session-ID: 84BF20BB5699F250177C680D4A074B9DB7E8976099C
67A996D64
    Session-ID-ctx:
    Resumption PSK: 13005B8986AB6F6B74E89CE87B9497FBA202411
4A58E9D83
```

Networking — More Key Topics

Network Configuration Issues

Linux offers auto-configuration tools to help connect network devices to Ethernet or Wi-Fi. The main causes of network ...

Causes and Symptoms of Network Configuration Issues

The most likely causes of network configuration issues are Subnet issues

Diagnosing and Fixing Network Configuration Issues

The ip command is the primary tool for viewing your network configuration. It's versatile and offers various subfunctions, with ...

Diagnosing Routes Using the ping Command

You will sometimes need to troubleshoot the route that your system takes to reach a remote network location. One method of doing ...

Diagnosing Routes Using the traceroute Command

Traceroute is a reliable diagnostic tool, though its effectiveness has diminished on the wide area network (WAN) or Internet due ...

Firewall Issues

This section offers a quick refresher on firewalls, focusing mainly on how misconfigurations can impact the traffic allowed to ...

Networking — More Key Topics

What Could Possibly Go Wrong?

While firewall configuration issues are complex, for the Linux+ exam, it's important to understand key problems that can arise, ...

Diagnosing with the ping Command

The ping command is essential in Linux for network troubleshooting. It sends packets to a specific IP address and waits for a ...

Interface Errors

An interface error is a problem that occurs on a network interface, which can be a physical port or a virtual interface. These ...

Dropped Packets and Collisions

Dropped packets occur when data fails to transmit between devices on a network, while network collisions happen when multiple ...

Diagnosing and Fixing Issues Related to Dropped ...

To diagnose and resolve dropped packet and collision issues, you can use these utilities: ethtool: A versatile command that ...

Diagnosing with the /sys/class/net//statistics ...

The /sys/net/class/<device>/statistics directory contains more detailed and less user-friendly data on dropped packets and ...

Lab 27 — Troubleshooting Network Connectivity

Break and repair the common network faults (DNS, routing, firewall, MTU, IP, link, dual-stack) while practicing the standard L3/L4 triage toolkit.

You'll build — Baseline the host, then inject DNS, gateway, firewall, MTU, duplicate-IP, and link-down faults, and finish with ...

Key commands — ip addr / ip route baseline, resolv.conf / dig, default gateway, ufw port block, MTU / ping -M do, duplicate IP / arping

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-27-*/README.md on the course GitHub repo

Troubleshooting Network Connectivity

Step 1 — Baseline and install tools

```
$ apt update
```

Step 2 — Break: DNS, then fix

```
$ cp /etc/resolv.conf /tmp/resolv.conf.bak
```

Step 3 — Break: default gateway, then revert

```
$ ORIG=$(ip route show default)
```

Step 4 — Break: firewall blocks a port

```
$ ufw --force enable
```


Troubleshooting Network Connectivity

Step 5 — Break: MTU mismatch

```
$ IF=$(ip -br link | awk '$1!="lo" && $2=="UP" {print $1; exit}')
```

Step 6 — Break: duplicate IP and link down

```
$ IF=$(ip -br link | awk '$1!="lo" && $2=="UP" {print $1; exit}')
```

Step 7 — IPv4 vs IPv6 and capture

```
$ curl -4 -s --max-time 3 https://ifconfig.co || true
```

✓ **Verify** — Each single-variable fault produces its expected symptom and reverts cleanly, and IPv4/IPv6 plus capture ...

5.4 — Security

Fix SELinux, permission, and vulnerability issues.

Exam objective — Given a scenario, troubleshoot security issues

Hands-on — Lab 28

Type it, break it, fix it — the exam rewards hands that have done the work.

ID Please

The `id` command displays a wealth of information about any user.

Key values to check for a user's account.

`uid` (User ID): The user's ID, as configured in the `/etc/passwd` or other name services database.

`gid` (Primary Group ID): The group ID used as the Group Owner for files or directories created by this account.

`groups` (Secondary Groups): Lists the groups the user is a member of, apart from the primary group.

What you're looking for here is anything out of the ordinary, and especially if you don't recall making any changes.

```
$ id
uid=1000(rossb) gid=1000(rossb) groups=1000(rossb),4(adm),20
24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46
17(netdev),118(lxd)
```

Get Entity

The **getent** command is useful for querying databases like **/etc/passwd** or **/etc/shadow** to return information about user accounts and other system data.

The **getent** command querying the **/etc/passwd** file for the "rossb" account yields the same information as using the **grep** command to search for that user in ...

The second query to the **/etc/shadow** file requires root privileges, as it is not publicly accessible. Running it without **sudo** would result in a blank ...

In the **/etc/shadow** file, the presence of an exclamation mark (!) in the password column indicates that the user has no password set, which could be the ...

```
$ getent passwd rossb
rossb:x:1000:1000:RossB:/home/rossb:/bin/bash
$ sudo getent shadow rossb
rossb!:19051:0:99999:7:::
```

The lastlog Command

The lastlog command provides information about users' login status, including the date and time of their last login, the port they logged in from, and ...

This is very useful, but there is a trick you can pull using the grep command that will allow you to determine only who has logged in, screening out all ...

The lastlog command provides valuable insights by allowing you to easily identify which user accounts have successfully logged in, and helps pinpoint ...

```
landscape                **Never logged
pollinate                **Never logged
systemd-coredump         **Never logged i
ubuntu                  pts/0      192.168.64.1    Sun Feb 12 20:31:
2023
lxd                      **Never logged
dnsmasq                 **Never logged i
```

Group Issues

Group issues can be more challenging to troubleshoot than user login problems, as they can impact multiple users in various ways. One common issue is ...

To find out which groups a user is a member of, you can use the `groups` or `id` command with the username as an argument.

To check if the proper group has access to a resource, inspect the `ls -l` output (or `ls -ld` for directories) and ensure the group has at least read (r) ...

```
$ groups rossb
rossb : rossb adm dialout cdrom floppy sudo audio dip video
netdev
```

Attribute Issues

File attributes are special settings on filesystem objects, like "append only," which prevents overwriting, or "noatime," which stops access times from ...

The three times—Access (atime), Modify (mtime), and Change (ctime)—appear at the bottom of the output. To view special attributes on a filesystem object, ...

The file le1 has attributes like "do not dump," "no atime updates," and "extent format" set. Other notable attributes include:

s (Secure Updates): Ensures secure file deletion by overwriting the file's disk resources.

d (No Dump): Prevents the file from being backed up during the dump command.

i (Immutable): Makes the file unchangeable unless set back to mutable.

```
$ stat file1
  File: file1
  Size: 0                Blocks: 0                IO Block: 4096   r
Device: fc01h/64513d    Inode: 258226         Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  ubuntu)   Gid: ( 1000/  ubuntu)
Access: 2022-06-01 23:15:35.864502122 -0600
Modify: 2022-06-01 23:15:35.864502122 -0600
Change: 2022-06-01 23:29:41.424625649 -0600
 Birth: -
```

Quota Issues

Quotas are largely outdated today. They were crucial when disks were smaller, more expensive, and shared by large numbers of users, such as in colleges ...

The main issue with quotas isn't how the system works, but users' understanding of it. Hard and soft limits often cause confusion, as they are not clearly ...

The output of the `edquota ross` command, which we'll use to try to make more sense of what is going on.

The first column shows the filesystem with quotas enabled. The second column displays the number of disk blocks the user is using, while the third and ...

```
$ sudo edquota rossb
Disk quotas for user rossb (uid 505):
```


Security — More Key Topics

User Login Issues

Chapter 9, "Implementing Identity Management," covers the user login process, user and group management, and common login issues. ...

Inspecting Account Details

To troubleshoot a user's login issues, start by checking the user's account details. Run the commands `id`, `getent`, and `last` to ...

Case-Sensitivity

Case sensitivity in Linux is crucial; even a single change in uppercase or lowercase characters can significantly affect system ...

Has the User Ever Logged In?

Knowing whether a user has logged in can be crucial for troubleshooting login issues, especially when the user is new or ...

The last Command

The `last` command provides a reverse chronological list of system activity, showing who signed in, from which port, and what they ...

User File Access Issues

As a system administrator, you'll need to troubleshoot file access issues for users, which can be caused by factors like group ...

Security — More Key Topics

Context Issues

Context issues related to AppArmor or SELinux can be challenging to troubleshoot. For more details on how these security context ...

Permission Issues

Permissions in Linux are simpler than in Windows, which often leads to confusion. While Windows applies and manages permissions ...

ACL Issues

For a detailed explanation of access control list (ACL) issues, including how ACLs work, how they are used, and how to ...

Policy/Non-Policy Issue

The Linux+ objectives are vague about specific user file access issues, but "silent access issues" are a common challenge, often ...

Password Issues

Aside from the common issue of users forgetting their passwords, two other password-related problems are common. One is when a ...

The faillog Command

The faillog command is highly recommended for daily use, and it can be configured to send its output via email. It can be run by ...

Lab 28 — Troubleshooting Security Issues

Diagnose and repair security faults spanning SELinux contexts, POSIX ACLs, certificate trust, account lockout, broken repos, and weak TLS, using a Rocky container ...

You'll build — Pull a Rocky container for SELinux practice, then break and fix ACLs, CA trust, pam_faillock lockout, a bad apt repo, ...

Key commands — SELinux context / chcon / restorecon, AVC denials / ausearch / audit2allow, POSIX ACL / getfacl / setfacl, CA trust / ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-28-*/README.md on the course GitHub repo

Troubleshooting Security Issues

Step 1 — Prep host and pull a Rocky container

```
$ apt update
```

Step 2 — Break: SELinux context, then restorecon

```
$ docker exec selab bash -c '
```

Step 3 — Inspect AVC denials with audit2allow

```
$ docker exec selab bash -c '
```

Step 4 — Break: ACL on Ubuntu host

```
$ useradd -m alice 2>/dev/null || true
```

Troubleshooting Security Issues

Step 5 — Break: certificate trust

```
$ mkdir -p /tmp/ca && cd /tmp/ca
```

Step 6 — Break: account lockout with pam_faillock

```
$ auth required pam_faillock.so preauth deny=3 unlock_time=120
```

Step 7 — Break: misconfigured apt repo and weak TLS

```
$ echo 'deb http://no.such.host.invalid/ubuntu jammy main' > /etc/apt/sources.list
```

✅ **Verify** — restorecon fixes SELinux type, getfacl/setfacl explains hidden ACL denials, the CA trust bundle rebuilds, ...

5.5 — Performance

Analyze CPU, memory, I/O, and optimize response times.

Exam objective — Given a scenario, troubleshoot performance issues

Hands-on — Lab 29

Type it, break it, fix it — the exam rewards hands that have done the work.

Removing Zombie Processes

A zombie process can be removed by sending a particular signal to the parent process, such as shown

When the SIGCHLD signal is sent to the parent process, it prompts the parent to "reap" or clean up any zombie child processes, which may or may not ...

```
$ kill -s SIGCHLD 12345
```

Viewing System Load Details with the uptime Command

One of the easiest ways to get the load numbers is to run the **uptime** command, which will bring back a series of numbers and other information such as that ...

The uptime command output shows:

Current system time

System uptime (e.g., 13 minutes)

Number of users

Three load averages:

```
$ uptime
16:28:23 up 13 min,  1 user,  load average: 0.47, 0.15, 0.08
```


High Run Queues

High run queue occurs when there are too many processes waiting to be executed by the CPU.

Causes of high run queues:

Too many processes running, overwhelming the CPU.

I/O bottlenecks preventing the kernel from scheduling new processes.

Memory pressure when there isn't enough available memory for new processes.

CPU bottlenecks slowing down process scheduling.

Viewing System Load Details with the tload Command

The **tload** command offers a graphical, text-based display of system load, but initially shows less information than the **uptime** command. It doesn't display ...

To get useful output from **tload**, use the **-s** option to display a side-scrolling graph that visually represents the system load, with asterisks indicating ...

When using **tload** with the **-s** option, a series of asterisks will appear at the bottom left of the screen and move from left to right, representing the ...

The user's computer needs to access a web server, but a firewall is blocking the request by default. The **tload** command shows system load at specified ...

```
$ tload -s 100  
0.04, 0.03, 0.03
```

Measuring CPU Time

To measure CPU utilization, you can use commands like `top` (covered elsewhere) and `time`. The `time` command is particularly useful as it measures how long a ...

The `time` command measures how long a command takes to run in real time, showing how much of that time is spent in user space versus system (kernel) space. ...

```
$ time find / -iname *.doc 2> /dev/null
/snap/gnome-3-34-1804/77/usr/lib/python2.7/pdb.doc
/snap/gnome-3-34-1804/72/usr/lib/python2.7/pdb.doc

real    0m6.997s
user    0m0.676s
sys     0m1.800s
```

Memory Exhaustion and Out of Memory

Memory exhaustion occurs when a system runs out of both physical RAM and swap space. Swap space, which can be a swap file or a dedicated partition, acts ...

Memory exhaustion occurs when a system continuously runs out of available memory, even with ample swap space. Eventually, the system becomes overwhelmed, ...

Memory exhaustion typically causes the system to become sluggish and unresponsive before any error messages appear. Users may experience interface and ...

A Linux system uses a combination of memory, swap (on disk or filesystem), and physical RAM, which can be confusing at times. The upcoming section on ...

The file cache is a memory area used to store recently accessed files, allowing the kernel to access them more quickly without reading from disk each ...

```
Allowed memory size of xxxxxxxx bytes exhausted (tried to al
bytes)
Allowed memory size of xxxxxxxx bytes exhausted (tried to al
bytes)
Allowed memory size of xxxxxxxx bytes exhausted (tried to al
bytes)
Allowed memory size of xxxxxxxx bytes exhausted (tried to al
bytes)
```

Swap, Caching, and Buffers

The `free` command provides detailed information about the system's memory usage, including free and used physical memory, swap space, and the amount of RAM ...

Unused system RAM is allocated for useful purposes like cache and buffers, allowing processes to have fully loaded working sets for efficient operation. ...

```
$ - $: free -h
```

	total	used	free	shared	buff/c
Mem:	3.8Gi	1.0Gi	1.2Gi	158Mi	1
Swap:	2.0Gi	0B	2.0Gi		

How Much Swap Is Enough?

Choosing the best swap size can vary, much like debating which car is better. Over the years, recommendations have changed frequently. Table 20-2 offers a ...

While the table provides helpful guidance, determining the right amount of swap can still be unclear in some cases. The Hibernation column is specifically ...

Amount of RAM Installed in System	Recommended Swap Space	Recommended Swap Space with Hibernation
≤ 2GB	2 × RAM	3 × RAM
2GB – 8GB	= RAM	2 × RAM
8GB – 64GB	4GB to 0.5 × RAM	1.5 × RAM
> 64GB	Minimum 4GB	Hibernation not recommended

Viewing CPU Info

The `lscpu` command retrieves CPU information from `/sys` and `/proc/cpuinfo` and formats it for easier reading. Since it produces extensive output, it's often ...

The `-C` option for the `lscpu` command is particularly useful for displaying information about the various levels of CPU caches. This option provides ...

```
$ lscpu -C
```

NAME	ONE-SIZE	ALL-SIZE	WAYS	TYPE	LEVEL	SETS	PHY-LINE
COHERENCY-SIZE							
L1d	32K	64K	8	Data	1	64	1
L1i	32K	64K	8	Instruction	1	64	1
L2	256K	512K	4	Unified	2	1024	1
L3	16M	16M	16	Unified	3	16384	1

Viewing Memory Info

The `lsmem` command reads system memory information from `/sys` and `/proc/meminfo` and presents it in a more understandable format. While the output is ...

```
$ lsmem
RANGE                                SIZE  STATE REMOVA
0x0000000000000000-0x00000000afffffff 2.8G online      yes
0x00000000100000000-0x0000000014ffffff 1.3G online      yes
Memory block size:                128M
Total online memory:                4G
Total offline memory:              0B
```


Performance — More Key Topics

Runaway Processes

A runaway process is a malfunctioning or potentially malicious process that replicates itself, consuming excessive CPU resources ...

What Causes Runaway Processes?

Typically, a program creates processes that are terminated when the main program exits. However, in Linux, issues can arise with ...

Reserving Some CPU for Non-Realtime Processes

In the /proc directory, two files help reserve CPU time for non-realtime or non-deadline processes ...

Identifying a Runaway Process

A runaway process can make a system unresponsive, making it hard to input commands. To identify such processes, you can use the ...

Ending a Runaway Process

To end a runaway process, you can use the built-in "kill" options in top or htop. Highlight the process and press F9 to send a ...

Zombie Processes

Processes have a parent process (PPID) and often child processes. A "zombie" process, marked with a "Z" in commands like top, ...

Performance — More Key Topics

What Causes Zombie Processes?

A zombie process doesn't handle its termination. Instead, it notifies its parent process, which is responsible for removing its ...

Are Zombie Processes Bad?

Zombie processes are generally harmless in small numbers, as they only take up minimal memory. However, if there are many zombie ...

High CPU Utilization/Load Average/Run Queues

CPU performance is crucial for system usability. Monitoring CPU utilization helps identify overuse, which could prompt moving ...

High CPU Utilization

High CPU utilization can be caused by several factors, such as resource-heavy processes, poorly written programs, memory leaks, ...

High Load Average

The load average represents the amount of CPU utilization on a Linux system, reflecting the number of processes currently being ...

CPU Times and CPU Process Priorities

CPU time refers to the actual time a task spends using the CPU, while elapsed time is the total time from the start to the end of ...

Lab 29 — Troubleshooting Performance Issues

Stress CPU, memory, disk I/O, and network in turn, then walk the universal performance loop: baseline, spike, identify the bottleneck with the right counter, and ...

You'll build — Install stress-ng, fio, and sysstat, baseline the host, then generate controlled CPU, memory, disk, and network spikes ...

Key commands — baseline: uptime/free/vmstat/mpstat, stress-ng CPU spike / load average, memory pressure / vmstat si-so, fio disk storm ...

Workbench — Killercoda Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-29-*/README.md on the course GitHub repo

Troubleshooting Performance Issues

Step 1 — Install the toolkit and baseline

```
$ apt update
```

Step 2 — Generate and observe a CPU spike

```
$ stress-ng --cpu 4 --timeout 30 &
```

Step 3 — Memory pressure and swap behavior

```
$ free -m
```

Step 4 — Disk I/O storm

```
$ fio --name=lab --rw=randwrite --bs=4k --size=128M --runtime=20 --time_based \
```

Troubleshooting Performance Issues

Step 5 — Network jitter and blocked processes

```
$ ping -i 0.2 -c 20 8.8.8.8 | tail -n 5
```

Step 6 — Context switches and interactive tools

```
$ pidstat -w 1 5
```

Step 7 — Walk the full loop on a synthetic incident

```
$ echo "BASELINE:"; uptime; free -m | head -n 2
```

✓ **Verify** — Each spike maps to its diagnostic counter (load vs mpstat, si/so vs free, %util vs await) and the triage ...

Lab 30 — Capstone: End-to-End Server Build and Triage

Integrate Objectives 1-5 into one coherent build: a hardened admin account, LVM storage, nginx with a systemd override, a locked firewall, a health-check timer, and ...

You'll build — Provision webops with hardened SSH, build LVM-backed /srv/web, deploy nginx, lock ufw to web ports, add a health-check ...

Key commands — hardened SSH / narrow sudo, LVM: pvcreate/vgcreate/lvcreate, systemd drop-in override, ufw default-deny allowlist, ...

Workbench — Killercode Ubuntu Playground, free in the browser

Full walkthrough — labs/lab-30-*/README.md on the course GitHub repo

Capstone: End-to-End Server Build and Triage

Step 1 — Provision a user and harden SSH

```
$ apt update
```

Step 2 — Build LVM-backed /srv/web over loopback

```
$ dd if=/dev/zero of=/var/lab-pv.img bs=1M count=200
```

Step 3 — Install nginx with a systemd override binding /srv/web

```
$ rm -f /etc/nginx/sites-enabled/default
```

Step 4 — Firewall: allow only 80/443 and SSH

```
$ ufw --force reset >/dev/null
```

Capstone: End-to-End Server Build and Triage

Step 5 — Health-check script with a systemd timer

```
$ set -euo pipefail
```

Step 6 — Commit /etc/nginx to a local git repo

```
$ cd /etc/nginx
```

Step 7 — Inject a fault and triage

```
$ systemctl stop nginx
```

✓ **Verify** — After remediation curl returns the Capstone page with a 200, confirming the full build and the ...

Recap — Troubleshooting

Lab 25 — Monitoring Concepts and Tools

Lab 26 — Troubleshooting Hardware, Storage, and OS Issues

Lab 27 — Troubleshooting Network Connectivity

Lab 28 — Troubleshooting Security Issues

Lab 29 — Troubleshooting Performance Issues

Lab 30 — Capstone: End-to-End Server Build and Triage

Wrap-Up & Certification

Summary | exam registration | practice exam | next steps

Type it, break it, fix it — the exam rewards hands that have done the work.

Learning Outcomes — Revisited

LO1 — System Management

Boot & FHS, kernel modules, LVM storage, networking, shell, backup, virtualization.

LO2 — Services & User Management

Files, accounts, processes, packages, systemd services, containers.

LO3 — Security

AAA/sudo/PAM, firewalls, OS & account hardening, cryptography, compliance & audit.

LO4 — Automation & Scripting

Ansible IaC, Bash, Python, Git version control, responsible AI use.

LO5 — Troubleshooting

Monitoring, and storage / network / security / performance triage.

Preparing for the Linux+ XK0-006 Exam

Redo every lab on Killercode until the commands are automatic.

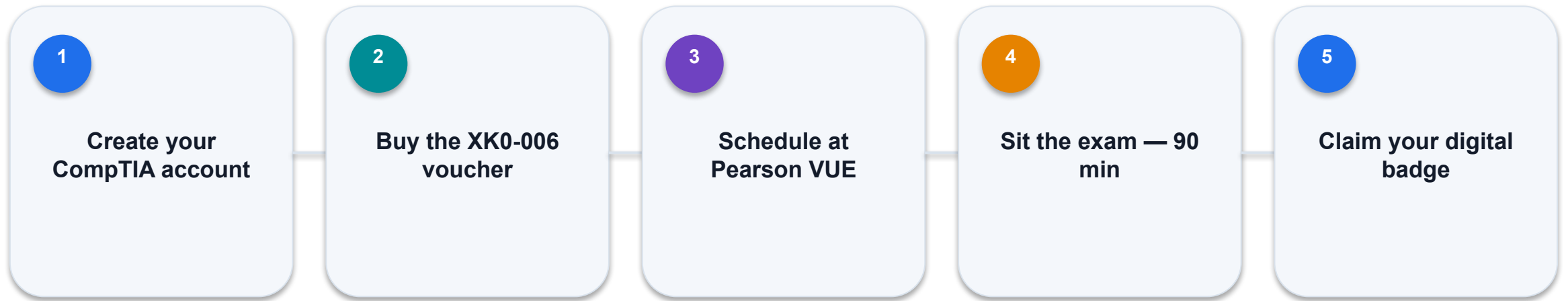
Review each lab's Verify takeaway and the Learner Guide.

Know which tool solves which problem — the exam is scenario-based.

Up to 90 questions (multiple-choice + performance-based) in 90 minutes; pass 720/900.

Book through Pearson VUE from your CompTIA account.

CompTIA Linux+ Exam Registration



Voucher: <https://store.comptia.org> · Schedule: <https://home.pearsonvue.com/comptia>

Exam Registration — Links & Details

Exam voucher

CompTIA Store: <https://store.comptia.org> (or through Tertiary Infotech)

Schedule the exam

Pearson VUE: <https://home.pearsonvue.com/comptia> — test centre or online proctored

Format

XK0-006 · up to 90 questions · 90 minutes · passing score 720 (scale 100–900)

Question types

Multiple-choice + performance-based items (live Linux tasks)

On exam day

Two forms of ID at a test centre · check online-testing system requirements for remote

Practice Exam

Sharpen your readiness

Attempt the Tertiary Infotech CompTIA Linux+ practice exam under timed conditions.

Practice exam link

<https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-linux-plus>

Review every explanation

Wrong answers are the syllabus for your next study session.

Close the gaps

Revisit any lab whose domain you miss, then re-take the practice exam.

Course Feedback

Your feedback shapes this course — please complete the feedback form before you leave.

Scan the feedback QR code displayed by the trainer, or use the link emailed to you.

Tell us what worked and what did not — be specific about labs, pace and depth.

Your certificate of completion will be emailed to you after the course.

THANK YOU

You are ready to administer Linux.

“

Thank you — and all the best for the CompTIA Linux+ XK0-006 exam. Register for the exam, take the practice test, and keep every lab within arm's reach: <https://github.com/tertiarycourses/C1162-CompTIA-LinuxPlus-Training>

SEE YOU AT THE EXAM